

# Training On 'C' Language



**HCL Perot Systems**  
B - 31, Sector 57  
NOIDA - 201301, India

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. INTRODUCTION TO C .....</b>                       | <b>4</b>  |
| 1.1 HISTORY OF C .....                                  | 4         |
| 1.2 CHARACTERISTICS OF C .....                          | 5         |
| <i>Structure of a C Program</i> .....                   | 6         |
| <i>The general format of a C Program</i> .....          | 6         |
| <i>Comments</i> .....                                   | 6         |
| <i>Preprocessor directives</i> .....                    | 7         |
| <i>Function Prototypes</i> .....                        | 7         |
| <b>2. STORING DATA, CONSTANT &amp; VARIABLE .....</b>   | <b>8</b>  |
| 1.3 COMPUTER MEMORY .....                               | 8         |
| 1.4 VARIABLES .....                                     | 9         |
| <i>Numeric Variable Types</i> .....                     | 10        |
| <i>Variable Declarations</i> .....                      | 11        |
| <i>The typedef Keyword</i> .....                        | 12        |
| <i>Initializing Numeric Variables</i> .....             | 12        |
| 1.5 CONSTANTS .....                                     | 13        |
| <i>Symbolic Constants</i> .....                         | 13        |
| <i>Defining Constants with the const Keyword</i> .....  | 15        |
| <b>3. STATEMENTS, EXPRESSIONS AND OPERATORS .....</b>   | <b>17</b> |
| 1.6 STATEMENTS .....                                    | 17        |
| <i>Statements and White Space</i> .....                 | 17        |
| <i>Null Statements</i> .....                            | 18        |
| <i>Compound Statements</i> .....                        | 18        |
| 1.7 EXPRESSIONS .....                                   | 18        |
| <i>Simple Expressions</i> .....                         | 18        |
| <i>Complex Expressions</i> .....                        | 19        |
| 1.8 OPERATORS .....                                     | 19        |
| <i>The Assignment Operator</i> .....                    | 19        |
| <i>Mathematical Operators</i> .....                     | 20        |
| <i>Unary Mathematical Operators</i> .....               | 20        |
| <i>Binary Mathematical Operators</i> .....              | 21        |
| <i>Operator Precedence and Parentheses</i> .....        | 22        |
| <i>Relational Operators</i> .....                       | 24        |
| <i>Logical Operators</i> .....                          | 25        |
| <i>The Precedence of Operators</i> .....                | 26        |
| <i>Logical operator precedence</i> .....                | 26        |
| <i>Compound Assignment Operators</i> .....              | 27        |
| <i>The Comma Operator</i> .....                         | 27        |
| <i>Operator Precedence Revisited</i> .....              | 28        |
| <i>The If Statement</i> .....                           | 28        |
| <i>Loops</i> .....                                      | 30        |
| <i>switch case</i> .....                                | 31        |
| <b>4. FUNCTIONS .....</b>                               | <b>33</b> |
| 1.9 A BRIEF OVERVIEW .....                              | 33        |
| 1.10 DEFINING A FUNCTION .....                          | 34        |
| 1.11 ACCESSING A FUNCTION .....                         | 35        |
| 1.12 PASSING ARGUMENTS TO A FUNCTION .....              | 38        |
| 1.13 FUNCTION PROTOTYPES .....                          | 39        |
| 1.14 RECURSION .....                                    | 40        |
| <b>5. ARRAYS .....</b>                                  | <b>43</b> |
| 1.15 ARRAY INITIALIZATION .....                         | 45        |
| 1.16 ARRAYS OF ARRAYS ("MULTIDIMENSIONAL" ARRAYS) ..... | 46        |
| <b>6. PROGRAM STRUCTURE .....</b>                       | <b>48</b> |

|           |                                                        |            |
|-----------|--------------------------------------------------------|------------|
| 1.17      | DEFAULT INITIALIZATION .....                           | 49         |
| 1.18      | EXAMPLES .....                                         | 50         |
| <b>7.</b> | <b>POINTER .....</b>                                   | <b>52</b>  |
| 1.19      | WHAT IS A POINTER? .....                               | 52         |
| 1.20      | POINTER TYPES AND ARRAYS .....                         | 55         |
| 1.21      | POINTERS AND STRINGS .....                             | 58         |
| 1.22      | MORE ON STRINGS .....                                  | 62         |
| <b>8.</b> | <b>STRUCTURES IN C .....</b>                           | <b>64</b>  |
| 1.23      | SIMPLE STRUCTURES .....                                | 64         |
| 1.24      | DEFINING AND DECLARING STRUCTURES .....                | 64         |
| 1.25      | STRUCTURES THAT CONTAIN ARRAYS .....                   | 68         |
|           | Arrays of Structures .....                             | 70         |
| 1.26      | INITIALIZING STRUCTURES .....                          | 72         |
| 1.27      | STRUCTURES AND POINTERS .....                          | 73         |
|           | Pointers as Structure Members .....                    | 73         |
|           | Pointers to Structures .....                           | 75         |
|           | Pointers and Arrays of Structures .....                | 76         |
|           | Passing Structures as Arguments to Functions .....     | 77         |
| 1.28      | UNIONS .....                                           | 79         |
|           | Defining, Declaring, and Initializing Unions .....     | 79         |
|           | The union Keyword .....                                | 80         |
|           | typedef and Structures .....                           | 82         |
| 1.29      | POINTERS TO ARRAYS .....                               | 88         |
|           | Pointers and Dynamic Allocation of Memory .....        | 90         |
| 1.30      | POINTERS TO FUNCTIONS .....                            | 96         |
| <b>9.</b> | <b>STREAMS AND DISK FILES .....</b>                    | <b>106</b> |
| 1.31      | TYPES OF DISK FILES .....                              | 106        |
| 1.32      | FILENAMES .....                                        | 106        |
| 1.33      | OPENING A FILE .....                                   | 107        |
| 1.34      | WRITING AND READING FILE DATA .....                    | 109        |
|           | Formatted File Input and Output .....                  | 110        |
|           | Formatted File Output .....                            | 110        |
|           | Formatted File Input .....                             | 111        |
|           | Character Input and Output .....                       | 112        |
|           | Direct File Input and Output .....                     | 114        |
| 1.35      | FILE BUFFERING: CLOSING AND FLUSHING FILES .....       | 116        |
| 1.36      | SEQUENTIAL VERSUS RANDOM FILE ACCESS .....             | 117        |
|           | The <i>ftell()</i> and <i>rewind()</i> Functions ..... | 118        |
|           | The <i>fseek()</i> Function .....                      | 119        |
| 1.37      | DETECTING THE END OF A FILE .....                      | 121        |
| 1.38      | FILE MANAGEMENT FUNCTIONS .....                        | 123        |
|           | Deleting a File .....                                  | 123        |
|           | Renaming a File .....                                  | 124        |
|           | Copying a File .....                                   | 124        |
| 1.39      | USING TEMPORARY FILES .....                            | 126        |

## 1. Introduction to C

C is a general-purpose, structured programming language. Its instructions consist of terms that resemble algebraic expressions augmented by certain English keywords such as if, else, for do and while.

In this respect C resembles other high-level structured programming languages such as Pascal and FORTRAN-77. C also contains certain additional features, however, that allow it to be used at a lower level, thus bridging the gap between machine languages and the more conventional high-level languages. This flexibility allows C to be used for systems programming (e.g. for writing operating systems ) as well as for applications programming (e.g. for writing a program to solve a complicated system of mathematical equations, or for writing a program to bill customers).

C is characterized by the ability to write very concise source programs, due in part to the large number of operators included within the language. It has a relatively small instruction set, though actual implementations include extensive library functions which enhance the basic instructions. Furthermore, the language encourages users to write additional library functions of their own. Thus, the user can easily extend the features and capabilities of the language.

C compilers are commonly available for computers of all sizes, and C interpreters are becoming increasingly common. The compilers are usually compact, and they generate object programs that are small and highly efficient when compared with programs compiled from other high-level languages.

The interpreters are less efficient, though they are easier to use when developing a new program. Many programmers begin with an interpreter, and then switch to a compiler once the program has been debugged (once all of the programming errors have been removed).

Another important characteristic of C is that its programs are highly portable, even more so than with other high – level languages. The reason for this is that C relegates most computers–dependent features to its library functions. Thus, every version of C is accompanied by its set of library functions, which are written for the particular characteristics of the host computer. These library functions are relatively standardized, however, and each individual library function is generally accessed in the same manner from one version of C to another. Therefore, most C programs can be processed on many different computers with little or no alteration.

### 1.1 History of C

Dennis Ritchie originally developed C in he 1970's at Bell Telephone Laboratories, Inc. (Now AT&T Bell Laboratories) It is an outgrowth of two earlier languages, called BCPL and B, which were also developed at Bell Laboratories. C was largely confined to use within Bell Laboratories until 1978, when Bran Kernighan and Ritchie published a definitive description of the language.\* The Kernighan and Ritchie description is commonly referred to as "K&R C.

Following the publication of the K& R description, professionals, impressed with C's many desirable features, began to promote the use of the language. By the mid 1980's the popularity of C had become widespread. Numerous C compilers and interpreters had been written for computers of all sizes and many commercial application programs has been developed. Moreover, many commercial software products that were originally written in other

languages were rewritten in C in order to take advantages of its efficiency and its portability.

Most commercial implementations of C differ somewhat from Kernighan and Ritchie original definition. This has created some minor incompatibilities among different implementation of the language, thus diminishing the portability that the language attempts to provide. Consequently, the American National Standards Institute (ANSI)\*\* has begun work on a standard definition of the C language (ANSI committee X3J11). At the time of this writing, the ANSI project is nearing completion, though it is still in progress. Most commercial C compilers and interpreters are expected to adopt the ANSI standard once it has been completed (many now follow the partially completed recommendations). They may also provide additional features of their own.

This book describes the features of C that are generally supported by commercial C compilers and interpreters. Most of the features recommended by the currently proposed ANSI standard are included

## 1.2 Characteristics of C

As is widely known, C as a language was designed and developed by a team of working programming, rather than by a group of managers. And therefore C gives the programmer what he wants – few restrictions, a compact set of keywords, stand-alone functions, a vast number of powerful operators and means to create complex data structures. Due to its conciseness, it is easy to learn this language.

The flexibility of the language allows the programmer to optimize a program by using low-level functions. Also, C is not a strongly typed language like Algol-68 or Pascal. It allows a lot of flexibility in data conversion from one type to another. Sometimes this flexibility can lead to bugs creeping in the program and even bad programming style.

Programs written in C are portable across various platforms. Portability is another aspect, which has added to the vast popularity and rapid spread of this language.

Another very important feature of the C language is that it is a structured language, which supports a modular approach to program building. Modular programs written in C are easier to write, debug, comprehend and maintain. Although not strictly block-structured like Pascal, it offers a facility to compartmentalize code and data.

A major difference between C and other high-level languages is that it has no in-built input/output facilities. Input and output is achieved by calling a set of functions.

Although, it has the ability to manipulate bits and address memory directly, which makes it well suited for systems programming, C is considered to be a general-purpose language. It has primarily been used for writing operating systems, compilers, interpreters, assemblers, editors, communication, software, DBMS, spreadsheets and word processors. It is also used for applications like business systems, animation software, engineering and process control systems.

No wonder C is the most preferred language for software development by professional programmers. Because of its power, elegance, flexibility and portability, C will continue to flourish more and more, may be under different garbs like C++.

## *Structure of a C Program*

Functions are the basic building blocks of C programs. All C programs whatever their size, consist of one or more functions which specify the actual computing operations to be done. Every program must have one function with the name main (). This function is the entry point of the program. The function main () invoke other functions to perform its job. The communication between functions takes place through arguments.

A sample C Program

```
/* This is my First C Program*/

#include <stdio.h>

void main ()

{

printf("Hello, world !");

}
```

The sample program has the main () function which is the entry point of the program, the function calls another function printf () from the standard library to do the output function.

## *The general format of a C Program*

A general format of a C program is given below:

Preprocessor directives

Function prototypes

Global variables

```
void main ()

{
    local variable declarations;
    program statements;
}
```

The various element of the program are:

## *Comments*

Comments are non-executable statements used to convey the objective of the program or of some particular statements to the programmer. All comments are enclosed in between /\* \*/ markers. Comments can appear anywhere in the program as long as they are enclosed in the markers. In the sample program the line.

```
/* This is my first C program */
```

is a comment.

## *Preprocessor directives*

A preprocessor directive, which begins with a # (hash), is a direction to the preprocessor. The preprocessor prepares the program file, which is to be handled over to compilers for compilation. It performs jobs like file inclusion, macro substitution and conditional compilation. The second line of the sample program is preprocessor directive, the directive #include is a file inclusion directive, it includes the header file stdio.h into the program file before the file is passed to the compiler. The header file contains function prototype declarations of the functions in the standard library. Preprocessor directive help in realizing the multiple file programs approach thereby keeping the main program file neat and hence easier to understand.

## *Function Prototypes*

An ANSI C standard requires declaring beforehand function prototypes of all functions to be used in a program. The prototypes of functions in the library are declared in various header files. Thus including the header file takes care of library function prototype declarations. The general format of a function prototype declaration is.

Return\_type function\_name (arguments, , , , );

Return\_type is the type of data the function returns not all functions return a value, those which don't return a value are declared as void functions (in such cases return statement is be omitted). The function name is the name by which the function is invoked by other functions.

E.g.

```
void display (void);  
int getchar (void);  
long factorial (int num);  
double cost (int quantity, double rate, float disc);
```

The first example gives the declaration of the display () function which does not take any argument and also does not return any value. The second example gives the declaration of the getchar () function, which does not require any arguments, but returns an integer. The keyword void may be used to specify that a function does not take any arguments. The third is an example of a function which requires an integer and will return a long integer (with some compilers, the variable name num can be omitted). The third example declares a function, which takes three arguments and returns a double.

## 2. Storing Data, Constant & Variable

Computer programs usually work with different types of data and need a way to store the values being used. These values can be numbers or characters. C has two ways of storing number values--variables and constants--with many options for each. A variable is a data storage location that has a value that can change during program execution. In contrast, a constant has a fixed value that can't change. You will learn:

- How to create variable names in C
- The use of different types of numeric variables
- The differences and similarities between character and numeric values
- How to declare and initialize numeric variables
- C's two types of numeric constants

Before you get to variables, however, you need to know a little about the operation of your computer's memory.

### 1.3 Computer Memory

If you already know how a computer's memory operates, you can skip this section. If you're not sure, however, read on. This information will help you better understand certain aspects of C programming.

A computer uses random-access memory (RAM) to store information while it's operating. RAM is located in integrated circuits, or chips, inside your computer. RAM is volatile, which means that it is erased and replaced with new information as often as needed. Being volatile also means that RAM "remembers" only while the computer is turned on and loses its information when you turn the computer off.

Each computer has a certain amount of RAM installed. The amount of RAM in a system is usually specified in kilobytes (KB) or megabytes (MB), such as 512KB, 640KB, 2MB, 4MB, or 8MB. One kilobyte of memory consists of 1,024 bytes. Thus, a system with 640KB of memory actually has  $640 * 1,024$ , or 65,536, bytes of RAM. One megabyte is 1,024 kilobytes. A machine with 4MB of RAM would have 4,096KB or 4,194,304 bytes of RAM.

The *byte* is the fundamental unit of computer data storage. Day 20, "Working with Memory," has more information about bytes. For now, to get an idea of how many bytes it takes to store certain kinds of data, refer to Table 3.1.

**Table 3.1. Memory space required storing data.**

| Data                               | Bytes Required      |
|------------------------------------|---------------------|
| The letter x                       | 1                   |
| The number 500                     | 2                   |
| The number 241.105                 | 4                   |
| The phrase <i>Teach Yourself C</i> | 17                  |
| One typewritten page               | Approximately 3,000 |

The RAM in your computer is organized sequentially, one byte following another. Each byte of memory has a unique address by which it is identified--an address that also distinguishes it from all other bytes in memory. Addresses are assigned to memory locations in order, starting



at zero and increasing to the system limit. For now, you don't need to worry about addresses; the C compiler handles it all automatically.

What is your computer's RAM used for? It has several uses, but only data storage need concern you as a programmer. Data is the information with which your C program works. Whether your program is maintaining an address list, monitoring the stock market, keeping a household budget, or tracking the price of hog bellies, the information (names, stock prices, expense amounts, or hog futures) is kept in your computer's RAM while the program is running.

Now that you understand a little about the nuts and bolts of memory storage, you can get back to C programming and how C uses memory to store information.

## 1.4 Variables

A *variable* is a named data storage location in your computer's memory. By using a variable's name in your program, you are, in effect, referring to the data stored there.

### Variable Names

To use variables in your C programs, you must know how to create variable names. In C, variable names must adhere to the following rules:

- The name can contain letters, digits, and the underscore character (\_).
- The first character of the name must be a letter. The underscore is also a legal first character, but its use is not recommended.
- Case matters (that is, upper- and lowercase letters). Thus, the names count and Count refer to two different variables.
- C keywords can't be used as variable names. A keyword is a word that is part of the C language. (A complete list of 33 C keywords can be found in Appendix B, "Reserved Words.")

The following list contains some examples of legal and illegal C variable names:

| Variable Name | Legality                                  |
|---------------|-------------------------------------------|
| Percent       | Legal                                     |
| Xyz_yud       | Legal                                     |
| Hello_world   | Legal                                     |
| _2001_tax     | Legal but not advised                     |
| Profit#loss   | Illegal: Contains the illegal character # |
| Double        | Illegal: Is a C keyword                   |
| 6summer       | Illegal: First character is a digit       |

Because C is case-sensitive, the names percent, PERCENT, and Percent would be considered three different variables. C programmers commonly use only lowercase letters in variable names, although this isn't required. Using all-uppercase letters is usually reserved for the names of constants (which are covered later in this chapter).

For many compilers, a C variable name can be up to 31 characters long. (It can actually be longer than that, but the compiler looks at only the first 31 characters of the name.) With this flexibility, you can create variable names that reflect the data being stored. For example, a program that calculates loan payments could store the value of the prime interest rate in a variable named `interest_rate`. The variable name helps make its usage clear. You could also have created a variable named `x` or even `Rahul_jain`; it doesn't matter to the C compiler. The use of the variable, however, wouldn't be nearly as clear to someone else looking at the source code. Although it might take a little more time to type descriptive variable names, the improvements in program clarity make it worthwhile.

Many naming conventions are used for variable names created from multiple words. You've seen one style: `interest_rate`. Using an underscore to separate words in a variable name makes it easy to interpret. The second style is called *camel notation*. Instead of using spaces, the first letter of each word is capitalized. Instead of `interest_rate`, the variable would be named `InterestRate`. Camel notation is gaining popularity, because it's easier to type a capital letter than an underscore. We use the underscore in this book because it's easier for most people to read. You should decide which style you want to adopt.

## Numeric Variable Types

C provides several different types of numeric variables. You need different types of variables because different numeric values have varying memory storage requirements and differ in the ease with which certain mathematical operations can be performed on them. Small integers (for example, 1, 199, and -8) require less memory to store, and your computer can perform mathematical operations (addition, multiplication, and so on) with such numbers very quickly. In contrast, large integers and floating-point values (123,000,000 or 0.000000871256, for example) require more storage space and more time for mathematical operations. By using the appropriate variable types, you ensure that your program runs as efficiently as possible. C's numeric variables fall into the following two main categories:

- Integer variables hold values that have no fractional part (that is, whole numbers only). Integer variables come in two flavors: signed integer variables can hold positive or negative values, whereas unsigned integer variables can hold only positive values (and 0).
- Floating-point variables hold values that have a fractional part (that is, real numbers).

Within each of these categories are two or more specific variable types. These are summarized in below which also shows the amount of memory, in bytes, required to hold a single variable of each type when you use a microcomputer with 16-bit architecture.

### C's numeric data types.

| Variable Type                                          | Keyword                     | Bytes Required | Range                               |
|--------------------------------------------------------|-----------------------------|----------------|-------------------------------------|
| Character                                              | <code>char</code>           | 1              | -128 to 127                         |
| Integer                                                | <code>int</code>            | 2              | -32768 to 32767                     |
| Short integer                                          | <code>short</code>          | 2              | -32768 to 32767                     |
| Long integer                                           | <code>long</code>           | 4              | -2,147,483,648 to 2,147,438,647     |
| Unsigned character                                     | <code>unsigned char</code>  | 1              | 0 to 255                            |
| Unsigned integer                                       | <code>unsigned int</code>   | 2              | 0 to 65535                          |
| Unsigned short integer                                 | <code>unsigned short</code> | 2              | 0 to 65535                          |
| Unsigned long integer                                  | <code>unsigned long</code>  | 4              | 0 to 4,294,967,295                  |
| Single-precision<br>floating-point                     | <code>float</code>          | 4              | 1.2E-38 to<br>3.4E38 <sup>1</sup>   |
| Double-precision<br>floating-point                     | <code>double</code>         | 8              | 2.2E-308 to<br>1.8E308 <sup>2</sup> |
| <sup>1</sup> Approximate range; precision = 7 digits.  |                             |                |                                     |
| <sup>2</sup> Approximate range; precision = 19 digits. |                             |                |                                     |

*Approximate range* means the highest and lowest values a given variable can hold. (Space limitations prohibit listing exact ranges for the values of these variables.) *Precision* means the accuracy with which the variable is stored. (For example, if you evaluate 1/3, the answer is 0.33333... with 3s going to infinity. A variable with a precision of 7 stores seven 3s.)

You might notice that the variable type `int` and `short` are identical. Why are two different types necessary? The `int` and `short` variable types are indeed identical on 16-bit IBM PC-compatible systems, but they might be different on other types of hardware. On a VAX system, a `short` and an `int` aren't the same size. Instead, a `short` is 2 bytes, whereas an `int` is 4. Remember that C is a flexible, portable language, so it provides different keywords for the two types. If you're working on a PC, you can use `int` and `short` interchangeably.

No special keyword is needed to make an integer variable signed; integer variables are signed by default. You can, however, include the signed keyword if you wish. The keywords shown in Table 3.2 are used in variable declarations, which are discussed in the next section.

Program below will help you determine the size of variables on your particular computer. Don't be surprised if your output doesn't match the output presented after the listing.

#### **A program that displays the size of variable types.**

```
1:  /* SIZEOF.C--Program to tell the size of the C variable */
2:  /*      type in bytes */
3:
4:  #include <stdio.h>
5:
6:  main ()
7:  {
8:
9:      printf ("\nA char is %d bytes", sizeof (char));
10:     printf ("\nAn int is %d bytes", sizeof (int));
11:     printf ("\nA short is %d bytes", sizeof (short));
12:     printf ("\nA long is %d bytes", sizeof (long));
13:     printf ("\nAn unsigned char is %d bytes", sizeof (unsigned char));
14:     printf ("\nAn unsigned int is %d bytes", sizeof (unsigned int));
15:     printf ("\nAn unsigned short is %d bytes", sizeof (unsigned short));
16:     printf ("\nAn unsigned long is %d bytes", sizeof (unsigned long));
17:     printf ("\nA float is %d bytes", sizeof (float));
18:     printf ("\nA double is %d bytes\n", sizeof (double));
19:
20:     return 0;
21: }
```

A char is 1 bytes  
An int is 2 bytes  
A short is 2 bytes  
A long is 4 bytes  
An unsigned char is 1 bytes  
An unsigned int is 2 bytes  
An unsigned short is 2 bytes  
An unsigned long is 4 bytes  
A float is 4 bytes  
A double is 8 bytes

## ***Variable Declarations***

Before you can use a variable in a C program, it must be declared. A variable declaration tells the compiler the name and type of a variable and optionally initializes the variable to a specific value. If your program attempts to use a variable that hasn't been declared, the compiler generates an error message. A variable declaration has the following form:

*typename varname;*

*typename* specifies the variable type and must be one of the keywords listed in Table 3.2. *varname* is the variable name, which must follow the rules mentioned earlier. You can declare multiple variables of the same type on one line by separating the variable names with commas:

```
int count, number, start; /* three integer variables */  
float percent, total;    /* two float variables */
```

The location of variable declarations in the source code is important, because it affects the ways in which your program can use the variables. For now, you can place all the variable declarations together just before the start of the main () function.

## *The typedef Keyword*

The typedef keyword is used to create a new name for an existing data type. In effect, typedef creates a synonym. For example, the statement

```
typedef int integer;  
Creates integer as a synonym for int. You then can use integer to define variables of type int,  
as in this example:
```

```
integer count;  
Note that typedef doesn't create a new data type; it only lets you use a different name for a  
predefined data type. The most common use of typedef concerns aggregate data type is  
"Structures."
```

## *Initializing Numeric Variables*

When you declare a variable, you instruct the compiler to set aside storage space for the variable. However, the value stored in that space--the value of the variable--isn't defined. It might be zero, or it might be some random "garbage" value. Before using a variable, you should always initialize it to a known value. You can do this independently of the variable declaration by using an assignment statement, as in this example:

```
int count; /* Set aside storage space for count */  
count = 0; /* Store 0 in count */
```

Note that this statement uses the equal sign (=), which is C's assignment operator and is discussed further in the topic "Statements, Expressions, and Operators." For now, you need to be aware that the equal sign in programming is not the same as the equal sign in algebra. If you write

```
X = 12
```

in an algebraic statement, you are stating a fact: "x equals 12." In C, however, it means something quite different: "Assign the value 12 to the variable named x."

You can also initialize a variable when it's declared. To do so, follow the variable name in the declaration statement with an equal sign and the desired initial value:

```
int count = 0;  
double percent = 0.01, taxrate = 28.5;
```

Be careful not to initialize a variable with a value outside the allowed range. Here are two examples of out-of-range initializations:

```
int weight = 100000;  
unsigned int value = -2500;
```

The C compiler doesn't catch such errors. Your program might compile and link, but you might get unexpected results when the program is run.

## 1.5 Constants

Like a variable, a *constant* is a data storage location used by your program. Unlike a variable, the value stored in a constant can't be changed during program execution. C has two types of constants, each with its own specific uses.

### Literal Constants

A *literal constant* is a value that is typed directly into the source code wherever it is needed. Here are two examples:

```
int count = 10;  
float tax = 0.18;
```

The 10 and the 0.18 are literal constants. The preceding statements store these values in the variables `count` and `tax`. Note that one of these constants contains a decimal point, whereas the other does not. The presence or absence of the decimal point distinguishes floating-point constants from integer constants.

A literal constant written with a decimal point is a floating-point constant and is represented by the C compiler as a double-precision number. Floating-point constants can be written in standard decimal notation, as shown in these examples:

```
123.456  
0.019  
100.
```

Note that the third constant, 100, is written with a decimal point even though it's an integer (that is, it has no fractional part). The decimal point causes the C compiler to treat the constant as a double-precision value. Without the decimal point, it is treated as an integer constant.

Floating-point constants also can be written in scientific notation. You might recall from high school math that scientific notation represents a number as a decimal part multiplied by 10 to a positive or negative power. Scientific notation is particularly useful for representing extremely large and extremely small values. In C, scientific notation is written as a decimal number followed immediately by an E or e and the exponent:

```
1.23E2 1.23 times 10 to the 2nd power, or 123  
4.08e6 4.08 times 10 to the 6th power, or 4,080,000  
0.85e-4 0.85 times 10 to the -4th power, or 0.000085
```

The compiler represents a constant written without a decimal point as an integer number. Integer constants can be written in three different notations:

- A constant starting with any digit other than 0 is interpreted as a decimal integer (that is, the standard base-10 number system). Decimal constants can contain the digits 0 through 9 and a leading minus or plus sign. (Without a leading minus or plus, a constant is assumed to be positive.)
- A constant starting with the digit 0 is interpreted as an octal integer (the base-8 number system). Octal constants can contain the digits 0 through 7 and a leading minus or plus sign.
- A constant starting with 0x or 0X is interpreted as a hexadecimal constant (the base-16 number system). Hexadecimal constants can contain the digits 0 through 9, the letters A through F, and a leading minus or plus sign.

---

## Symbolic Constants

A *symbolic constant* is a constant that is represented by a name (symbol) in your program. Like a literal constant, a symbolic constant can't change. Whenever you need the constant's value in your program, you use its name as you would use a variable name. The actual value of the symbolic constant needs to be entered only once, when it is first defined.

Symbolic constants have two significant advantages over literal constants, as the following example shows. Suppose that you're writing a program that performs a variety of geometrical calculations. The program frequently needs the value, (3.14159) for its calculations. (You might recall from geometry class that, is the ratio of a circle's circumference to its diameter.) For example, to calculate the circumference and area of a circle with a known radius, you could write

```
Circumference = 3.14159 * (2 * radius);  
area = 3.14159 * (radius)*(radius);
```

The asterisk (\*) is C's multiplication operator. Thus, the first of these statements means "Multiply 2 times the value stored in the variable radius, and then multiply the result by 3.14159. Finally, assign the result to the variable named circumference."

If, however, you define a symbolic constant with the name PI and the value 3.14, you could write

```
circumference = PI * (2 * radius);  
Area = PI * (radius)*(radius);
```

The resulting code is clearer. Rather than puzzling over what the value 3.14 is for, you can see immediately that the constant PI is being used.

The second advantage of symbolic constants becomes apparent when you need to change a constant. Continuing with the preceding example, you might decide that for greater accuracy your program needs to use a value of PI with more decimal places: 3.14159 rather than 3.14. If you had used literal constants for PI, you would have to go through your source code and change each occurrence of the value from 3.14 to 3.14159. With a symbolic constant, you need to make a change only in the place where the constant is defined.

C has two methods for defining a symbolic constant: the `#define` directive and the `const` keyword. The `#define` directive is one of C's preprocessor directives, and it is discussed fully on Day 21, "Advanced Compiler Use." The `#define` directive is used as follows:

```
#define CONSTNAME literal
```

This creates a constant named *CONSTNAME* with the value of *literal*. *literal* represents a literal constant, as described earlier. *CONSTNAME* follows the same rules described earlier for variable names. By convention, the names of symbolic constants are uppercase. This makes them easy to distinguish from variable names, which by convention are lowercase. For the previous example, the required `#define` directive would be

```
#define PI 3.14159
```

Note that `#define` lines don't end with a semicolon (;). `#defines` can be placed anywhere in your source code, but they are in effect only for the portions of the source code that follow the `#define` directive. Most commonly, programmers group all `#defines` together, near the beginning of the file and before the start of `main` ().

### How a #define Works

The precise action of the `#define` directive is to instruct the compiler as follows: "In the source code, replace *CONSTNAME* with *literal*." The effect is exactly the same as if you had used your editor to go through the source code and make the changes manually. Note that `#define` doesn't replace instances of its target that occur as parts of longer names, within double quotes, or as part of a program comment. For example, in the following code, the instances of PI in the second and third lines would not get changed:

```
#define PI 3.14159
```

```
/* You have defined a constant for PI. */  
#define sum 100
```

## *Defining Constants with the const Keyword*

The second way to define a symbolic constant is with the const keyword. const is a modifier that can be applied to any variable declaration. A variable declared to be const can't be modified during program execution--only initialized at the time of declaration. Here are some examples:

```
const int count = 90;  
const float pi = 3.14159;  
const long debt = 11000000, float tax = 0.11;
```

const affects all variables on the declaration line. In the last line, debt and tax are symbolic constants. If your program tries to modify a const variable, the compiler generates an error message, as shown here:

```
const int count = 1000;  
count = 100;      /* Does not compile! Cannot reassign or alter */  
                  /* the value of a constant. */
```

What are the practical differences between symbolic constants created with the #define directive and those created with the const keyword? The differences have to do with pointers and variable scope. Pointers and variable scope are two very important aspects of C programming, and they will be discussed later.

Now let's look at a program that demonstrates variable declarations and the use of literal and symbolic constants. The below program explains the user to input his or her weight and year of birth. It then calculates and displays a user's weight in grams and his or her age in the year 2000.

### **Listing 3.2. A program that demonstrates the use of variables and constants.**

```
1:  /* Demonstrates variables and constants */  
2:  #include <stdio.h>  
3:  
4:  /* Define a constant to convert from pounds to grams */  
5:  #define GRAMS_PER_POUND 454  
6:  
7:  /* Define a constant for the start of the next century */  
8:  const int NEXT_CENTURY = 2000;  
9:  
10: /* Declare the needed variables */  
11: long weight_in_grams, weight_in_pounds;  
12: int year_of_birth, age_in_2000;  
13:  
14: main ()  
15: {  
16:     /* Input data from user */  
17:  
18:     printf ("Enter your weight in pounds: ");  
19:     scanf ("%d", &weight_in_pounds);  
20:     printf ("Enter your year of birth: ");  
21:     scanf ("%d", &year_of_birth);  
22:  
23:     /* Perform conversions */  
24:  
25:     weight_in_grams = weight_in_pounds * GRAMS_PER_POUND;
```

```
26:    age_in_2000 = NEXT_CENTURY - year_of_birth;
27:
28:    /* Display results on the screen */
29:
30:    printf ("\nyour weight in grams = %ld", weight_in_grams);
31:    printf ("\nIn 2000 you will be %d years old\n", age_in_2000);
32:
33:    return 0;
34: }
```

Enter your weight in pounds: **175**  
Enter your year of birth: **1960**  
Your weight in grams = 79450  
In 2000 you will be 40 years old



## 3. Statements. Expressions and Operators

C programs consist of statements, and most statements are composed of expressions and operators. You need to understand these three topics in order to be able to write C programs. Today you will learn

- What a statement is
- What an expression is
- C's mathematical, relational, and logical operators
- What operator precedence is
- The if statement
- The Loops

### 1.6 Statements

A *statement* is a complete direction instructing the computer to carry out some task. In C, statements are usually written one per line, although some statements span multiple lines. C statements always end with a semicolon (except for preprocessor directives such as `#define` and `#include`). You've already been introduced to some of C's statement types. For example:

`X = 2 + 3;`

Is an assignment statement. It instructs the computer to add 2 and 3 and to assign the result to the variable x.

### *Statements and White Space*

The term *white space* refers to spaces, tabs, and blank lines in your source code. The C compiler isn't sensitive to white space. When the compiler reads a statement in your source code, it looks for the characters in the statement and for the terminating semicolon, but it ignores white space. Thus, the statement

`X=2+3;`

Is equivalent to this statement:

`X = 2 + 3;`

It is also equivalent to this:

```
X      =  
2  
+  
3;
```

This gives you a great deal of flexibility in formatting your source code. You shouldn't use formatting like the previous example, however. Statements should be entered one per line with a standardized scheme for spacing around variables and operators. If you follow the formatting conventions used in this book, you should be in good shape. As you become more experienced, you might discover that you prefer slight variations. The point is to keep your source code readable.

However, the rule that C doesn't care about white space has one exception: Within literal string constants, tabs and spaces aren't ignored; they are considered part of the string. A *string* is a series of characters. Literal string constants are strings that are enclosed within quotes and interpreted literally by the compiler, space for space. Although it's extremely bad form, the following is legal:

```
printf(  
"Hello, world!"
```

```
);  
This, however, is not legal:  
printf("Hello,  
world!");
```

To break a literal string constant line, you must use the backslash character (\) just before the break. Thus, the following is legal:

```
printf("Hello,\nworld!");
```

## Null Statements

If you place a semicolon by itself on a line, you create a *null statement*--a statement that doesn't perform any action. This is perfectly legal in C. Later you will learn how the null statement can be useful.

## Compound Statements

A compound statement, also called a *block*, is a group of two or more C statements enclosed in braces. Here's an example of a block:

```
{  
    printf("Hello, ");  
    printf("world!");  
}
```

In C, a block can be used anywhere a single statement can be used. Many examples note that the enclosing braces can be positioned in different ways. The following is equivalent to the preceding example:

```
{printf("Hello, ");  
printf("world!");}
```

It's a good idea to place braces on their own lines, making the beginning and end of blocks clearly visible. Placing braces on their own lines also makes it easier to see whether you've left one out.

## 1.7 Expressions

In C, an *expression* is anything that evaluates to a numeric value. C expressions come in all levels of complexity.

### Simple Expressions

The simplest C expression consists of a single item: a simple variable, literal constant, or symbolic constant. Here are four expressions:

| <i>Expression</i> | <i>Description</i>                           |
|-------------------|----------------------------------------------|
| PI                | A symbolic constant (defined in the program) |
| 60                | A literal constant                           |
| Tax               | A variable                                   |
| -9.89             | Another literal constant                     |

A *literal constant* evaluates to its own value. A *symbolic constant* evaluates to the value it was given when you created it using the `#define` directive. A variable evaluates to the current value assigned to it by the program.

## Complex Expressions

*Complex expressions* consist of simpler expressions connected by operators.

For example:

`6 + 9`

is an expression consisting of the subexpressions 6 and 9 and the addition operator `+`. The expression `6 + 9` evaluates, as you know, to 15. You can also write C expressions of great complexity:

`1.25 / 8 + 5 * rate + rate * rate / cost`

When an expression contains multiple operators, the evaluation of the expression depends on operator precedence.

C expressions get even more interesting. Look at the following assignment statement:

`x = a + 19;`

This statement evaluates the expression `a + 19` and assigns the result to `x`. In addition, the entire statement `x = a + 19` is itself an expression that evaluates to the value of the variable on the left side of the equal sign.

Thus, you can write statements such as the following, which assigns the value of the expression `a + 19` to both variables, `x` and `y`:

`y = x = a + 19;`

*An assignment statement is itself an expression.*

You can also write statements such as this:

`x = 6 + (y = 4 + 5);`

The result of this statement is that `y` has the value 9 and `x` has the value 15. Note the parentheses, which are required in order for the statement to compile. The use of parentheses is covered later in this chapter.

## 1.8 Operators

An *operator* is a symbol that instructs C to perform some operation, or action, on one or more operands. An *operand* is something that an operator acts on. In C, all operands are expressions. C operators fall into several categories:

- The assignment operator
- Mathematical operators
- Relational operators
- Logical operators

### The Assignment Operator

The *assignment operator* is the equal sign (`=`). Its use in programming is somewhat different from its use in regular math. If you write

`x = y;`

in a C program, it doesn't mean "x is equal to y." Instead, it means "assign the value of y to x."

In a C assignment statement, the right side can be any expression, and the left side must be a variable name. Thus, the form is as follows:

*variable* = *expression*;

When executed, *expression* is evaluated, and the resulting value is assigned to *variable*.

## Mathematical Operators

C's mathematical operators perform mathematical operations such as addition and subtraction. C has two unary mathematical operators and five binary mathematical operators.

### Unary Mathematical Operators

The *unary* mathematical operators are so named because they take a single operand. C has two unary mathematical operators, listed in Table 4.1.

#### C's unary mathematical operators.

| Operator  | Symbol | Action                        | Examples |
|-----------|--------|-------------------------------|----------|
| Increment | ++     | Increments the operand by one | ++x, x++ |
| Decrement | --     | Decrements the operand by one | --x, x-- |

The increment and decrement operators can be used only with variables, not with constants. The operation performed is to add one to or subtract one from the operand. In other words, the statements

```
++x;
```

```
--y;
```

are the equivalent of these statements:

```
x = x + 1;
```

```
y = y - 1;
```

You should note from above table that either unary operator could be placed before its operand (*prefix* mode) or after its operand (*postfix* mode). These two modes are not equivalent. They differ in terms of when the increment or decrement is performed:

- When used in prefix mode, the increment and decrement operators modify their operand before it's used.
- When used in postfix mode, the increment and decrement operators modify their operand after it's used.

An example should make this clearer. Look at these two statements:

```
x = 10;
```

```
y = x++;
```

After these statements are executed, x has the value 11, and y has the value 10. The value of x was assigned to y, and then x was incremented. In contrast, the following statement result in both y and x having the value 11. x is incremented, and then its value is assigned to y.

```
x = 10;
```

```
y = ++x;
```

Remember that = is the assignment operator, not a statement of equality. As an analogy, think of = as the "photocopy" operator. The statement y = x means to copy x into y. Subsequent changes to x, after the copy has been made, have no effect on y.

#### UNARY.C: Demonstrates prefix and postfix modes.

```
1: /* Demonstrates unary operator prefix and postfix modes */
```

```
2:
```

```
3: #include <stdio.h>
```

```
4:
```

```
5: int a, b;
6:
7: main ()
8: {
9:     /* Set a and b both equal to 5 */
10:
11:     a = b = 5;
12:
13:     /* Print them, decrementing each time. */
14:     /* Use prefix mode for b, postfix mode for a */
15:
16:     printf("\n%d  %d", a--, --b);
17:     printf("\n%d  %d", a--, --b);
18:     printf("\n%d  %d", a--, --b);
19:     printf("\n%d  %d", a--, --b);
20:     printf("\n%d  %d\n", a--, --b);
21:
22:     return 0;
23:}
```

5 4  
4 3  
3 2  
2 1  
1 0

## Binary Mathematical Operators

C's binary operators take two operands. The binary operators, which include the common mathematical operations found on a calculator

### C's binary mathematical operators.

| Operator       | Symb<br>ol | Action                                                                      | Exam<br>ple |
|----------------|------------|-----------------------------------------------------------------------------|-------------|
| Addition       | +          | Adds two operands                                                           | x + y       |
| Subtraction    | -          | Subtracts the second operand from the first operand                         | x - y       |
| Multiplication | *          | Multiplies two operands                                                     | x * y       |
| Division       | /          | Divides the first operand by the second operand                             | x / y       |
| Modulus        | %          | Gives the remainder when the first operand is divided by the second operand | x % y       |

The first four operators listed above should be familiar to you, and you should have little trouble using them. The fifth operator, modulus, might be new. Modulus returns the remainder when the first operand is divided by the second operand. For example, 11 modulus 4 equals 3 (that is, 4 goes into 11 two times with 3 left over). Here are some more examples:

100 modulus 9 equals 1  
10 modulus 5 equals 0  
40 modulus 6 equals 4

### SECONDS.C: Demonstrates the modulus operator.

```
1: /* Illustrates the modulus operator. */
2: /* Inputs a number of seconds, and converts to hours, */
3: /* minutes, and seconds. */
4:
5: #include <stdio.h>
```

```
6:
7:  /* Define constants */
8:
9:  #define SECS_PER_MIN 60
10: #define SECS_PER_HOUR 3600
11:
12: unsigned seconds, minutes, hours, secs_left, and mins_left;
13:
14: main ()
15: {
16:     /* Input the number of seconds */
17:
18:     printf("Enter number of seconds (< 65000): ");
19:     scanf("%d", &seconds);
20:
21:     hours = seconds / SECS_PER_HOUR;
22:     minutes = seconds / SECS_PER_MIN;
23:     mins_left = minutes % SECS_PER_MIN;
24:     secs_left = seconds % SECS_PER_MIN;
25:
26:     printf("%u seconds is equal to ", seconds);
27:     printf("%u h, %u m, and %u s\n", hours, mins_left, secs_left);
28:
29:     return 0;
30:}
Enter number of seconds (< 65000): 60
60 seconds is equal to 0 h, 1 m, and 0 s
Enter number of seconds (< 65000): 10000
10000 seconds is equal to 2 h, 46 m, and 40 s
```

## Operator Precedence and Parentheses

In an expression that contains more than one operator, what is the order in which operations are performed? The importance of this question is illustrated by the following assignment statement:

```
x = 4 + 5 * 3;
```

Performing the addition first results in the following, and x is assigned the value 27:

```
x = 9 * 3;
```

In contrast, if the multiplication is performed first, you have the following, and x is assigned the value 19:

```
x = 4 + 15;
```

Clearly, some rules are needed about the order in which operations are performed. This order, called *operator precedence*, is strictly spelled out in C. Each operator has a specific precedence. When an expression is evaluated, operators with higher precedence are performed first. The table below lists the precedence of C's mathematical operators. Number 1 is the highest precedence and thus is evaluated first.

**Table 4.3. The precedence of C's mathematical operators.**

| Operators | Relative Precedence |
|-----------|---------------------|
| ++ --     | 1                   |
| * / %     | 2                   |

|     |   |
|-----|---|
| + - | 3 |
|-----|---|

Looking at table you can see that in any C expression, operations are performed in the following order:

- Unary increment and decrement
- Multiplication, division, and modulus
- Addition and subtraction

If an expression contains more than one operator with the same precedence level, the operators are performed in left-to-right order as they appear in the expression. For example, in the following expression, the % and \* have the same precedence level, but the % is the leftmost operator, so it is performed first:

$12 \% 5 * 2$

The expression evaluates to 4 ( $12 \% 5$  evaluates to 2; 2 times 2 is 4).

Returning to the previous example, you see that the statement  $x = 4 + 5 * 3$ ; assigns the value 19 to x because the multiplication is performed before the addition.

What if the order of precedence doesn't evaluate your expression as needed? Using the previous example, what if you wanted to add 4 to 5 and then multiply the sum by 3? C uses parentheses to modify the evaluation order. A subexpression enclosed in parentheses is evaluated first, without regard to operator precedence. Thus, you could write

$x = (4 + 5) * 3$ ;

The expression  $4 + 5$  inside parentheses is evaluated first, so the value assigned to x is 27.

You can use multiple and nested parentheses in an expression. When parentheses are nested, evaluation proceeds from the innermost expression outward. Look at the following complex expression:

$x = 25 - (2 * (10 + (8 / 2)))$ ;

The evaluation of this expression proceeds as follows:

1. The innermost expression,  $8 / 2$ , is evaluated first, yielding the value 4:

$25 - (2 * (10 + 4))$

2. Moving outward, the next expression,  $10 + 4$ , is evaluated, yielding the value 14:

$25 - (2 * 14)$

3. The last, or outermost, expression,  $2 * 14$ , is evaluated, yielding the value 28:

$25 - 28$

4. The final expression,  $25 - 28$ , is evaluated, assigning the value -3 to the variable x:

$x = -3$

You might want to use parentheses in some expressions for the sake of clarity, even when they aren't needed for modifying operator precedence. Parentheses must always be in pairs, or the compiler generates an error message.

### Order of Subexpression Evaluation

As was mentioned in the previous section, if C expressions contain more than one operator with the same precedence level, they are evaluated left to right.

For example, in the expression

$w * x / y * z$

w is multiplied by x, the result of the multiplication is then divided by y, and the result of the division is then multiplied by z.

Across precedence levels, however, there is no guarantee of left-to-right order. Look at this expression:

$w * x / y + z / y$

Because of precedence, the multiplication and division are performed before the addition. However, C doesn't specify whether the subexpression  $w * x / y$  is to be evaluated before or after  $z / y$ . It might not be clear to you why this matters. Look at another example:

$w * x / ++y + z / y$

If the left subexpression is evaluated first,  $y$  is incremented when the second expression is evaluated. If the right expression is evaluated first,  $y$  isn't incremented, and the result is different. Therefore, you should avoid this sort of indeterminate expression in your programming.

## Relational Operators

C's relational operators are used to compare expressions, asking questions such as, "Is  $x$  greater than 100?" or "Is  $y$  equal to 0?" An expression containing a relational operator evaluates to either true (1) or false (0). C's six relational operators are listed in Table below.

Table below shows some examples of how relational operators might be used. These examples use literal constants, but the same principles hold with variables.

### C's relational operators.

| Operator                 | Symb<br>ol         | Question Asked                                   | Examp<br>le            |
|--------------------------|--------------------|--------------------------------------------------|------------------------|
| Equal                    | <code>==</code>    | Is operand 1 equal to operand 2?                 | <code>x == y</code>    |
| Greater than             | <code>&gt;</code>  | Is operand 1 greater than operand 2?             | <code>x &gt; y</code>  |
| Less than                | <code>&lt;</code>  | Is operand 1 less than operand 2?                | <code>x &lt; y</code>  |
| Greater than or equal to | <code>&gt;=</code> | Is operand 1 greater than or equal to operand 2? | <code>x &gt;= y</code> |
| Less than or equal to    | <code>&lt;=</code> | Is operand 1 less than or equal to operand 2?    | <code>x &lt;= y</code> |
| Not equal                | <code>!=</code>    | Is operand 1 not equal to operand 2?             | <code>x != y</code>    |

### Relational operators in use.

| Expression                       | How It Reads                  | What It Evaluates To |
|----------------------------------|-------------------------------|----------------------|
| <code>5 == 1</code>              | Is 5 equal to 1?              | 0 (false)            |
| <code>5 &gt; 1</code>            | Is 5 greater than 1?          | 1 (true)             |
| <code>5 != 1</code>              | Is 5 not equal to 1?          | 1 (true)             |
| <code>(5 + 10) == (3 * 5)</code> | Is (5 + 10) equal to (3 * 5)? | 1 (true)             |

### The Precedence of Relational Operators

Like the mathematical operators discussed earlier in this chapter, the relational operators each have a precedence that determines the order in which they are performed in a multiple-operator expression. Similarly, you can use parentheses to modify precedence in expressions that use relational operators. First, all the relational operators have a lower precedence than the mathematical operators. Thus, if you write the following, 2 is added to  $x$ , and the result is compared to  $y$ :

`if (x + 2 > y)`

This is the equivalent of the following line, which is a good example of using parentheses for the sake of clarity:

`if ((x + 2) > y)`

Although they aren't required by the C compiler, the parentheses surrounding  $(x + 2)$  make it clear that it is the sum of  $x$  and 2 that is to be compared with  $y$ .

There is also a two-level precedence within the relational operators, as shown in table below.



### The order of precedence of C's relational operators.

| Operators | Relative Precedence |
|-----------|---------------------|
| < <= > >= | 1                   |
| ! ==      | 2                   |

Thus, if you write

```
x == y > z
```

it is the same as

```
x == (y > z)
```

Because C first evaluates the expression `y > z`, resulting in a value of 0 or 1. Next, C determines whether `x` is equal to the 1 or 0 obtained in the first step.

## Logical Operators

Sometimes you might need to ask more than one relational question at once. For example, "If it's 7:00 a.m. and a weekday and not my vacation, ring the alarm." C's logical operators let you combine two or more relational expressions into a single expression that evaluates to either true or false. Table below lists C's three logical operators.

### C's logical operators.

| Operator | Symbol | Example                           |
|----------|--------|-----------------------------------|
| AND      | &&     | <code>exp1 &amp;&amp; exp2</code> |
| OR       |        | <code>exp1    exp2</code>         |
| NOT      | !      | <code>!exp1</code>                |

The way these logical operators work is explained in Table

### C's logical operators in use.

| Expression                          | What It Evaluates To                                                                                |
|-------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>(exp1 &amp;&amp; exp2)</code> | True (1) only if both <code>exp1</code> and <code>exp2</code> are true; false (0) otherwise         |
| <code>(exp1    exp2)</code>         | True (1) if either <code>exp1</code> or <code>exp2</code> is true; false (0) only if both are false |
| <code>(!exp1)</code>                | False (0) if <code>exp1</code> is true; true (1) if <code>exp1</code> is false                      |

You can see that expressions that use the logical operators evaluate to either true or false, depending on the true/false value of their operand(s). Table below shows some actual code examples.

### Code examples of C's logical operators.

| Expression                                | What It Evaluates To                     |
|-------------------------------------------|------------------------------------------|
| <code>(5 == 5) &amp;&amp; (6 != 2)</code> | True (1), because both operands are true |
| <code>(5 &gt; 1)    (6 &lt; 1)</code>     | True (1), because one operand is true    |
| <code>(2 == 1) &amp;&amp; (5 == 5)</code> | False (0), because one operand is false  |
| <code>!(5 == 4)</code>                    | True (1), because the operand is false   |

You can create expressions that use multiple logical operators. For example, to ask the question "Is `x` equal to 2, 3, or 4?" you would write

```
(x == 2) || (x == 3) || (x == 4)
```

The logical operators often provide more than one way to ask a question. If `x` is an integer variable, the proceeding question also could be written in either of the following ways:

```
(x > 1) && (x < 5)
```

```
(x >= 2) && (x <= 4)
```

### More on True/False Values

You've seen that C's relational expressions evaluate to 0 to represent false and to 1 to represent true. It's important to be aware, however, that any numeric value is interpreted as either true or false when it is used in a C expression or statement that is expecting a logical value (that is, a true or false value). The rules are as follows:

- A value of zero represents false.
- Any nonzero value represents true.

This is illustrated by the following example, in which the value of x is printed:

```
x = 125;  
if (x)  
    printf("%d", x);
```

Because x has a nonzero value, the if statement interprets the expression (x) as true. You can further generalize this because, for any C expression, writing

*(expression)*

is equivalent to writing

*(expression != 0)*

Both evaluate to true if *expression* is nonzero and to false if *expression* is 0. Using the not (!) operator, you can also write

*(!expression)*

which is equivalent to

*(expression == 0)*

## *The Precedence of Operators*

As you might have guessed, C's logical operators also have a precedence order both among themselves and in relation to other operators. The ! operator has a precedence equal to the unary mathematical operators ++ and --. Thus, ! has a higher precedence than all the relational operators and all the binary mathematical operators.

In contrast, the && and || operators have much lower precedence, lower than all the mathematical and relational operators, although && has a higher precedence than ||. As with all of C's operators, parentheses can be used to modify the evaluation order when using the logical operators. Consider the following example:

You want to write a logical expression that makes three individual comparisons:

1. Is a less than b?
2. Is a less than c?
3. Is c less than d?

You want the entire logical expression to evaluate to true if condition 3 is true and if either condition 1 or condition 2 is true. You might write

```
a < b || a < c && c < d
```

However, this won't do what you intended. Because the && operator has higher precedence than ||, the expression is equivalent to

```
a < b || (a < c && c < d)
```

and evaluates to true if (a < b) is true, whether or not the relationships (a < c) and (c < d) are true. You need to write

```
(a < b || a < c) && c < d
```

which forces the || to be evaluated before the &&. . The variables are set so that, if written correctly, the expression should evaluate to false (0).

### *Logical operator precedence.*

```
1: #include <stdio.h>  
2:  
3: /* Initialize variables. Note that c is not less than d, */  
4: /* which is one of the conditions to test for. */  
5: /* Therefore, the entire expression should evaluate as false.*/  
6:  
7: int a = 5, b = 6, c = 5, d = 1;
```

```
8: int x;
9:
10: main ()
11: {
12:     /* evaluate the expression without parentheses */
13:
14:     x = a < b || a < c && c < d;
15:     printf ("without parentheses the expression evaluates as %d", x);
16:
17:     /* evaluate the expression with parentheses */
18:
19:     x = (a < b || a < c) && c < d;
20:     printf ("with parentheses the expression evaluates as %d\n", x);
21:     return 0;
22: }
```

Without parentheses the expression evaluates as 1

With parentheses the expression evaluates as 0

## Compound Assignment Operators

C's compound assignment operators provide a shorthand method for combining a binary mathematical operation with an assignment operation. For example, say you want to increase the value of x by 5, or, in other words, add 5 to x and assign the result to x. You could write

```
x = x + 5;
```

Using a compound assignment operator, whom you can think of as a shorthand method of assignment, you would write

```
x += 5;
```

In more general notation, the compound assignment operators have the following syntax (where op represents a binary operator):

```
exp1 op= exp2
```

This is equivalent to writing

```
exp1 = exp1 op exp2;
```

You can create compound assignment operators using the five binary mathematical operators discussed earlier in this chapter. Table below lists some examples.

### Examples of compound assignment operators.

| When You Write This... | It Is Equivalent To This |
|------------------------|--------------------------|
| x *= y                 | x = x * y                |
| y -= z + 1             | y = y - z + 1            |
| a /= b                 | a = a / b                |
| x += y / 8             | x = x + y / 8            |
| y %= 3                 | y = y % 3                |

The compound operators provide convenient shorthand, the advantages of that are particularly evident when the variable on the left side of the assignment operator has a long name. As with all other assignment statements, a compound assignment statement is an expression and evaluates to the value assigned to the left side. Thus, executing the following statements results in both x and z having the value 14:

```
x = 12;
```

```
z = x += 2;
```

## The Comma Operator

The comma is frequently used in C as a simple punctuation mark, serving to separate variable declarations, function arguments, and so on. In certain situations, the comma acts as an operator rather than just as a separator. You can form an expression by separating two subexpressions with a comma. The result is as follows:

- Both expressions are evaluated, with the left expression being evaluated first.
- The entire expression evaluates to the value of the right expression.

For example, the following statement assigns the value of b to x, then increments a, and then increments b:

```
x = (a++ , b++);
```

Because the ++ operator is used in postfix mode, the value of b--before it is incremented--is assigned to x. using parentheses is necessary because the comma operator has low precedence, even lower than the assignment operator.

As you'll learn in the next chapter, the most common use of the comma operator is in for statements.

## Operator Precedence Revisited

Table below lists all the C operators in order of decreasing precedence. Operators on the same line have the same precedence.

### C operator precedence.

| Level                                                  | Operators                                                                     |
|--------------------------------------------------------|-------------------------------------------------------------------------------|
| 1                                                      | () [] -> .                                                                    |
| 2                                                      | ! ~ ++ -- * (indirection) & (address-of) (type)<br>sizeof + (unary) - (unary) |
| 3                                                      | * (multiplication) / %                                                        |
| 4                                                      | + -                                                                           |
| 5                                                      | << >>                                                                         |
| 6                                                      | < <= > >=                                                                     |
| 7                                                      | ==, !=                                                                        |
| 8                                                      | & (bitwise AND)                                                               |
| 9                                                      | ^                                                                             |
| 10                                                     |                                                                               |
| 11                                                     | &&                                                                            |
| 12                                                     |                                                                               |
| 13                                                     | ?:                                                                            |
| 14                                                     | = += -= *= /= %= &= ^=  = <<= >>=                                             |
| 15                                                     | ,                                                                             |
| () is the function operator; [] is the array operator. |                                                                               |

## The If Statement

The if statement is one of the most powerful devices in programming. It allows the program to choose an action based upon input. For example, a program can be written to decide if a user can access the program by using an if statement to check if the password is correct or not.

Without a conditional statement such as if programs would run almost the exact same way every time. If statements allow the flow of the program to be changed.

Please note that all Boolean operators or values will be capitalized to differentiate them from normal English. Note, as well, that the actual C symbols to the operators are described later,

but the C symbols are not OR, NOT, and AND. The symbols simply mean the same thing.

There are many things to understand when using if statements. You must understand Boolean operators such as OR, NOT, and AND. You must also understand comparisons between numbers and most especially, the idea of true and false in the same way computers do.

True and false in C are denoted by a non-zero number, and zero. Therefore, 1 is TRUE and 8.3 is TRUE but 0 is FALSE. No other number is interpreted as being false.

When programming, the program often will require the checking of one value stored by a variable against another value, to determine which is larger, smaller, or if the two are equal.

The structure of an if statement is essentially:

if (TRUE)

Do whatever follows on the next line.

To have more than one statement execute after an if statement (when it evaluates to true) use brackets.

For example,

if (TRUE)

{

Do everything between the brackets.

}

There is also the else statement. The code after it (whether a single line or code between brackets) is executed if the IF statement is FALSE.

It can look like this:

if(FALSE)

{

Not executed if it's false

}

else

{

do all of this

}

One use for else is if there are two conditional statements that will both evaluate to true with the same value (but only at times. For example,  $x < 30$  and  $x < 50$  will both return true if  $x$  is less than 30 but not otherwise), but you wish only one of them to be checked. You can use else after the if statement. If the if statement was true the else statement will not be checked. It is possible to use numerous else statements followed by if statements.

Let's look at a simple program for you to try out on your own...

```
#include <stdio.h>
```

```
int main()
```

```
//Most important part of the program!
```

```
{
```

```
int age;
```

```
//Need a variable...
```

```
printf("enter your age");
```

```
scanf("%d", &age);
```

```
//Asks for age
```

```
//The input is put in age
```

```
//If the age is less than 100
```

```
if(age<100)
```

```
{
```

```
printf("You are pretty young!"); //Just to show it works
```

```
}
```

```
else if(age==100)
```

```
//I use else just to show an example
```

```
{
```

```
printf("You are old");
```

```
//Just to show you it works...
```

```
}
```

```
else if(age>100)
```

```
{
```

```
    printf("You are really old");    //Proof    that    it    works    for    any    condition
}
return 0;
}
```

This program did not use && || ! or in it. This is because it did not need too. Its purpose was to demonstrate if statements

## Loops

Loops are used to repeat a block of code. You should understand the concept of C true and false, because it will be necessary when working with loops. There are three types of loops: for, while, (and) do while. Each of them has their specific uses. They are all outlined below.

FOR - for loops are the most useful type. The layout is for(variable initialization, conditional expression, modification of variable) The variable initialization allows you to either declare a variable and give it a value or give a value to an already declared variable. Second, the conditional expression tells the program that while the conditional expression is true the loop should continue to repeat itself. The variable modification section is the easiest way for a for loop to handle changing of the variable. It is possible to do things like x++, x=x+10, or even x=random (5), and if you really wanted to, you could call other functions that do nothing to the variable. That would be something ridiculous probably.

Example:

```
#include <stdio.h>    //We only need one header file
int main()           //We always need this
{
    //The loop goes while x<100, and x increases by one every loop
    for(int x=0;x<100;x++) //Keep in mind that the loop condition checks
    {
        //the conditional statement before it loops again.
        //consequently, when x equals 100 the loop breaks
        printf("%d", x);    //Outputting x
    }
    return 0;
}
```

This program is a very simple example of a for loop. x is set to zero, while x is less than 100 it calls printf("%d",x); and it adds 1 to x until the loop ends. Keep in mind also that the variable is incremented after the code in the loop is run for the first time. WHILE loops are very simple. The basic structure is...while (true) then executes all the code in the loop. The true represents a boolean expression which could be x==1 or while (x!=7) (x does not equal 7). It can be any combination of boolean statements that are legal. Even, (while x==5 || v==7) which says execute the code while x equals five or while v equals 7.

Example:

```
#include <stdio.h>    //We only need this header file
int main()           //Of course...
{
    int x=0;          //Don't forget to declare variables
    while(x<100)      //While x is less than 100 do
    {
        printf("%d", x);    //Same output as the above loop
        x++;               //Adds 1 to x every time it repeats, in for loops the
                           //loop structure allows this to be done in the structure
    }
    return 0;
}
```

This was another simple example, but it is longer than the above FOR loop. The easiest way to think of the loop is to think the code executes, when it reaches the brace at the end it goes all the way back up to while, which checks the boolean expression.

DO WHILE - DO WHILE loops are useful for only things that want to loop at least once. The structure is DO {THIS} WHILE (TRUE);

Example:

```
#include <stdio.h>
int main()
{
    int x;
    x=0;
    do
    {
        printf("Hello world!");
    }while(x!=0);    //Loop while x is not zero, then first execute the
                    //code in the section. (It outputs "Hello..." once
    return 0;
}
```

Keep in mind that you must include a trailing semi-colon after while in the above example. Notice that this loop will also execute once, because it automatically executes before checking the truth statement.

## *switch case*

Switch case statements are a substitute for long if statements. The basic format for using switch case is outlined below.

Switch (expression or variable)

```
{
case variable equals this:
do this;
break;
case variable equals this:
do this;
break;
case variable equals this:
do this;
break;
...
default:
do this
}
```

The expression or variable has a value. The case says that if it has the value of whatever is after that cases then do whatever follows the colon. The break is used to break out of the case statements. Break is a keyword that breaks out of the code block, usually surrounded by braces, which it is in. In this case, break prevents the program from testing the next case statement also.

Switch case serves as a simple way to write long if statements. Often it can be used to process input from a user.

Below is a sample program, in which not all of the proper functions are actually declared, but which shows how one would use switch case in a program.

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int input;
    printf("1. Play game");
    printf("2. Load game");
```

```
printf("3. Play multiplayer");  
printf("4. Exit");  
scanf("%d", &input);  
switch (input)  
{  
    case 1: playgame();  
           break;  
    case 2:  
           loadgame();  
           break;  
    case 3: //Note use of : not ;  
           playmultiplayer();  
           break;  
    case 4:  
           return 0;  
    default:  
           printf("Error, bad input, quitting");  
}  
return 0;  
}
```

This program will not compile yet, but it serves as a model (albeit simple) for processing input.



## 4. Functions

### 1.9 A Brief Overview

A function is self-conditional program segment that carries out some specific, well-defined task. Every C program consists of one or more functions. One of these functions must be called main. Program execution will always begin by carrying out the instructions in main. Additional functions will be subordinate to main, and perhaps to one another.

If a program contains multiple functions, their definitions may appear in any order, though they must be independent of one another. That is, one function definition cannot be embedded within another. A function will carry out its intended action whenever it is accessed from some other portion of the program. The same function can be accessed from several different places within a program.

Generally, a function will process information passed to it from the calling portion of the program, and return a single value. Information will be passed to the function via special identifiers called arguments (also called parameters) and returned via the return statement. Some functions, however, accept information but do not return anything (for example function printf), whereas other functions (the library functions scanf, for example) return multiple values.

/\* Convert a lower-case character to upper-case using a programmer-defined function\*/

```
main()
{
    char lower, upper;
    char lower_to_upper (char lower);

    printf("Please enter a lower-case character:");
    scanf("%c", &lower);

    upper = lower_to_upper(lower);
    printf("\n The upper case equivalent is %c\n\n", upper);
}

char lower_to_upper (char c1)
{
    char c2;
    c2 = (c1 >= 'a' && c1 <= 'z') ? ('A' + c1 - 'a') : c1;
    return (c2);
}
```

This program consists of two functions – the required main function, and the programmer-defined function `lower_to_upper`, which converts lowercase characters to uppercase. Notice that main reads in a character (which may or may not be a lowercase letter) and assigns it to the char-type variable `lower`. Then main calls `lower_to_upper`, transferring the lowercase character (`lower`) to `lower_to_upper` and receiving the equivalent uppercase character (`upper`). The uppercase character is then written out, and the program ends.

Now consider the function `lower_to_upper`. This function converts only lowercase letters; all other characters are returned intact.

## 1.10 Defining a function

Now that you should have learned about variables, loops, and if statements it is time to learn about functions. You should have an idea of their uses. `printf` is an example of a function. In general, functions perform a number of pre-defined commands to accomplish something productive.

Functions that a programmer writes will generally require a prototype. Just like an blueprint, the prototype tells the compiler what the function will return, what the function will be called, as well as what arguments the function can be passed. When I say that the function returns a value, I mean that the function can be used in the same manner as a variable would be. For example, a variable can be set equal to a function that returns a value between zero and four.

For example:  
`int a;`  
`a=random(5);` //random is sometimes defined by the compiler  
//Yes, it returns between 0 and the argument minus 1

Do not think that 'a' will change at random, it will be set to the value returned when the function is called, but it will not change again.

The general format for a prototype is simple:

return-type                      function\_name(arg\_type                      arg);

There can be more than one argument passed to a function, and it does not have to return a value. Lets look at a function prototype:

`int mult(int x, int y);`

This prototype specifies that the function `mult` will accept two arguments, both integers, and that it will return an integer. Do not forget the trailing semi-colon. Without it, the compiler will probably think that you are trying to write the actual definition of the function.

When the programmer actually defines the function, it will begin with the prototype, minus the semi-colon. Then there should always be a bracket (remember, functions require brackets around them) followed by code, just as you would write it for the main function.

Lets look at an example program:

```
#include <stdio.h>

int mult(int x, int y);

int main()
{
```

```
int x, y;
printf("Please input two numbers to be multiplied: ");
scanf("%d %d", &x, &y);
printf("The product of your two numbers is:%d", mult(x,y));
return 0;
}
int mult(int x, int y)
{
    return x*y;
}
```

This program begins with the only necessary include file. It is followed by the prototype of the function. Notice that it has the final semi-colon! The main function is an integer, which you should always have, to conform to the standard. You should not have trouble understanding the input and output functions. It is fine to use scanf to input to variables as the program does.

Notice how printf actually outputs what appears to be the mult function. What is really happening is that mult acts as a variable. Because it returns a value it is possible for the printf function to output the return value.

The mult function is actually defined below main. Due to its prototype being above main, the compiler still recognizes it as being defined, and so the compiler will not give an error about mult being undefined, although the definition is below where it is used.

return is the keyword used to force the function to return a value. Note that it is possible to have a function that returns no value. In that case, the prototype would have a return type of void.

The most important functional (Pun semi-intended) question is why. Functions have many uses. For example, a programmer may have a block of code that he has repeated forty times throughout the program. A function to execute that code would save a great deal of space, and it would also make the program more readable.

Another reason for functions is to break down a complex program into something manageable. For example, take a menu program that runs complex code when a menu choice is selected. The program would probably best be served by making functions for each of the actual menu choices, and then breaking down the complex tasks into smaller, more manageable takes, which could be in their own functions. In this way, a program can be designed that makes sense when read.

## 1.11 Accessing a Function

You were given the basic information on functions. When you declare a function you do not have to prototype it! You must give the function definition physically before you call the function. You simply type in the entire definition of the function where you would normally put the prototype.

For example:

```
#include <stdio.h>
void function(void)
{
    //Normally this would be the prototype. Do not include a
    //Only prototypes have semicolons
    printf("HA! NO PROTOTYPE!");
}

int main()
```

```
{  
    function();  
    //It works like a normal function now.  
    return 0;  
}
```

The other programming concept is the **inline function**. Inline functions are not very important, but it is good to understand them. The basic idea is to save time at a cost in space. Inline functions are a lot like a placeholder. Once you define an inline function, using the 'inline' keyword, whenever you call that function the compiler will replace the function call with the actual code from the function.

How does this make the program go faster? Simple, function calls are simply more time consuming than writing all of the code without functions. To go through your program and replace a function you have used 100 times with the code from the function would be time consuming. Of course, by using the **inline function** to replace the function calls with code you will also greatly increase the size of your program.

Using the inline keyword is simple, just put it before the name of a function. Then, when you use that function, pretend it is a non-inline function.

For example:

```
#include <stdio.h>  
inline void hello(void)  
{  
    //Just use the inline keyword before the function  
    printf("hello");  
}  
int main()  
{  
    hello();  
    //Call it like a normal function...  
    return 0;  
}
```

However, once the program is compiled, the call to hello(); will be replaced by the code making up the function .

A WORD OF WARNING: Inline functions are very good for saving time, but if you use them too often or with large functions you will have a tremendously large program. Sometimes large programs are actually less efficient, and therefore they will run more slowly than before. Inline functions are best for small functions that are called often.

```
/* determine the largest of three integer quantities*/
```

```
main()  
{  
  
    int a, b, c, d;  
  
    /* read the integer quantities */  
  
    printf("\n a = ");  
    scanf("%d", &a);  
  
    printf("\n b = ");  
    scanf("%d", &b);  
  
    printf("\n c = ");  
    scanf("%d", &c);  
  
    d = maximum (a,b);  
}
```

```
printf("\n\n maximum = \"%d\", maximum (c, d));
```

```
}
```

```
maximum (x,y)
```

```
    int x, y;  
    {  
        int z;  
        z = (x>= y) ? x:y;  
        return (z);  
    }
```

The function maximum is accessed from two different places in main. In the first call to maximum the actual arguments are the variables a and b, whereas the arguments are c and d in the second call ( d is a temporary variable representing the maximum value of a and b).

Note that the two statements which access maximum.

```
d = maximum (a,b);
```

```
printf("\n\n maximum = %d", maximum (c, d));
```

Can be replaced by the single statement.

```
printf("\n\n maximum = %d", maximum (c, maximum (a,b)));
```

Example : Calculating Factorials :-

```
#define <stdio.h>
```

```
main ()
```

```
{
```

```
    int n;  
    long int factorial ();
```

```
/* read in the integer quantity */
```

```
    printf( "\n n = ");  
    scanf("%d", &n);  
  
    printf(" \n n! = %1d", factorial (n));  
}
```

```
    long int factorial (n)
```

```
/* calculate the factorial of n*/
```

```
    int n;  
  
    long int prod = 1;  
  
    if (n>1)  
  
        for (l=2; l<=n; ++l)  
            prod *=l;  
    return (prod);  
}
```

## 1.12 Passing Arguments to a Function

When a single value is passed to a function via an actual argument, the value of the actual argument is copied into the function. Therefore, the value of the corresponding formal argument can be altered within the function, but the value of the actual argument within the calling routine will not change. This procedure for passing the value of an argument to a function is known as passing by value.

Perhaps you would like to have a function that will accept any number of values and then return the average. You don't know how many arguments will be passed in to the function. One way you could make the function would be to accept a pointer to an array. Another way would be to write a function that can take any number of arguments. So you could write `avg(4, 12.2, 23.3, 33.3, 12.1);` or you could write `avg(2, 2.3, 34.4);` Some library functions can accept a variable list of arguments (such as the venerable `printf`).

To use a function with variable number of arguments, or more precisely, a function without a set number of arguments, you would use the `stdarg.h` header file. There are four parts needed: `va_list`, which stores the list of arguments, `va_start`, which initializes the list, `va_arg`, which returns the next argument in the list, and `va_end`, which cleans up the variable argument list. Whenever a function is declared to have an indeterminate number of arguments, in place of the last argument you should place an ellipsis (which looks like `'...'`), so, `int a_function(int x, ...);` would tell the compiler the function should accept however many arguments that the programmer uses, as long as it is equal to at least one, the one being the first, x.

`va_list` is like any other variable. For example, `va_list a_list;`

`va_start` is a macro which accepts two arguments, a `va_list` and the name of the variable that directly precedes the ellipsis (...). So, in the function `a_function`, to initialize `a_list` with `va_start`, you would write `va_start(a_list, x);`

`va_arg` takes a `va_list` and a variable type, and returns the next argument in the list in the form of whatever variable type it is told. It then moves down the list to the next argument. For example, `va_arg(a_list, double)` will return the next argument, assuming it exists, in the form of a double. The next time it is called, it will return the argument following the last returned number, if one exists.

To show how each of the parts works, take an example function:

```
#include <stdarg.h>
#include <stdio.h>
double average(int num, ...)
{
    va_list arguments;          //A place to store the list of arguments
    va_start(arguments, num);   //Initializing arguments to store all
    values
    int sum=0;                  // passed in after num
    for(int x=0; x<num; x++)    //Loop until all numbers are added
        sum+=va_arg(arguments, double); //Adds the next value in
                                        argument list to sum.
    va_end(arguments);         // up the list
    return sum/(double)num;     //Returns some number
    (typecast prevents truncation)
}
```

```
int main()
{
    printf("%d",average (3, 12.2, 22.3, 4.5));
    printf("%d",average (5, 3.3, 2.2, 1.1, 5.5, 3.3));
    return 0;
}
```

It isn't necessarily a good idea to use a variable argument list at all times, because the potential exists for assuming a value is of one type, while it is in fact another, such as a NULL pointer being assumed to be an integer. Consequently, variable argument lists should be used sparingly.

## 1.13 Function Prototypes

Many C compilers support a more comprehensive system for handling argument specifications in function declarations and function definitions. In particular, the proposed ANSI standard permits each of the argument data types within a function declaration to be followed by an argument name, that is:

Data-type name (type 1 arg 1, type 2 arg 2, . . . , type n arg n);

Where arg1, arg2, . . . , arg n refer to the first argument, the second argument, and so on.

Function declarations written in this form are called function prototypes. Their use is not mandatory in C. Function prototypes are desirable, however, because they further facilitate error checking between the calls to a function and the corresponding function definition.

Example:- Several function declarations (function prototypes) are shown below. Two different forms of each declaration are presented, illustrating the older method for specifying argument types described and the new method discussed in this section.

### Older Method

```
int sample (int, int);
float funct (int, float);
void f(char, long, double);
void demo (void);
```

### Newer Method

```
int sample (int a, int b);
float funct (int i, float x);
void f(char c, long j, double z);
void demo (void);
```

The names of the arguments within the function declaration need not be declared elsewhere in the program, since these are "dummy" argument names recognized only within the declaration. In practice, however, the argument names are often the same as the names of the actual arguments appearing in one of the function calls.

When the function is called, the names of the actual arguments need not be the same as the names shown in the declaration. However, the data types of the actual arguments must confirm to the data types of the arguments within the declaration.

Example: The skeletal outline of a C program is shown below.

```
main( )
{
    int a, b, c, d, i, j;
```

```
int funct (int a, int b);  
  
...  
I = funct (a, b);  
  
....  
  
J = funct(c, d);  
}
```

The functions prototype indicates that a and b are integer-type arguments. The function funct is called twice within main. In the first call, the actual arguments are the integer variable a and b. In the second call, the actual arguments are the integer variable c and d.

The ANSI standard also allows function definitions to be written more concisely, by combining formal argument declaration with the first line of the function definition. Thus, the first line in the function definition can be written as follows.

Data-type name (type 1 arg1, type 2 arg 2, , type n arg n)

Here, data-type represents the type of the quantity returned by a function, name represents the function name, and type 1, type 2,....., type n represent the data types of the arguments arg 1, arg2..... , arg n.

## 1.14 Recursion

Recursion is defined as a function calling itself. It is in some ways similar to a loop because it repeats the same code, but it requires passing in the looping variable and being more careful. Many programming languages allow it because it can simplify some tasks, and it is often more elegant than a loop.

A simple example of recursion would be:

```
void recurse()  
{  
    recurse(); //Function calls itself  
}  
int main()  
{  
    recurse(); //Sets off the recursion  
    return 0; //Rather pitiful, it will never be reached  
}
```

This program will not continue forever, however. The computer keeps function calls on a stack and once too many are called without ending, the program will terminate. Why not write a program to see how many times the function is called before the program terminates?

```
#include <stdio.h>
```

```
void recurse(int count) //The count variable is initialized by each  
function call
```



```
{
    printf("%d",count);
    recurse(count+1); //It is not necessary to increment count
//each function's variables
} //are separate (so each count will be initialized one
greater)
int main()
{
    recurse(1);          //First function call, so it starts at one
    return 0;
}
```

This simple program will show the number of times the recurse function has been called by initializing each individual function call's count variable one greater than it was previous by passing in count+1. Keep in mind, it is not a function restarting itself, it is hundreds of functions that are each unfinished with the last one calling a new recurse function.

Normally, a recursive function will have a variable that performs a similar action; one that controls when the function will finally exit. The condition where the function will not call itself is termed the base case of the function. Basically, it is an if-statement that checks some variable for a condition (such as a number being less than zero, or greater than some other number) and if that condition is true, it will not allow the function to call itself again. (Or, it could check if a certain condition is true and only then allow the function to call itself).

A quick example:

```
void doll(int size)
{
    if(size==0)//No doll can be smaller than 1 atom (10^0==1) so
    doesn't call itself
        return; //Return does not have to return something, it can
                //be used to exit a function
    doll(size-1);
    //Decrements the size variable so the next doll will be
    smaller.
}

int main()
{
    doll(10); //Starts off with a large doll (its a logarithmic
scale)
    return 0; //Finally, it will be used
}
```

This program ends when size equals one. This is a good base case, but if it is not properly set up, it is possible to have an base case that is always true (or always false).

Once a function has called itself, it will be ready to go to the next line after the call. It can still perform operations. One function you could write could print out the numbers 123456789987654321. How can you use recursion to write a function to do this? Simply have it keep incrementing a variable passed in, and then output the variable...twice, once before the function recurses, and once after...

```
void printnum(int begin)
{
    printf("%d",begin);
    if(begin<9) //The base case is when begin is greater than 9
        printnum(begin+1); //for it will not recurse after the if-
statement
    printf("%d",begin); //Outputs the second begin, after the program
has
```

```
output                                     //gone through and  
}                                     //the numbers from begin to 9.
```

This function works because it will go through and print the numbers begin to 9, and then as each printnum function terminates it will continue printing the value of begin in each function  
from 9 to begin.

## 5. Arrays

So far, we've been declaring simple variables: the declaration

```
int i;
```

Declares a single variable, named *i*, of type *int*. It is also possible to declare an *array* of several elements. The declaration

```
int a[10];
```

Declares an array, named *a*, consisting of ten elements, each of type *int*. Simply speaking, an array is a variable that can hold more than one value. You specify which of the several values you're referring to at any given time by using a numeric *subscript*. (Arrays in programming are similar to vectors or matrices in mathematics.) We can represent the array *a* above with a picture like this:



In C, arrays are *zero-based*: the ten elements of a 10-element array are numbered from 0 to 9. The subscript, which specifies a single element of an array, is simply an integer expression in square brackets. The first element of the array is *a[0]*, the second element is *a[1]*, etc. You can use these "array subscript expressions" anywhere you can use the name of a simple variable, for example:

```
a[0] = 10;
a[1] = 20;
a[2] = a[0] + a[1];
```

Notice that the subscripted array references (i.e. expressions such as *a[0]* and *a[1]*) can appear on either side of the assignment operator.

The subscript does not have to be a constant like 0 or 1; it can be any integral expression. For example, it's common to loop over all elements of an array:

```
int i;

for(i = 0; i < 10; i = i + 1)
    a[i] = 0;
```

This loop sets all ten elements of the array *a* to 0.

Arrays are a real convenience for many problems, but there is not a lot that C will do with them for you automatically. In particular, you can neither set all elements of an array at once nor assign one array to another; both of the assignments

```
a = 0;                                /* WRONG */
and
int b[10];
b = a;                                /* WRONG */
are illegal.
```

To set all of the elements of an array to some value, you must do so one by one, as in the loop example above. To copy the contents of one array to another, you must again do so one by one:

```
int b[10];
```

```
for(i = 0; i < 10; i = i + 1)  
    b[i] = a[i];
```

Remember that for an array declared  

```
int a[10];
```

There is no element `a[10]`; the topmost element is `a[9]`. This is one reason that zero-based loops are also common in C. Note that the for loop  

```
for(i = 0; i < 10; i = i + 1)
```

...

does just what you want in this case: it starts at 0, the number 10 suggests (correctly) that it goes through 10 iterations, but the less-than comparison means that the last trip through the loop has `i` set to 9. (The comparison `i <= 9` would also work, but it would be less clear and therefore poorer style.)

In the little examples so far, we've always looped over all 10 elements of the sample array `a`. It's common, however, to use an array that's bigger than necessarily needed, and to use a second variable to keep track of how many elements of the array are currently in use. For example, we might have an integer variable

```
int na;                /* number of elements of a[] in use */
```

Then, when we wanted to do something with `a` (such as print it out), the loop would run from 0 to `na`, not 10 (or whatever `a`'s size was):

```
for(i = 0; i < na; i = i + 1)  
    printf("%d\n", a[i]);
```

Naturally, we would have to ensure that `na`'s value was always less than or equal to the number of elements actually declared in `a`.

Arrays are not limited to type `int`; you can have arrays of `char` or `double` or any other type.

Here is a slightly larger example of the use of arrays. Suppose we want to investigate the behavior of rolling a pair of dice. The total roll can be anywhere from 2 to 12, and we want to count how often each roll comes up. We will use an array to keep track of the counts: `a[2]` will count how many times we've rolled 2, etc.

We'll simulate the roll of a die by calling C's random number generation function, `rand()`. Each time you call `rand()`, it returns a different, pseudo-random integer. The values that `rand()` returns typically span a large range, so we'll use C's *modulus* (or "remainder") operator `%` to produce random numbers in the range we want. The expression `rand() % 6` produces random numbers in the range 0 to 5, and `rand() % 6 + 1` produces random numbers in the range 1 to 6.

Here is the program:

```
#include <stdio.h>  
#include <stdlib.h>
```

```
main()  
{  
    int i;  
    int d1, d2;  
    int a[13]; /* uses [2..12] */  
  
    for(i = 2; i <= 12; i = i + 1)  
        a[i] = 0;  
  
    for(i = 0; i < 100; i = i + 1)  
    {  
        d1 = rand() % 6 + 1;  
        d2 = rand() % 6 + 1;
```

```
        a[d1 + d2] = a[d1 + d2] + 1;
    }

    for(i = 2; i <= 12; i = i + 1)
        printf("%d: %d\n", i, a[i]);

    return 0;
}
```

We include the header <stdlib.h> because it contains the necessary declarations for the rand() function. We declare the array of size 13 so that its highest element will be a[12]. (We're wasting a[0] and a[1]; this is no great loss.) The variables d1 and d2 contain the rolls of the two individual dice; we add them together to decide which cell of the array to increment, in the line

```
a[d1 + d2] = a[d1 + d2] + 1;
```

After 100 rolls, we print the array out. Typically (as craps players well know), we'll see mostly 7's, and relatively few 2's and 12's.

## 1.15 Array Initialization

Although it is not possible to assign to all elements of an array at once using an assignment expression, it is possible to initialize some or all elements of an array when the array is defined. The syntax looks like this:

```
int a[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

The list of values, enclosed in braces {}, separated by commas, provides the initial values for successive elements of the array.

(Under older, pre-ANSI C compilers, you could not always supply initializers for "local" arrays inside functions; you could only initialize "global" arrays, those outside of any function. Those compilers are now rare, so you shouldn't have to worry about this distinction any more. We'll talk more about local and global variables later in this chapter.)

If there are fewer initializers than elements in the array, the remaining elements are automatically initialized to 0. For example,

```
int a[10] = {0, 1, 2, 3, 4, 5, 6};
```

Would initialize a[7], a[8], and a[9] to 0. When an array definition includes an initializer, the array dimension may be omitted, and the compiler will infer the dimension from the number of initializers. For example,

```
int b[] = {10, 11, 12, 13, 14};
```

Would declare, define, and initialize an array b of 5 elements (i.e. just as if you'd typed int b[5]). Only the dimension is omitted; the brackets [] remain to indicate that b is in fact an array.

In the case of arrays of char, the initializer may be a string constant:

```
char s1[7] = "Hello,";
char s2[10] = "there,";
char s3[] = "world!";
```

As before, if the dimension is omitted, it is inferred from the size of the string initializer. (We haven't covered strings in detail yet--we'll do so in chapter 8--but it turns out that all strings in

C are terminated by a special character with the value 0. Therefore, the array s3 will be of size 7, and the explicitly-sized s1 does need to be of size at least 7. For s2, the last 4 characters in the array will all end up being this zero-value character.)

## 1.16 Arrays of Arrays ("Multidimensional" Arrays)

When we said that "Arrays are not limited to type int; you can have arrays of... any other type," we meant that more literally than you might have guessed. If you have an "array of int," it means that you have an array each of whose elements is of type int. But you can have an array each of whose elements is of type x, where x is any type you choose. In particular, you can have an array each of whose elements is another array! We can use these arrays of arrays for the same sorts of tasks as we'd use multidimensional arrays in other computer languages (or matrices in mathematics). Naturally, we are not limited to arrays of arrays, either; we could have an array of arrays of arrays, which would act like a 3-dimensional array, etc.

The declaration of an array of arrays looks like this:

```
int a2[5][7];
```

You have to read complicated declarations like these "inside out." What this one says is that a2 is an array of 5 something's, and that each of the something's is an array of 7 ints. More briefly, "a2 is an array of 5 arrays of 7 ints," or, "a2 is an array of array of int." In the declaration of a2, the brackets closest to the identifier a2 tell you what a2 first and foremost is. That's how you know it's an array of 5 arrays of size 7, not the other way around. You can think of a2 as having 5 "rows" and 7 "columns," although this interpretation is not mandatory. (You could also treat the "first" or inner subscript as "x" and the second as "y." Unless you're doing something fancy, all you have to worry about is that the subscripts when you access the array match those that you used when you declared it, as in the examples below.)

To illustrate the use of multidimensional arrays, we might fill in the elements of the above array a2 using this piece of code:

```
int i, j;
for(i = 0; i < 5; i = i + 1)
{
    for(j = 0; j < 7; j = j + 1)
        a2[i][j] = 10 * i + j;
}
```

This pair of nested loops sets a[1][2] to 12, a[4][1] to 41, etc. Since the first dimension of a2 is 5, the first subscripting index variable, i, runs from 0 to 4. Similarly, the second subscript varies from 0 to 6.

We could print a2 out (in a two-dimensional way, suggesting its structure) with a similar pair of nested loops:

```
for(i = 0; i < 5; i = i + 1)
{
    for(j = 0; j < 7; j = j + 1)
        printf("%d\t", a2[i][j]);
    printf("\n");
}
```

(The character \t in the printf string is the tab character.)

Just to see more clearly what's going on, we could make the "row" and "column" subscripts explicit by printing them, too:

```
for(j = 0; j < 7; j = j + 1)
    printf("\t%d:", j);
printf("\n");

for(i = 0; i < 5; i = i + 1)
{
    printf("%d:", i);
    for(j = 0; j < 7; j = j + 1)
        printf("\t%d", a2[i][j]);
    printf("\n");
}
```

This last fragment would print

|    | 0: | 1: | 2: | 3: | 4: | 5: | 6: |
|----|----|----|----|----|----|----|----|
| 0: | 0  | 1  | 2  | 3  | 4  | 5  | 6  |
| 1: | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| 2: | 20 | 21 | 22 | 23 | 24 | 25 | 26 |
| 3: | 30 | 31 | 32 | 33 | 34 | 35 | 36 |
| 4: | 40 | 41 | 42 | 43 | 44 | 45 | 46 |

Finally, there's no reason we have to loop over the ``rows" first and the ``columns" second; depending on what we wanted to do, we could interchange the two loops, like this:

```
for(j = 0; j < 7; j = j + 1)
{
    for(i = 0; i < 5; i = i + 1)
        printf("%d\t", a2[i][j]);
    printf("\n");
}
```

Notice that *i* is still the first subscript and it still runs from 0 to 4, and *j* is still the second subscript and it still runs from 0 to 6.

## 6. Program Structure

We haven't said so explicitly, but variables are channels of communication within a program. You set a variable to a value at one point in a program, and at another point (or points) you read the value out again. The two points may be in adjoining statements, or they may be in widely separated parts of the program.

How long does a variable last? How widely separated can the setting and fetching parts of the program be, and how long after a variable is set does it persist? Depending on the variable and how you're using it, you might want different answers to these questions.

The *visibility* of a variable determines how much of the rest of the program can access that variable. You can arrange that a variable is visible only within one part of one function, or in one function, or in one source file, or anywhere in the program. (We haven't really talked about source files yet; we'll be exploring them soon.)

Why would you want to limit the visibility of a variable? For maximum flexibility, wouldn't it be handy if all variables were potentially visible everywhere? As it happens, that arrangement would be *too* flexible: everywhere in the program, you would have to keep track of the names of all the variables declared anywhere else in the program, so that you didn't accidentally re-use one. Whenever a variable had the wrong value by mistake, you'd have to search the entire program for the bug, because any statement in the entire program could potentially have modified that variable. You would constantly be stepping all over yourself by using a common variable name like *i* in two parts of your program, and having one snippet of code accidentally overwrite the values being used by another part of the code. The communication would be sort of like an old party line--you'd always be accidentally interrupting other conversations, or having your conversations interrupted.

To avoid this confusion, we generally give variables the narrowest or smallest visibility they need. A variable declared within the braces `{ }` of a function is visible only within that function; variables declared within functions are called *local variables*. If another function somewhere else declares a local variable with the same name, it's a different variable entirely, and the two don't clash with each other.

On the other hand, a variable declared outside of any function is a *global variable*, and it is potentially visible anywhere within the program. You use global variables when you *do* want the communications path to be able to travel to any part of the program. When you declare a global variable, you will usually give it a longer, more descriptive name (not something generic like *i*) so that whenever you use it you will remember that it's the same variable everywhere.

Another word for the visibility of variables is *scope*.

How long do variables last? By default, local variables (those declared within a function) have *automatic duration*: they spring into existence when the function is called, and they (and their values) disappear when the function returns. Global variables, on the other hand, have *static duration*: they last, and the values stored in them persist, for as long as the program does. (Of course, the values can in general still be overwritten, so they don't necessarily persist forever.)

Finally, it is possible to split a function up into several source files, for easier maintenance. When several source files are combined into one program (we'll be seeing how in the next chapter) the compiler must have a way of correlating the global variables which might be used to communicate between the several source files. Furthermore, if a global variable is going to



be useful for communication, there must be exactly one of it: you wouldn't want one function in one source file to store a value in one global variable named `globalvar`, and then have another function in another source file read from a *different* global variable named `globalvar`. Therefore, a global variable should have exactly one *defining instance*, in one place in one source file. If the same variable is to be used anywhere else (i.e. in some other source file or files), the variable is declared in those other file(s) with an *external declaration*, which is not a defining instance. The external declaration says, "hey, compiler, here's the name and type of a global variable I'm going to use, but don't define it here, don't allocate space for it; it's one that's defined somewhere else, and I'm just referring to it here." If you accidentally have two distinct defining instances for a variable of the same name, the compiler (or the linker) will complain that it is "multiply defined."

It is also possible to have a variable which is global in the sense that it is declared outside of any function, but private to the one source file it's defined in. Such a variable is visible to the functions in that source file but not to any functions in any other source files, even if they try to issue a matching declaration.

You get any extra control you might need over visibility and lifetime, and you distinguish between defining instances and external declarations, by using *storage classes*. A storage class is an extra keyword at the beginning of a declaration which modifies the declaration in some way. Generally, the storage class (if any) is the first word in the declaration, preceding the type name. (Strictly speaking, this ordering has not traditionally been necessary, and you may see some code with the storage class, type name, and other parts of a declaration in an unusual order.)

We said that, by default, local variables had automatic duration. To give them static duration (so that, instead of coming and going as the function is called, they persist for as long as the function does), you precede their declaration with the static keyword:

```
static int i;
```

By default, a declaration of a global variable (especially if it specifies an initial value) is the defining instance. To make it an external declaration, of a variable which is defined somewhere else, you precede it with the keyword `extern`:

```
extern int j;
```

Finally, to arrange that a global variable is visible only within its containing source file, you precede it with the static keyword:

```
static int k;
```

Notice that the static keyword can do two different things: it adjusts the duration of a local variable from automatic to static, or it adjusts the visibility of a global variable from truly global to private-to-the-file.

We can also distinguish between file-scope global variables and truly global variables, based on the presence or absence of the static keyword.

We can also distinguish between external declarations and defining instances of global variables, based on the presence or absence of the `extern` keyword.

## 1.17 Default Initialization

The duration of a variable (whether static or automatic) also affects its default initialization. If you do not explicitly initialize them, automatic-duration variables (that is, local, non-static ones) are not guaranteed to have any particular initial value; they will typically contain garbage. It is therefore a fairly serious error to attempt to use the value of an automatic variable which has never been initialized or assigned to: the program will either work

incorrectly, or the garbage value may just happen to be "correct" such that the program appears to work correctly! However, the particular value that the garbage takes on can vary depending literally on *anything*: other parts of the program, which compiler was used, which hardware or operating system the program is running on, the time of day, the phase of the moon. (Okay, maybe the phase of the moon is a bit of an exaggeration.) So you hardly want to say that a program which uses an uninitialized variable "works"; it may seem to work, but it works for the wrong reason, and it may stop working tomorrow.

Static-duration variables (global and static local), on the other hand, are guaranteed to be initialized to 0 if you do not use an explicit initializer in the definition.

(Once upon a time, there was another distinction between the initialization of automatic vs. static variables: you could initialize *aggregate* objects, such as arrays, only if they had static duration. If your compiler complains when you try to initialize a local array, it's probably an old, pre-ANSI compiler. Modern, ANSI-compatible compilers remove this limitation, so it's no longer much of a concern.)

## 1.18 Examples

Here is an example demonstrating almost everything we've seen so far:

```
int globalvar = 1;
extern int anotherglobalvar;
static int privatevar;

f()
{
    int localvar;
    int localvar2 = 2;
    static int persistentvar;
}
```

Here we have six variables, three declared outside and three declared inside of the function `f()`.

`globalvar` is a global variable. The declaration we see is its defining instance (it happens also to include an initial value). `globalvar` can be used anywhere in this source file, and it could be used in other source files, too (as long as corresponding external declarations are issued in those other source files).

`anotherglobalvar` is a second global variable. It is *not* defined here; the defining instance for it (and its initialization) is somewhere else.

`privatevar` is a "private" global variable. It can be used anywhere within this source file, but functions in other source files cannot access it, even if they try to issue external declarations for it. (If other source files try to declare a global variable called "privatevar", they'll get their own; they won't be sharing this one.) Since it has static duration and receives no explicit initialization, `privatevar` will be initialized to 0.

`localvar` is a local variable within the function `f()`. It can be accessed only within the function `f()`. (If any other part of the program declares a variable named "localvar", that variable will be distinct from the one we're looking at here.) `localvar` is conceptually "created" each time `f()` is called, and disappears when `f()` returns. Any value which was stored in `localvar` last time `f()` was running will be lost and will not be available next time `f()` is called. Furthermore, since it has no explicit initializer, the value of `localvar` will in general be garbage each time `f()` is called.

localvar2 is also local, and everything that we said about localvar applies to it, except that since its declaration includes an explicit initializer, it will be initialized to 2 each time f() is called.

Finally, persistentvar is again local to f(), but it *does* maintain its value between calls to f(). It has static duration but no explicit initializer, so its initial value will be 0.

The defining instances and external declarations we've been looking at so far have all been of simple variables. There are also defining instances and external declarations of functions, which we'll be looking at in the next chapter.

(Also, don't worry about static variables for now if they don't make sense to you; they're a relatively sophisticated concept, which you won't need to use at first.)

The term *declaration* is a general one which encompasses defining instances and external declarations; defining instances and external declarations are two different kinds of declarations. Furthermore, either kind of declaration suffices to inform the compiler of the name and type of a particular variable (or function). If you have the defining instance of a global variable in a source file, the rest of that source file can use that variable without having to issue any external declarations. It's only in source files where the defining instance hasn't been seen that you need external declarations.

You will sometimes hear a defining instance referred to simply as a "definition," and you will sometimes hear an external declaration referred to simply as a "declaration." These usages are mildly ambiguous, in that you can't tell out of context whether a "declaration" is a generic declaration (that might be a defining instance or an external declaration) or whether it's an external declaration that specifically is not a defining instance. (Similarly, there are other constructions that can be called "definitions" in C, namely the definitions of preprocessor macros, structures, and typedefs, none of which we've met.) In these notes, we'll try to make things clear by using the unambiguous terms *defining instance* and *external declaration*. Elsewhere, you may have to look at the context to determine how the terms "definition" and "declaration" are being used.

## 7 Pointer

### 1.19 What is a pointer?

One of those things beginners in C find difficult is the concept of pointers. The purpose of this chapter is to provide an introduction to pointers and their use to beginners.

We start with a discussion of C variables in general.

A variable in a program is something with a name, the value of which can vary. The way the compiler and linker handle this is that it assigns a specific block of memory within the computer to hold the value of that variable. The size of that block depends on the range over which the variable is allowed to vary. For example, on PC's the size of an integer variable is 2 bytes, and that of a long integer is 4 bytes. In C the size of a variable type such as an integer need not be the same on all types of machines.

When we declare a variable we inform the compiler of two things, the name of the variable and the type of the variable. For example, we declare a variable of type integer with the name **k** by writing:

```
int k;
```

On seeing the "int" part of this statement the compiler sets aside 2 bytes of memory (on a PC) to hold the value of the integer. It also sets up a symbol table. In that table it adds the symbol **k** and the relative address in memory where those 2 bytes were set aside.

Thus, later if we write:

```
k = 2;
```

We expect that, at run time when this statement is executed, the value 2 will be placed in that memory location reserved for the storage of the value of **k**. In C we refer to a variable such as the integer **k** as an "object".

In a sense there are two "values" associated with the object **k**. One is the value of the integer stored there (2 in the above example) and the other the "value" of the memory location, i.e., the address of **k**. Some texts refer to these two values with the nomenclature **rvalue** (right value, pronounced "are value") and **lvalue** (left value, pronounced "el value") respectively.

In some languages, the lvalue is the value permitted on the left side of the assignment operator '=' (i.e. the address where the result of evaluation of the right side ends up). The rvalue is that which is on the right side of the assignment statement, the **2** above. Rvalues cannot be used on the left side of the assignment statement. Thus: **2 = k;** is illegal.

Actually, the above definition of "lvalue" is somewhat modified for C. According to K&R II (page 197): [1]

"An **object** is a named region of storage; an **lvalue** is an expression referring to an object."

As we become more familiar with pointers we will go into more detail on this.

Okay, now consider:

```
int j, k;
```

```
k = 2;
```

```
j = 7;  <-- line 1
```

```
k = j;  <-- line 2
```

In the above, the compiler interprets the **j** in line 1 as the address of the variable **j** (its lvalue) and creates code to copy the value 7 to that address. In line 2, however, the **j** is interpreted as its rvalue (since it is on the right hand side of the assignment operator '='). That is, here the **j** refers to the value **stored** at the memory location set aside for **j**, in this case 7. So, the 7 is copied to the address designated by the lvalue of **k**.

In all of these examples, we are using 2 byte integers so all copying of rvalues from one storage location to the other is done by copying 2 bytes. Had we been using long integers, we would be copying 4 bytes.

Now, let's say that we have a reason for wanting a variable designed to hold an lvalue (an address). The size required to hold such a value depends on the system. On older desk top computers with 64K of memory total, the address of any point in memory can be contained in 2 bytes. Computers with more memory would require more bytes to hold an address. Some computers, such as the IBM PC might require special handling to hold a segment and offset under certain circumstances. The actual size required is not too important so long as we have a way of informing the compiler that what we want to store is an address.

Such a variable is called a **pointer variable** (for reasons which hopefully will become clearer a little later). In C when we define a pointer variable we do so by preceding its name with an asterisk. In C we also give our pointer a type which, in this case, refers to the type of data stored at the address we will be storing in our pointer. For example, consider the variable declaration:

```
int *ptr;
```

**ptr** is the name of our variable (just as **k** was the name of our integer variable). The '\*' informs the compiler that we want a pointer variable, i.e. to set aside however many bytes is required to store an address in memory. The **int** says that we intend to use our pointer variable to store the address of an integer. Such a pointer is said to "point to" an integer. However, note that when we wrote **int k**; we did not give **k** a value. If this definition is made outside of any function ANSI compliant compilers will initialize it to zero. Similarly, **ptr** has no value, that is we haven't stored an address in it in the above declaration. In this case, again if the declaration is outside of any function, it is initialized to a value guaranteed in such a way that it is guaranteed to not point to any C object or function. A pointer initialized in this manner is called a "null" pointer.

The actual bit pattern used for a null pointer may or may not evaluate to zero since it depends on the specific system on which the code is developed. To make the source code compatible between various compilers on various systems, a macro is used to represent a null pointer.

That macro goes under the name NULL. Thus, setting the value of a pointer using the NULL macro, as with an assignment statement such as **ptr = NULL**, guarantees that the pointer has become a null pointer. Similarly, just as one can test for an integer value of zero, as in **if(k == 0)**, we can test for a null pointer using **if (ptr == NULL)**.

But, back to using our new variable **ptr**. Suppose now that we want to store in **ptr** the address of our integer variable **k**. To do this we use the unary **&** operator and write:

```
ptr = &k;
```

What the **&** operator does is retrieve the lvalue (address) of **k**, even though **k** is on the right hand side of the assignment operator '=', and copies that to the contents of our pointer **ptr**. Now, **ptr** is said to "point to" **k**. The "dereferencing operator" is the asterisk and it is used as follows:

```
*ptr = 7;
```

will copy 7 to the address pointed to by **ptr**. Thus if **ptr** "points to" (contains the address of) **k**, the above statement will set the value of **k** to 7. That is, when we use the '\*' this way we are referring to the value of that which **ptr** is pointing to, not the value of the pointer itself.

Similarly, we could write:

```
printf("%d\n", *ptr);
```

to print to the screen the integer value stored at the address pointed to by **ptr**.

One way to see how all this stuff fits together would be to run the following program and then review the code and the output carefully.

----- Program 1.1 -----

```
#include <stdio.h>

int j, k;
int *ptr;

int main(void)
{
    j = 1;
    k = 2;
    ptr = &k;
    printf("\n");
    printf("j has the value %d and is stored at %p\n", j, (void *)&j);
    printf("k has the value %d and is stored at %p\n", k, (void *)&k);
    printf("ptr has the value %p and is stored at %p\n", ptr, (void *)&ptr);
    printf("The value of the integer pointed to by ptr is %d\n", *ptr);

    return 0;
}
```

Note: We have yet to discuss those aspects of C which require the use of the **(void \*)** expression used here. For now, include it in your test code. We'll explain the reason behind this expression later.



## 1.20 Pointer types and Arrays

Let us consider why we need to identify the **type** of variable that a pointer points to, as in:  
`int *ptr;`

One reason for doing this is so that later, once `ptr` "points to" something, if we write:  
`*ptr = 2;`

the compiler will know how many bytes to copy into that memory location pointed to by **ptr**. If **ptr** was declared as pointing to an integer, 2 bytes would be copied, if a long, 4 bytes would be copied. Similarly for floats and doubles the appropriate number will be copied. But, defining the type that the pointer points to permits a number of other interesting ways a compiler can interpret code. For example, consider a block in memory consisting of ten integers in a row. That is, 20 bytes of memory are set aside to hold 10 integers.

Now, let's say we point our integer pointer **ptr** at the first of these integers. Furthermore let's say that integer is located at memory location 100 (decimal). Now what will happen when we write:  
`ptr + 1;`

Because the compiler "knows" this is a pointer (i.e. its value is an address) and that it points to an integer (its current address, 100, is the address of an integer), it adds 2 to **ptr** instead of 1, so the pointer "points to" the **next integer**, at memory location 102. Similarly, were the **ptr** declared as a pointer to a long, it would add 4 to it instead of 1. The same goes for other data types such as floats, doubles, or even user defined data types such as structures. This is obviously not the same kind of "addition" that we normally think of. In C it is referred to as addition using "pointer arithmetic".

Similarly, since `++ptr` and `ptr++` are both equivalent to `ptr + 1` (though the point in the program when **ptr** is incremented may be different), incrementing a pointer using the unary ++ operator, either pre- or post-, increments the address it stores by the amount `sizeof(type)` where "type" is the type of the object pointed to. (i.e. 2 for an integer, 4 for a long, etc.). Since a block of 10 integers located contiguously in memory is, by definition, an array of integers, this brings up an interesting relationship between arrays and pointers.

Consider the following:

```
int my_array[] = {1,23,17,4,-5,100};
```

Here we have an array containing 6 integers. We refer to each of these integers by means of a subscript to **my\_array**, i.e. using **my\_array[0]** through **my\_array[5]**. But, we could alternatively access them via a pointer as follows:

```
int *ptr;  
ptr = &my_array[0];    /* point our pointer at the first  
                        integer in our array */
```

And then we could print out our array either using the array notation or by dereferencing our pointer. The following code illustrates this:

----- Program 2.1 -----

```
#include <stdio.h>  
  
int my_array[] = {1,23,17,4,-5,100};  
int *ptr;  
  
int main(void)  
{  
    int i;  
    ptr = &my_array[0];    /* point our pointer to the first  
                           element of the array */
```

```
printf("\n\n");
for (i = 0; i < 6; i++)
{
    printf("my_array[%d] = %d ",i,my_array[i]); /*<-- A */
    printf("ptr + %d = %d\n",i, *(ptr + i));    /*<-- B */
}
return 0;
}
```

Compile and run the above program and carefully note lines A and B and that the program prints out the same values in either case. Also observe how we dereferenced our pointer in line B, i.e. we first added i to it and then dereferenced the new pointer. Change line B to read:

```
printf("ptr + %d = %d\n",i, *ptr++);
and run it again... then change it to:
printf("ptr + %d = %d\n",i, *(++ptr));
and try once more. Each time try and predict the outcome and carefully look at the actual outcome.
```

In C, the standard states that wherever we might use **&var\_name[0]** we can replace that with **var\_name**, thus in our code where we wrote:

```
ptr = &my_array[0];
we can write:
ptr = my_array;
to achieve the same result.
```

This leads many texts to state that the name of an array is a pointer or "the name of the array is the address of first element in the array".

For example, while we can write

```
ptr = my_array;
we cannot write
my_array = ptr;
```

The reason is that while **ptr** is a variable, **my\_array** is a constant. That is, the location at which the first element of **my\_array** will be stored cannot be changed once **my\_array[]** has been declared.

Earlier when discussing the term "lvalue"

"An **object** is a named region of storage; an **lvalue** is an expression referring to an object".

This raises an interesting problem. Since **my\_array** is a named region of storage, why is **my\_array** in the above assignment statement not an lvalue? To resolve this problem, some refer to **my\_array** as an "unmodifiable lvalue".

Modify the example program above by changing

```
ptr = &my_array[0];
to
```

```
ptr = my_array;
and run it again to verify the results are identical.
```

Now, let's delve a little further into the difference between the names **ptr** and **my\_array** as used above. Some writers will refer to an array's name as a **constant** pointer. What do we mean by that? Well, to understand the term "constant" in this sense, let's go back to our definition of the term "variable". When we declare a variable we set aside a spot in memory to hold the value of the appropriate type. Once that is done the name of the variable can be interpreted in one of two ways. When used on the left side of the assignment operator, the compiler interprets it as the memory location to which to move that value resulting from evaluation of the right side of the assignment operator. But, when used on the right side of the assignment operator, the name of a variable is interpreted to mean the contents stored at that memory address set aside to hold the value of that variable.

With that in mind, let's now consider the simplest of constants, as in:

```
int i, k;
i = 2;
```



Here, while **i** is a variable and then occupies space in the data portion of memory, **2** is a constant and, as such, instead of setting aside memory in the data segment, it is imbedded directly in the code segment of memory. That is, while writing something like **k = i;** tells the compiler to create code which at run time will look at memory location **&i** to determine the value to be moved to **k**, code created by **i = 2;** simply puts the **2** in the code and there is no referencing of the data segment. That is, both **k** and **i** are objects, but **2** is not an object.

Similarly, in the above, since **my\_array** is a constant, once the compiler establishes where the array itself is to be stored, it "knows" the address of **my\_array[0]** and on seeing:

```
ptr = my_array;
```

it simply uses this address as a constant in the code segment and there is no referencing of the data segment beyond that.

This might be a good place explain further the use of the **(void \*)** expression used in Program

As we have seen we can have pointers of various types. So far we have discussed pointers to integers and pointers to characters. In coming chapters we will be learning about pointers to structures and even pointer to pointers.

Also we have learned that on different systems the size of a pointer can vary. As it turns out it is also possible that the size of a pointer can vary depending on the data type of the object to which it points. Thus, as with integers where you can run into trouble attempting to assign a long integer to a variable of type short integer, you can run into trouble attempting to assign the values of pointers of various types to pointer variables of other types.

To minimize this problem, C provides for a pointer of type void. We can declare such a pointer by writing:

```
void *vptr;
```

A void pointer is sort of a generic pointer. For example, while C will not permit the comparison of a pointer to type integer with a pointer to type character, for example, either of these can be compared to a void pointer. Of course, as with other variables, casts can be used to convert from one type of pointer to another under the proper circumstances. In Program 1.1. of cast the pointers to integers into void pointers to make them compatible with the **%p** conversion specification.

let's move on to the relationship between **pointers, character arrays, and strings.**



## 1.21 Pointers and Strings

The study of strings is useful to further tie in the relationship between pointers and arrays. It also makes it easy to illustrate how some of the standard C string functions can be implemented. Finally it illustrates how and when pointers can and should be passed to functions.

In C, strings are arrays of characters. This is not necessarily true in other languages. In BASIC, Pascal, Fortran and various other languages, a string has its own data type. But in C it does not. In C a string is an array of characters terminated with a binary zero character (written as '\0'). To start off our discussion we will write some code which, while preferred for illustrative purposes, you would probably never write in an actual program. Consider, for example:

```
char my_string[40];

my_string[0] = 'T';
my_string[1] = 'e';
my_string[2] = 'd';
my_string[3] = '\0';
```

While one would never build a string like this, the end result is a string in that it is an array of characters **terminated with a nul character**. By definition, in C, a string is an array of characters terminated with the nul character. Be aware that "nul" is **not** the same as "NULL". The nul refers to a zero as defined by the escape sequence '\0'. That is it occupies one byte of memory. NULL, on the other hand, is the name of the macro used to initialize null pointers. NULL is #defined in a header file in your C compiler, nul may not be #defined at all.

Since writing the above code would be very time consuming, C permits two alternate ways of achieving the same thing. First, one might write:

```
char my_string[40] = {'T', 'e', 'd', '\0'};
```

But this also takes more typing than is convenient. So, C permits:

```
char my_string[40] = "Ted";
```

When the double quotes are used, instead of the single quotes as was done in the previous examples, the nul character ( '\0' ) is automatically appended to the end of the string.

In all of the above cases, the same thing happens. The compiler sets aside a contiguous block of memory 40 bytes long to hold characters and initialized it such that the first 4 characters are **Ted\0**.

Now, consider the following program:

```
/* Program 3.1 */

#include <stdio.h>

char strA[80] = "A string to be used for demonstration purposes";
char strB[80];

int main(void)
{
    char *pA; /* a pointer to type character */
    char *pB; /* another pointer to type character */
    puts(strA); /* show string A */
    pA = strA; /* point pA at string A */
    puts(pA); /* show what pA is pointing to */
    pB = strB; /* point pB at string B */
    putchar('\n'); /* move down one line on the screen */
    while(*pA != '\0') /* line A (see text) */
    {
```

```
        *pB++ = *pA++; /* line B (see text) */
    }
    *pB = '\0';      /* line C (see text) */
    puts(strB);      /* show strB on screen */
    return 0;
}
```

----- end program 3.1 -----

In the above we start out by defining two character arrays of 80 characters each. Since these are globally defined, they are initialized to all '\0's first. Then, **strA** has the first 42 characters initialized to the string in quotes.

Now, moving into the code, we declare two character pointers and show the string on the screen. We then "point" the pointer **pA** at **strA**. That is, by means of the assignment statement we copy the address of **strA[0]** into our variable **pA**. We now use **puts()** to show that which is pointed to by **pA** on the screen. Consider here that the function prototype for **puts()** is:

```
int puts(const char *s);
```

For the moment, ignore the **const**. The parameter passed to **puts()** is a pointer, that is the **value** of a pointer (since all parameters in C are passed by value), and the value of a pointer is the address to which it points, or, simply, an address. Thus when we write **puts(strA)**; as we have seen, we are passing the address of **strA[0]**.

Similarly, when we write **puts(pA)**; we are passing the same address, since we have set **pA = strA**;

Given that, follow the code down to the **while()** statement on line A. Line A states: While the character pointed to by **pA** (i.e. **\*pA**) is not a nul character (i.e. the terminating '\0'), do the following:

Line B states: copy the character pointed to by **pA** to the space pointed to by **pB**, then increment **pA** so it points to the next character and **pB** so it points to the next space.

When we have copied the last character, **pA** now points to the terminating nul character and the loop ends. However, we have not copied the nul character. And, by definition a string in C **must** be nul terminated. So, we add the nul character with line C.

It is very educational to run this program with your debugger while watching **strA**, **strB**, **pA** and **pB** and single stepping through the program. It is even more educational if instead of simply defining **strB[]** as has been done above, initialize it also with something like:

```
strB[80] = "123456789012345678901234567890123456789012345678901234567890"
```

where the number of digits used is greater than the length of **strA** and then repeat the single stepping procedure while watching the above variables.

Getting back to the prototype for **puts()** for a moment, the "const" used as a parameter modifier informs the user that the function will not modify the string pointed to by **s**, i.e. it will treat that string as a constant.

Of course, what the above program illustrates is a simple way of copying a string. After playing with the above until you have a good understanding of what is happening, we can proceed to creating our own replacement for the standard **strcpy()** that comes with C. It might look like:

```
char *my_strcpy(char *destination, char *source)
{
```

```
char *p = destination;
while (*source != '\0')
{
    *p++ = *source++;
}
*p = '\0';
return destination;
}
```

Again, the function is designed to accept the values of two character pointers, i.e. addresses, and thus in the previous program we could write:

```
int main(void)
{
    my_strcpy(strB, strA);
    puts(strB);
}
```

The form used in standard C which would have the prototype:

```
char *my_strcpy(char *destination, const char *source);
```

Here the "const" modifier is used to assure the user that the function will not modify the contents pointed to by the source pointer. You can prove this by modifying the function above, and its prototype, to include the "const" modifier as shown. Then, within the function you can add a statement which attempts to change the contents of that which is pointed to by source, such as:

```
*source = 'X';
```

Which would normally change the first character of the string to an X. The const modifier should cause your compiler to catch this as an error.

Now, let's consider some of the things the above examples have shown us. First off, consider the fact that **\*ptr++** is to be interpreted as returning the value pointed to by **ptr** and then incrementing the pointer value. This has to do with the precedence of the operators. Were we to write **(\*ptr)++** we would increment, not the pointer, but that which the pointer points to! i.e. if used on the first character of the above example string the 'T' would be incremented to a 'U'. You can write some simple example code to illustrate this.

Recall again that a string is nothing more than an array of characters, with the last character being a **"\0"**. What we have done above is deal with copying an array. It happens to be an array of characters but the technique could be applied to an array of integers, doubles, etc. In those cases, however, we would not be dealing with strings and hence the end of the array would not be marked with a special value like the nul character. We could implement a version that relied on a special value to identify the end. For example, we could copy an array of positive integers by marking the end with a negative integer. On the other hand, it is more usual that when we write a function to copy an array of items other than strings we pass the function the number of items to be copied as well as the address of the array, e.g. something like the following prototype might indicate:

```
void int_copy(int *ptrA, int *ptrB, int nbr);
```

Where **nbr** is the number of integers to be copied. You might want to play with this idea and create an array of integers and see if you can write the function **int\_copy()** and make it work. This permits using functions to manipulate large arrays. For example, if we have an array of 5000 integers that we want to manipulate with a function, we need only pass to that function the address of the array (and any auxiliary information such as **nbr** above, depending on what we are doing). The array itself does **not** get passed, i.e. the whole array is not copied and put on the stack before calling the function, only its address is sent.

This is different from passing, say an integer, to a function. When we pass an integer we make a copy of the integer, i.e. get its value and put it on the stack. Within the function any manipulation of the value passed can in no way effect the original integer. But, with arrays

and pointers we can pass the address of the variable and hence manipulate the values of the original variables.



## 1.22 More on Strings

Now copying of strings but in a different light. Consider the following function:

```
char *my_strcpy(char dest[], char source[])
{
    int i = 0;
    while (source[i] != '\0')
    {
        dest[i] = source[i];
        i++;
    }
    dest[i] = '\0';
    return dest;
}
```

Recall that strings are arrays of characters. Here we have chosen to use array notation instead of pointer notation to do the actual copying. The results are the same, i.e. the string gets copied using this notation just as accurately as it did before. This raises some interesting points which we will discuss.

Since parameters are passed by value, in both the passing of a character pointer or the name of the array as above, what actually gets passed is the address of the first element of each array. Thus, the numerical value of the parameter passed is the same whether we use a character pointer or an array name as a parameter. This would tend to imply that somehow **source[i]** is the same as **\*(p+i)**.

In fact, this is true, i.e wherever one writes **a[i]** it can be replaced with **\*(a + i)** without any problems. In fact, the compiler will create the same code in either case. Thus we see that pointer arithmetic is the same thing as array indexing. Either syntax produces the same result. This is NOT saying that pointers and arrays are the same thing, they are not. We are only saying that to identify a given element of an array we have the choice of two syntaxes, one using array indexing and the other using pointer arithmetic, which yield identical results.

Now, looking at this last expression, part of it.. **(a + i)**, is a simple addition using the **+** operator and the rules of C state that such an expression is commutative. That is **(a + i)** is identical to **(i + a)**. Thus we could write **\*(i + a)** just as easily as **\*(a + i)**.

But **\*(i + a)** could have come from **i[a]** ! From all of this comes the curious truth that if:

```
char a[20];
int i;
writing
a[3] = 'x';
is the same as writing
3[a] = 'x';
```

Try it! Set up an array of characters, integers or longs, etc. and assigned the 3rd or 4th element a value using the conventional approach and then print out that value to be sure you have that working. Then reverse the array notation as I have done above. A good compiler will not balk and the results will be identical.

Now, looking at our function above, when we write:

```
dest[i] = source[i];
```

due to the fact that array indexing and pointer arithmetic yield identical results, we can write this as:

```
*(dest + i) = *(source + i);
```

But, this takes 2 additions for each value taken on by i. Additions, generally speaking, take more time than incrementations (such as those done using the **++** operator as in **i++**). This may not be true in modern optimizing compilers, but one can never be sure. Thus, the pointer version may be a bit faster than the array version.

Another way to speed up the pointer version would be to change:

```
    while (*source != '\0')  
to simply  
    while (*source)
```

since the value within the parenthesis will go to zero (FALSE) at the same time in either case. We will come back to strings and their manipulation through pointers in a future chapter. For now, let's move on and discuss structures for a bit.

## 8. Structures in C

### 1.23 Simple Structures

A *structure* is a collection of one or more variables grouped under a single name for easy manipulation. The variables in a structure, unlike those in an array, can be of different variable types. A structure can contain any of C's data types, including arrays and other structures. Each variable within a structure is called a *member* of the structure. The next section shows a simple example.

Note that the C language makes no distinction between simple and complex structures, but it's easier to explain structures in this way.

### 1.24 Defining and Declaring Structures

If you're writing a graphics program, your code needs to deal with the coordinates of points on the screen. Screen coordinates are written as an x value, giving the horizontal position, and a y value, giving the vertical position. You can define a structure named `coord` that contains both the x and y values of a screen location as follows:

```
struct coord {  
    int x;  
    int y;  
};
```

The `struct` keyword, which identifies the beginning of a structure definition, must be followed immediately by the structure name, or *tag* (which follows the same rules as other C variable names). Within the braces following the structure name is a list of the structure's member variables. You must give a variable type and name for each member.

The preceding statements define a structure type named `coord` that contains two integer variables, x and y. They do not, however, actually create any instances of the structure `coord`. In other words, they don't *declare* (set aside storage for) any structures. There are two ways to declare structures. One is to follow the structure definition with a list of one or more variable names, as is done here:

```
struct coord {  
    int x;  
    int y;  
} first, second;
```

These statements define the structure type `coord` and declare two structures, `first` and `second`, of type `coord`. `first` and `second` are each *instances* of type `coord`; `first` contains two integer members named x and y, and so does `second`.

This method of declaring structures combines the declaration with the definition. The second method is to declare structure variables at a different location in your source code from the definition. The following statements also declare two instances of type `coord`:

```
struct coord {  
    int x;  
    int y;  
};  
/* Additional code may go here */  
struct coord first, second;
```

## Accessing Structure Members



Individual structure members can be used like other variables of the same type. Structure members are accessed using the *structure member operator* (`.`), also called the *dot operator*, between the structure name and the member name. Thus, to have the structure named `first` refer to a screen location that has coordinates `x=50`, `y=100`, you could write

```
first.x = 50;  
first.y = 100;
```

To display the screen locations stored in the structure `second`, you could write  
`printf("%d,%d", second.x, second.y);`

At this point, you might be wondering what the advantage is of using structures rather than individual variables. One major advantage is that you can copy information between structures of the same type with a simple equation statement. Continuing with the preceding example, the statement

```
first = second;  
is equivalent to this statement:  
first.x = second.x;  
first.y = second.y;
```

When your program uses complex structures with many members, this notation can be a great time-saver. Other advantages of structures will become apparent as you learn some advanced techniques. In general, you'll find structures to be useful whenever information of different variable types needs to be treated as a group. For example, in a mailing list database, each entry could be a structure, and each piece of information (name, address, city, and so on) could be a structure member.

The `struct` Keyword

```
struct tag {  
    structure_member(s);  
    /* additional statements may go here */  
} instance;
```

The `struct` keyword is used to declare structures. A structure is a collection of one or more variables (*structure\_members*) that have been grouped under a single name for easy manipulation. The variables don't have to be of the same variable type, nor do they have to be simple variables. Structures also can hold arrays, pointers, and other structures.

The keyword `struct` identifies the beginning of a structure definition. It's followed by a tag that is the name given to the structure. Following the tag are the structure members, enclosed in braces. An *instance*, the actual declaration of a structure, can also be defined. If you define the structure without the instance, it's just a template that can be used later in a program to declare structures. Here is a template's format:

```
struct tag {  
    structure_member(s);  
    /* additional statements may go here */  
};
```

To use the template, you use the following format:

```
struct tag instance;
```

To use this format, you must have previously declared a structure with the given tag.

### Example 1

```
/* Declare a structure template called SSN */
```

```
struct SSN {  
    int first_three;  
    char dash1;  
    int second_two;  
    char dash2;  
    int last_four;  
}
```

```
/* Use the structure template */  
struct SSN customer_ssn;
```

### Example 2

```
/* Declare a structure and instance together */  
struct date {  
    char month[2];  
    char day[2];  
    char year[4];  
} current_date;
```

### Example 3

```
/* Declare and initialize a structure */  
struct time {  
    int hours;  
    int minutes;  
    int seconds;  
} time_of_birth = { 8, 45, 0 };  
More-Complex Structures
```

Now that you have been introduced to simple structures, you can get to the more interesting and complex types of structures. These are structures that contain other structures as members and structures that contain arrays as members.

## Structures That Contain Structures

As mentioned earlier, a C structure can contain any of C's data types. For example, a structure can contain other structures. The previous example can be extended to illustrate this.

Assume that your graphics program needs to deal with rectangles. A rectangle can be defined by the coordinates of two diagonally opposite corners. You've already seen how to define a structure that can hold the two coordinates required for a single point. You need two such structures to define a rectangle. You can define a structure as follows (assuming, of course, that you have already defined the type coord structure):

```
struct rectangle {  
    struct coord topleft;  
    struct coord bottomrt;  
};
```

This statement defines a structure of type rectangle that contains two structures of type coord. These two type coord structures are named topleft and bottomrt.

The preceding statement defines only the type rectangle structure. To declare a structure, you must then include a statement such as

```
struct rectangle mybox;
```

You could have combined the definition and declaration, as you did before for the type coord:

```
struct rectangle {  
    struct coord topleft;  
    struct coord bottomrt;  
} mybox;
```

To access the actual data locations (the type int members), you must apply the member operator (.) twice. Thus, the expression

mybox.topleft.x

refers to the x member of the topleft member of the type rectangle structure named mybox.

To define a rectangle with coordinates (0,10),(100,200), you would write

```
mybox.topleft.x = 0;
mybox.topleft.y = 10;
mybox.bottomrt.x = 100;
mybox.bottomrt.y = 200;
```

Maybe this is getting a bit confusing. You might understand better if you look at Figure 11.1, which shows the relationship between the type rectangle structure, the two type coord structures it contains, and the two type int variables each type coord structure contains. These structures are named as in the preceding example.

Let's look at an example of using structures that contain other structures. Listing 11.1 takes input from the user for the coordinates of a rectangle and then calculates and displays the rectangle's area. Note the program's assumptions, given in comments near the start of the program (lines 3 through 8).

## ***The relationship between a structure, structures within a structure, and the structure members.***

Listing 11.1. A demonstration of structures that contain other structures.

```
1: /* Demonstrates structures that contain other structures. */
2:
3: /* Receives input for corner coordinates of a rectangle and
4:  calculates the area. Assumes that the y coordinate of the
5:  upper-left corner is greater than the y coordinate of the
6:  lower-right corner, that the x coordinate of the lower-
7:  right corner is greater than the x coordinate of the upper-
8:  left corner, and that all coordinates are positive. */
9:
10: #include <stdio.h>
11:
12: int length, width;
13: long area;
14:
15: struct coord{
16:     int x;
17:     int y;
18: };
19:
20: struct rectangle{
21:     struct coord topleft;
22:     struct coord bottomrt;
23: } mybox;
24:
25: main()
26: {
27:     /* Input the coordinates */
28:
29:     printf("\nEnter the top left x coordinate: ");
30:     scanf("%d", &mybox.topleft.x);
31:
32:     printf("\nEnter the top left y coordinate: ");
33:     scanf("%d", &mybox.topleft.y);
34:
35:     printf("\nEnter the bottom right x coordinate: ");
36:     scanf("%d", &mybox.bottomrt.x);
37:
38:     printf("\nEnter the bottom right y coordinate: ");
39:     scanf("%d", &mybox.bottomrt.y);
40:
41:     /* Calculate the length and width */
42:
```

```
43: width = mybox.bottomrt.x - mybox.topleft.x;
44: length = mybox.bottomrt.y - mybox.topleft.y;
45:
46: /* Calculate and display the area */
47:
48: area = width * length;
49: printf("\nThe area is %ld units.\n", area);
50:
51: return 0;
52: }
```

Enter the top left x coordinate: 1  
Enter the top left y coordinate: 1  
Enter the bottom right x coordinate: 10  
Enter the bottom right y coordinate: 10  
The area is 81 units.

**ANALYSIS:** The coord structure is defined in lines 15 through 18 with its two members, x and y. Lines 20 through 23 declare and define an instance, called mybox, of the rectangle structure. The two members of the rectangle structure are topleft and bottomrt, both structures of type coord.

Lines 29 through 39 fill in the values in the mybox structure. At first it might seem that there are only two values to fill, because mybox has only two members. However, each of mybox's members has its own members. topleft and bottomrt have two members each, x and y from the coord structure. This gives a total of four members to be filled. After the members are filled with values, the area is calculated using the structure and member names. When using the x and y values, you must include the structure instance name. Because x and y are in a structure within a structure, you must use the instance names of both structures--mybox.bottomrt.x, mybox.bottomrt.y, mybox.topleft.x, and mybox.topleft.y--in the calculations. C places no limits on the nesting of structures. While memory allows, you can define structures that contain structures that contain structures that contain structures--well, you get the idea! Of course, there's a limit beyond which nesting becomes unproductive. Rarely are more than three levels of nesting used in any C program.

## 1.25 Structures That Contain Arrays

You can define a structure that contains one or more arrays as members. The array can be of any C data type (int, char, and so on). For example, the statements

```
struct data{
    int x[4];
    char y[10];
};
```

define a structure of type data that contains a four-element integer array member named x and a 10-element character array member named y. You can then declare a structure named record of type data as follows:

```
struct data record;
```

The organization of this structure is shown in Figure 11.2. Note that, in this figure, the elements of array x take up twice as much space as the elements of array y. This is because a type int typically requires two bytes of storage, whereas a type char usually requires only one byte

### ***The organization of a structure that contains arrays as members.***

You access individual elements of arrays that are structure members using a combination of the member operator and array subscripts:

```
record.x[2] = 100;  
record.y[1] = 'x';
```

You probably remember that character arrays are most frequently used to store strings. You should also remember that the name of an array, without brackets, is a pointer to the array. Because this holds true for arrays that are structure members, the expression

`record.y`

is a pointer to the first element of array `y[]` in the structure `record`. Therefore, you could print the contents of `y[]` on-screen using the statement

```
puts(record.y)
```

Another example. Listing 11.2 uses a structure that contains a type float variable and two type char arrays.

Listing 11.2. A structure that contains array members.

```
1: /* Demonstrates a structure that has array members. */  
2:  
3: #include <stdio.h>  
4:  
5: /* Define and declare a structure to hold the data. */  
6: /* It contains one float variable and two char arrays. */  
7:  
8: struct data{  
9:     float amount;  
10:    char fname[30];  
11:    char lname[30];  
12: } rec;  
13:  
14: main()  
15: {  
16:     /* Input the data from the keyboard. */  
17:  
18:     printf("Enter the donor's first and last names,\n");  
19:     printf("separated by a space: ");  
20:     scanf("%s %s", rec.fname, rec.lname);  
21:  
22:     printf("\nEnter the donation amount: ");  
23:     scanf("%f", &rec.amount);  
24:  
25:     /* Display the information. */  
26:     /* Note: %.2f specifies a floating-point value */  
27:     /* to be displayed with two digits to the right */  
28:     /* of the decimal point. */  
29:  
30:     /* Display the data on the screen. */  
31:  
32:     printf("\nDonor %s %s gave $%.2f.\n", rec.fname, rec.lname,  
33:           rec.amount);  
34:  
35:     return 0;  
36: }
```

Enter the donor's first and last names,  
separated by a space: **Bradley Jones**

Enter the donation amount: **1000.00**

Donor Bradley Jones gave \$1000.00.

**ANALYSIS:** This program includes a structure that contains array members named `fname[30]` and `lname[30]`. Both are arrays of characters that hold a person's first name and last name, respectively. The structure declared in lines 8 through 12 is called `data`. It contains the `fname` and `lname` character arrays with a type float variable called `amount`. This structure is ideal for holding a person's name (in two parts, first name and last name) and a value, such as the amount the person donated to a charitable organization.

An instance of the array, called `rec`, has also been declared in line 12. The rest of the program uses `rec` to get values from the user (lines 18 through 23) and then print them (lines 32 and 33).

## Arrays of Structures

If you can have structures that contain arrays, can you also have arrays of structures? You bet you can! In fact, arrays of structures are very powerful programming tools. Here's how it's done.

You've seen how a structure definition can be tailored to fit the data your program needs to work with. Usually a program needs to work with more than one instance of the data. For example, in a program to maintain a list of phone numbers, you can define a structure to hold each person's name and number:

```
struct entry{
    char fname[10];
    char lname[12];
    char phone[8];
};
```

A phone list must hold many entries, however, so a single instance of the `entry` structure isn't of much use. What you need is an array of structures of type `entry`. After the structure has been defined, you can declare an array as follows:

```
struct entry list[1000];
```

This statement declares an array named `list` that contains 1,000 elements. Each element is a structure of type `entry` and is identified by subscript like other array element types. Each of these structures has three elements, each of which is an array of type `char`. This entire complex creation is diagrammed in Figure 11.3.

### ***The organization of the array of structures defined in the text.***

When you have declared the array of structures, you can manipulate the data in many ways. For example, to assign the data in one array element to another array element, you would write

```
list[1] = list[5];
```

This statement assigns to each member of the structure `list[1]` the values contained in the corresponding members of `list[5]`. You can also move data between individual structure members. The statement

```
strcpy(list[1].phone, list[5].phone);
```

copies the string in `list[5].phone` to `list[1].phone`. (The `strcpy()` library function copies one string to another string. If you want to, you can also move data between individual elements of the structure member arrays:

```
list[5].phone[1] = list[2].phone[3];
```

This statement moves the second character of `list[5]`'s phone number to the fourth position in `list[2]`'s phone number. (Don't forget that subscripts start at offset 0.)

Listing 11.3 demonstrates the use of arrays of structures. Moreover, it demonstrates arrays of structures that contain arrays as members.

#### **Arrays of structures.**

```
1: /* Demonstrates using arrays of structures. */
2:
3: #include <stdio.h>
4:
5: /* Define a structure to hold entries. */
6:
```

```
7: struct entry {
8:     char fname[20];
9:     char lname[20];
10:    char phone[10];
11: };
12:
13: /* Declare an array of structures. */
14:
15: struct entry list[4];
16:
17: int i;
18:
19: main()
20: {
21:
22:     /* Loop to input data for four people. */
23:
24:     for (i = 0; i < 4; i++)
25:     {
26:         printf("\nEnter first name: ");
27:         scanf("%s", list[i].fname);
28:         printf("Enter last name: ");
29:         scanf("%s", list[i].lname);
30:         printf("Enter phone in 123-4567 format: ");
31:         scanf("%s", list[i].phone);
32:     }
33:
34:     /* Print two blank lines. */
35:
36:     printf("\n\n");
37:
38:     /* Loop to display data. */
39:
40:     for (i = 0; i < 4; i++)
41:     {
42:         printf("Name: %s %s", list[i].fname, list[i].lname);
43:         printf("\t\tPhone: %s\n", list[i].phone);
44:     }
45:
46:     return 0;
47: }
```

Enter first name: **Bradley**  
Enter last name: **Jones**  
Enter phone in 123-4567 format: **555-1212**  
Enter first name: **Peter**  
Enter last name: **Aitken**  
Enter phone in 123-4567 format: **555-3434**  
Enter first name: **Melissa**  
Enter last name: **Jones**  
Enter phone in 123-4567 format: **555-1212**  
Enter first name: **Deanna**  
Enter last name: **Townsend**  
Enter phone in 123-4567 format: **555-1234**  
Name: Bradley Jones            Phone: 555-1212  
Name: Peter Aitken            Phone: 555-3434  
Name: Melissa Jones           Phone: 555-1212  
Name: Deanna Townsend        Phone: 555-1234

**ANALYSIS:** This listing follows the same general format as most of the other listings. It starts with the comment in line 1 and, for the input/output functions, the #include file STDIO.H in line 3. Lines 7 through 11 define a template structure called entry that contains three character arrays: fname, lname, and phone. Line 15 uses the template to define an array of four entry structure variables called list. Line 17 defines a variable of type int to be used as a counter throughout the program. main() starts in line 19. The first function of main() is to perform a loop four times with a for statement. This loop is used to get information for the array of structures. This can be seen in lines 24 through 32. Line 36 provides a break from the input before starting with the output. It prints two new lines in a manner that shouldn't be new to you. Lines 40 through 44 display the data that the user entered in the preceding step. The values in the array of structures are printed with the subscripted array name followed by the member operator (.) and the structure member name.

Familiarize yourself with the techniques used in Listing 11.3. Many real-world programming tasks are best accomplished by using arrays of structures containing arrays as members.

---

## 1.26 Initializing Structures

Like other C variable types, structures can be initialized when they're declared. This procedure is similar to that for initializing arrays. The structure declaration is followed by an equal sign and a list of initialization values separated by commas and enclosed in braces. For example, look at the following statements:

```
1: struct sale {
2:   char customer[20];
3:   char item[20];
4:   float amount;
5: } mysale = { "Acme Industries",
6:             "Left-handed widget",
7:             1000.00
8:   };
```

When these statements are executed, they perform the following actions:

1. Define a structure type named sale (lines 1 through 5).
2. Declare an instance of structure type sale named mysale (line 5).
3. Initialize the structure member mysale.customer to the string "Acme Industries" (line 5).
4. Initialize the structure member mysale.item to the string "Left-handed widget" (line 6).
5. Initialize the structure member mysale.amount to the value 1000.00 (line 7).

For a structure that contains structures as members, list the initialization values in order. They are placed in the structure members in the order in which the members are listed in the structure definition. Here's an example that expands on the previous one:

```
1: struct customer {
2:   char firm[20];
3:   char contact[25];
4: }
5:
6: struct sale {
7:   struct customer buyer;
8:   char item[20];
9:   float amount;
10: } mysale = { { "Acme Industries", "George Adams"},
11:             "Left-handed widget",
12:             1000.00
13:   };
```

These statements perform the following initializations:

1. The structure member mysale.buyer.firm is initialized to the string "Acme Industries" (line 10).
2. The structure member mysale.buyer.contact is initialized to the string "George Adams" (line 10).



3. The structure member `mysale.item` is initialized to the string "Left-handed widget" (line 11).  
4. The structure member `mysale.amount` is initialized to the amount 1000.00 (line 12).  
You can also initialize arrays of structures. The initialization data that you supply is applied, in order, to the structures in the array. For example, to declare an array of structures of type `sale` and initialize the first two array elements (that is, the first two structures), you could write

```
1: struct customer {  
2:     char firm[20];  
3:     char contact[25];  
4: };  
5:  
6: struct sale {  
7:     struct customer buyer;  
8:     char item[20];  
9:     float amount;  
10: };  
11:  
12:  
13: struct sale y1990[100] = {  
14:     { { "Acme Industries", "George Adams",  
15:         "Left-handed widget",  
16:         1000.00  
17:     }  
18:     { { "Wilson & Co.", "Ed Wilson",  
19:         "Type 12 gizmo",  
20:         290.00  
21:     }  
22: };
```

This is what occurs in this code:

1. The structure member `y1990[0].buyer.firm` is initialized to the string "Acme Industries" (line 14).
2. The structure member `y1990[0].buyer.contact` is initialized to the string "George Adams" (line 14).
3. The structure member `y1990[0].item` is initialized to the string "Left-handed widget" (line 15).
4. The structure member `y1990[0].amount` is initialized to the amount 1000.00 (line 16).
5. The structure member `y1990[1].buyer.firm` is initialized to the string "Wilson & Co." (line 18).
6. The structure member `y1990[1].buyer.contact` is initialized to the string "Ed Wilson" (line 18).
7. The structure member `y1990[1].item` is initialized to the string "Type 12 gizmo" (line 19).
8. The structure member `y1990[1].amount` is initialized to the amount 290.00 (line 20).

## 1.27 Structures and Pointers

Given that pointers are such an important part of C, you shouldn't be surprised to find that they can be used with structures. You can use pointers as structure members, and you can also declare pointers to structures. These topics are covered in the following sections.

### Pointers as Structure Members

You have complete flexibility in using pointers as structure members. Pointer members are declared in the same manner as pointers that aren't members of structures—that is, by using the indirection operator (\*). Here's an example:

```
struct data {  
    int *value;  
    int *rate;  
} first;
```

These statements define and declare a structure whose two members are both pointers to type int. As with all pointers, declaring them is not enough; you must, by assigning them the address of a variable, initialize them to point to something. If cost and interest have been declared to be type int variables, you could write

```
first.value = &cost;
```

```
first.rate = &interest;
```

Now that the pointers have been initialized, you can use the indirection operator (\*). The expression \*first.value evaluates to the value of cost, and the expression \*first.rate evaluates to the value of interest.

Perhaps the type of pointer most frequently used as a structure member is a pointer to type char. A string is a sequence of characters delineated by a pointer that points to the string's first character and a null character that indicates the end of the string. To refresh your memory, you can declare a pointer to type char and initialize it to point at a string as follows:

```
char *p_message;
```

```
p_message = "Teach Yourself C ";
```

You can do the same thing with pointers to type char that are structure members:

```
struct msg {
```

```
    char *p1;
```

```
    char *p2;
```

```
} myptrs;
```

```
myptrs.p1 = "Teach Yourself C ";
```

```
myptrs.p2 = "By XYZ ";
```

Figure 11.4 illustrates the result of executing these statements. Each pointer member of the structure points to the first byte of a string, stored elsewhere in memory. Contrast this with Figure 11.3, which shows how data is stored in a structure that contains arrays of type char.

You can use pointer structure members anywhere a pointer can be used. For example, to print the pointed-to strings, you would write

```
printf("%s %s", myptrs.p1, myptrs.p2);
```

#### *A structure that contains pointers to type char.*

What's the difference between using an array of type char as a structure member and using a pointer to type char? These are both methods for "storing" a string in a structure, as shown here in the structure msg, which uses both methods:

```
struct msg {
```

```
    char p1[30];
```

```
    char *p2;
```

```
} myptrs;
```

Recall that an array name without brackets is a pointer to the first array element. Therefore, you can use these two structure members in similar fashion:

```
strcpy(myptrs.p1, "Teach Yourself C ");
```

```
strcpy(myptrs.p2, "By XYZ ");
```

```
/* additional code goes here */
```

```
puts(myptrs.p1);
```

```
puts(myptrs.p2);
```

What's the difference between these methods? It is this: If you define a structure that contains an array of type char, every instance of that structure type contains storage space for an array of the specified size. Furthermore, you're limited to the specified size; you can't store a larger string in the structure. Here's an example:

```
struct msg {
```

```
    char p1[10];
```

```
    char p2[10];
```

```
} myptrs;
```

```
...
```

```
strcpy(p1, "Minneapolis"); /* Wrong! String longer than array.*/
```

```
strcpy(p2, "MN");          /* OK, but wastes space because */
```

```
/* string shorter than array. */
```

If, on the other hand, you define a structure that contains pointers to type char, these restrictions don't apply. Each instance of the structure contains storage space for only the pointer. The actual strings are stored elsewhere in memory (but you don't need to worry about *where* in memory). There's no length restriction or wasted space. The actual strings aren't stored as part of the structure. Each pointer in the structure can point to a string of any length. That string becomes part of the structure, even though it isn't stored in the structure.

## Pointers to Structures

A C program can declare and use pointers to structures, just as it can declare pointers to any other data storage type. As you'll see later in this chapter, pointers to structures are often used when passing a structure as an argument to a function. For now, take a look at how your program can create and use pointers to structures. First, define a structure:

```
struct part {  
    int number;  
    char name[10];  
};
```

Now declare a pointer to type part:  
`struct part *p_part;`

Remember, the indirection operator (\*) in the declaration says that p\_part is a pointer to type part, not an instance of type part.

Can the pointer be initialized now? No, because even though the structure part has been defined, no instances of it have been declared. Remember that it's a declaration, not a definition, that sets aside storage space in memory for a data object. Because a pointer needs a memory address to point to, you must declare an instance of type part before anything can point to it. Here's the declaration:  
`struct part gizmo;`

Now you can perform the pointer initialization:  
`p_part = &gizmo;`  
This statement assigns the address of gizmo to p\_part.

### ***A pointer to a structure points to the structure's first byte.***

Now that you have a pointer to the structure gizmo, how do you make use of it? One method uses the indirection operator (\*). Recall that if ptr is a pointer to a data object, the expression \*ptr refers to the object pointed to.

Applying this to the current example, you know that p\_part is a pointer to the structure gizmo, so \*p\_part refers to gizmo. You then apply the structure member operator (.) to access individual members of gizmo. To assign the value 100 to gizmo.number, you could write  
`(*p_part).number = 100;`  
\*p\_part must be enclosed in parentheses because the (.) operator has a higher precedence than the (\*) operator.

A second way to access structure members using a pointer to the structure is to use the *indirect membership operator*, which consists of the characters -> (a hyphen followed by the greater-than symbol). (Note that when they are used together in this way, C treats them as a single operator, not two.) This symbol is placed between the pointer name and the member name. For example, to access the number member of gizmo with the p\_part pointer, you would write  
`p_part->number`

Looking at another example, if str is a structure, p\_str is a pointer to str, and memb is a member of str, you can access str.memb by writing

p\_str->memb

Therefore, there are three ways to access a structure member:

Using the structure name

Using a pointer to the structure with the indirection operator (\*)

Using a pointer to the structure with the indirect membership operator (->)

If p\_str is a pointer to the structure str, the following expressions are all equivalent:

str.memb

(\*p\_str).memb

p\_str->memb

## Pointers and Arrays of Structures

You've seen that arrays of structures can be a very powerful programming tool, as can pointers to structures. You can combine the two, using pointers to access structures that are array elements.

To illustrate, here is a structure definition from an earlier example:

```
struct part {  
    int number;  
    char name[10];  
};
```

After the part structure is defined, you can declare an array of type part:

```
struct part data[100];
```

Next you can declare a pointer to type part and initialize it to point to the first structure in the array data:

```
struct part *p_part;
```

```
p_part = &data[0];
```

Recall that the name of an array without brackets is a pointer to the first array element, so the second line could also have been written as

```
p_part = data;
```

You now have an array of structures of type part and a pointer to the first array element (that is, the first structure in the array). For example, you could print the contents of the first element using the statement

```
printf("%d %s", p_part->number, p_part->name);
```

What if you wanted to print all the array elements? You would probably use a for loop, printing one array element with each iteration of the loop. To access the members using pointer notation, you must change the pointer p\_part so that with each iteration of the loop it points to the next array element (that is, the next structure in the array). How do you do this?

C's pointer arithmetic comes to your aid. The unary increment operator (++) has a special meaning when applied to a pointer: It means "increment the pointer by the size of the object it points to." Put another way, if you have a pointer ptr that points to a data object of type obj, the statement

```
ptr++;
```

has the same effect as

```
ptr += sizeof(obj);
```

This aspect of pointer arithmetic is particularly relevant to arrays because array elements are stored sequentially in memory. If a pointer points to array element  $n$ , incrementing the pointer with the (++) operator causes it to point to element  $n + 1$ . This is illustrated in Figure 11.6, which shows an array named x[] that consists of four-byte elements (for example, a structure containing two type int members, each two bytes long). The pointer ptr was initialized to point to x[0]; each time ptr is incremented, it points to the next array element.

**Figure 11.6.** *With each increment, a pointer steps to the next array element.*

What this means is that your program can step through an array of structures (or an array of any other data type, for that matter) by incrementing a pointer. This sort of notation is usually easier to use and more concise than using array subscripts to perform the same task. Listing 11.4 shows how you do this.

Listing 11.4. **Accessing successive array elements by incrementing a pointer.**

```
1: /* Demonstrates stepping through an array of structures */
```

```
2: /* using pointer notation. */
3:
4: #include <stdio.h>
5:
6: #define MAX 4
7:
8: /* Define a structure, then declare and initialize */
9: /* an array of four structures. */
10:
11: struct part {
12:     int number;
13:     char name[10];
14: } data[MAX] = {1, "Smith",
15:               2, "Jones",
16:               3, "Adams",
17:               4, "Wilson"
18:             };
19:
20: /* Declare a pointer to type part, and a counter variable. */
21:
22: struct part *p_part;
23: int count;
24:
25: main()
26: {
27:     /* Initialize the pointer to the first array element. */
28:
29:     p_part = data;
30:
31:     /* Loop through the array, incrementing the pointer */
32:     /* with each iteration. */
33:
34:     for (count = 0; count < MAX; count++)
35:     {
36:         printf("At address %d: %d %s\n", p_part, p_part->number,
37:              p_part->name);
38:         p_part++;
39:     }
40:
41:     return 0;
42: }
At address 96: 1 Smith
At address 108: 2 Jones
At address 120: 3 Adams
At address 132: 4 Wilson
```

**ANALYSIS:** First, in lines 11 through 18, this program declares and initializes an array of structures called data. A pointer called p\_part is then defined in line 22 to be used to point to the data structure. The main() function's first task in line 29 is to set the pointer, p\_part, to point to the part structure that was declared. All the elements are then printed using a for loop in lines 34 through 39 that increments the pointer to the array with each iteration. The program also displays the address of each element.

Look closely at the addresses displayed. The precise values might differ on your system, but they are in equal-sized increments—just the size of the structure part (most systems will have an increment of 12). This clearly illustrates that incrementing a pointer increases it by an amount equal to the size of the data object it points to.

## *Passing Structures as Arguments to Functions*

Like other data types, a structure can be passed as an argument to a function. Listing 11.5 shows how to do this. This program is a modification of the program shown in Listing 11.2. It uses a function to display data on the screen, whereas Listing 11.2 uses statements that are part of main().

Listing 11.5. Passing a structure as a function argument.

```
1: /* Demonstrates passing a structure to a function. */
2:
3: #include <stdio.h>
4:
5: /* Declare and define a structure to hold the data. */
6:
7: struct data{
8:     float amount;
9:     char fname[30];
10:    char lname[30];
11: } rec;
12:
13: /* The function prototype. The function has no return value, */
14: /* and it takes a structure of type data as its one argument. */
15:
16: void print_rec(struct data x);
17:
18: main()
19: {
20:     /* Input the data from the keyboard. */
21:
22:     printf("Enter the donor's first and last names,\n");
23:     printf("separated by a space: ");
24:     scanf("%s %s", rec.fname, rec.lname);
25:
26:     printf("\nEnter the donation amount: ");
27:     scanf("%f", &rec.amount);
28:
29:     /* Call the display function. */
30:     print_rec( rec );
31:
32:     return 0;
33: }
34: void print_rec(struct data x)
35: {
36:     printf("\nDonor %s %s gave $%.2f.\n", x.fname, x.lname,
37:           x.amount);
38: }
```

```
Enter the donor's first and last names,
separated by a space: Bradley Jones
Enter the donation amount: 1000.00
Donor Bradley Jones gave $1000.00.
```

**ANALYSIS:** Looking at line 16, you see the function prototype for the function that is to receive the structure. As you would with any other data type that was going to be passed, you need to include the proper arguments. In this case, it is a structure of type data. This is repeated in the header for the function in line 34. When calling the function, you only need to pass the structure instance name--in this case, rec (line 30). That's all there is to it. Passing a structure to a function isn't very different from passing a simple variable.

You can also pass a structure to a function by passing the structure's address (that is, a pointer to the structure). In fact, in older versions of C, this was the only way to pass a structure as an argument. It's not necessary now, but you might see older programs that still use this method. If you pass a pointer to a structure as an argument, remember that you must use the indirect membership operator (->) to access structure members in the function.

## 1.28 Unions

*Unions* are similar to structures. A union is declared and used in the same ways that a structure is. A union differs from a structure in that only one of its members can be used at a time. The reason for this is simple. All the members of a union occupy the same area of memory. They are laid on top of each other.

### *Defining, Declaring, and Initializing Unions*

Unions are defined and declared in the same fashion as structures. The only difference in the declarations is that the keyword `union` is used instead of `struct`. To define a simple union of a `char` variable and an integer variable, you would write the following:

```
union shared {  
    char c;  
    int i;  
};
```

This union, `shared`, can be used to create instances of a union that can hold either a character value `c` or an integer value `i`. This is an OR condition. Unlike a structure that would hold both values, the union can hold only one value at a time. Figure 11.7 illustrates how the shared union would appear in memory.

A union can be initialized on its declaration. Because only one member can be used at a time, only one can be initialized. To avoid confusion, only the first member of the union can be initialized. The following code shows an instance of the shared union being declared and initialized:

```
union shared generic_variable = { '@'};
```

Notice that the `generic_variable` union was initialized just as the first member of a structure would be initialized.

**Figure 11.7.** *The union can hold only one value at a time.*

### Accessing Union Members

Individual union members can be used in the same way that structure members can be used—by using the member operator (`.`). However, there is an important difference in accessing union members. Only one union member should be accessed at a time. Because a union stores its members on top of each other, it's important to access only one member at a time. Listing 11.6 presents an example.

Listing 11.6. **An example of the wrong use of unions.**

```
1: /* Example of using more than one union member at a time */  
2: #include <stdio.h>  
3:  
4: main()  
5: {  
6:     union shared_tag {  
7:         char c;  
8:         int i;  
9:         long l;  
10:        float f;  
11:        double d;  
12:    } shared;  
13:  
14:    shared.c = '$';  
15:  
16:    printf("\nchar c = %c", shared.c);  
17:    printf("\nint i = %d", shared.i);  
18:    printf("\nlong l = %ld", shared.l);  
19:    printf("\nfloat f = %f", shared.f);  
20:    printf("\ndouble d = %f", shared.d);
```

```
21:
22:     shared.d = 123456789.8765;
23:
24:     printf("\n\nchar c = %c", shared.c);
25:     printf("\nint i = %d", shared.i);
26:     printf("\nlong l = %ld", shared.l);
27:     printf("\nfloat f = %f", shared.f);
28:     printf("\ndouble d = %f\n", shared.d);
29:
30:     return 0;
31: }
char c = $
int i = 4900
long l = 437785380
float f = 0.000000
double d = 0.000000
char c = 7
int i = -30409
long l = 1468107063
float f = 284852666499072.000000
double d = 123456789.876500
```

**ANALYSIS:** In this listing, you can see that a union named `shared` is defined and declared in lines 6 through 12. `shared` contains five members, each of a different type. Lines 14 and 22 initialize individual members of `shared`. Lines 16 through 20 and 24 through 28 then present the values of each member using `printf()` statements.

Note that, with the exceptions of `char c = $` and `double d = 123456789.876500`, the output might not be the same on your computer. Because the character variable, `c`, was initialized in line 14, it is the only value that should be used until a different member is initialized. The results of printing the other union member variables (`i`, `l`, `f`, and `d`) can be unpredictable (lines 16 through 20). Line 22 puts a value into the double variable, `d`. Notice that the printing of the variables again is unpredictable for all but `d`. The value entered into `c` in line 14 has been lost because it was overwritten when the value of `d` in line 22 was entered. This is evidence that the members all occupy the same space.

## The union Keyword

```
union tag {
    union_member(s);
    /* additional statements may go here */
};
```

The `union` keyword is used for declaring unions. A union is a collection of one or more variables (*union\_members*) that have been grouped under a single name. In addition, each of these union members occupies the same area of memory.

The keyword `union` identifies the beginning of a union definition. It's followed by a tag that is the name given to the union. Following the tag are the union members enclosed in braces. An *instance*, the actual declaration of a union, also can be defined. If you define the structure without the instance, it's just a template that can be used later in a program to declare structures. The following is a template's format:

```
union tag {
    union_member(s);
    /* additional statements may go here */
};
```

To use the template, you would use the following format:

```
union tag instance;
```

To use this format, you must have previously declared a union with the given tag.

### Example 1

```
/* Declare a union template called tag */
union tag {
    int nbr;
```



```
    char character;  
}  
/* Use the union template */  
union tag mixed_variable;
```

### Example 2

```
/* Declare a union and instance together */  
union generic_type_tag {  
    char c;  
    int i;  
    float f;  
    double d;  
} generic;
```

### Example 3

```
/* Initialize a union. */  
union date_tag {  
    char full_date[9];  
    struct part_date_tag {  
        char month[2];  
        char break_value1;  
        char day[2];  
        char break_value2;  
        char year[2];  
    } part_date;  
} date = {"01/01/97"};
```

Listing 11.7 shows a more practical use of a union. Although this use is simplistic, it's one of the more common uses of a union.

Listing 11.7. A practical use of a union.

```
1: /* Example of a typical use of a union */  
2:  
3: #include <stdio.h>  
4:  
5: #define CHARACTER 'C'  
6: #define INTEGER 'I'  
7: #define FLOAT 'F'  
8:  
9: struct generic_tag{  
10:     char type;  
11:     union shared_tag {  
12:         char c;  
13:         int i;  
14:         float f;  
15:     } shared;  
16: };  
17:  
18: void print_function( struct generic_tag generic );  
19:  
20: main()  
21: {  
22:     struct generic_tag var;  
23:  
24:     var.type = CHARACTER;  
25:     var.shared.c = '$';  
26:     print_function( var );  
27:  
28:     var.type = FLOAT;  
29:     var.shared.f = (float) 12345.67890;  
30:     print_function( var );  
31:  
32:     var.type = 'x';  
33:     var.shared.i = 111;
```

```
34:  print_function( var );
35:  return 0;
36: }
37: void print_function( struct generic_tag generic )
38: {
39:  printf("\n\nThe generic value is...");
40:  switch( generic.type )
41:  {
42:    case CHARACTER: printf("%c", generic.shared.c);
43:                    break;
44:    case INTEGER:   printf("%d", generic.shared.i);
45:                    break;
46:    case FLOAT:     printf("%f", generic.shared.f);
47:                    break;
48:    default:        printf("an unknown type: %c\n",
49:                           generic.type);
50:                    break;
51:  }
52: }
```

The generic value is...\$

The generic value is...12345.678711

The generic value is...an unknown type: x

**ANALYSIS:** This program is a very simplistic version of what could be done with a union. This program provides a way of storing multiple data types in a single storage space. The `generic_tag` structure lets you store either a character, an integer, or a floating-point number within the same area. This area is a union called `shared` that operates just like the examples in Listing 11.6. Notice that the `generic_tag` structure also adds an additional field called `type`. This field is used to store information on the type of variable contained in `shared`. `type` helps prevent `shared` from being used in the wrong way, thus helping to avoid erroneous data such as that presented in Listing 11.6.

A formal look at the program shows that lines 5, 6, and 7 define constants `CHARACTER`, `INTEGER`, and `FLOAT`. These are used later in the program to make the listing more readable. Lines 9 through 16 define a `generic_tag` structure that will be used later. Line 18 presents a prototype for the `print_function()`. The structure `var` is declared in line 22 and is first initialized to hold a character value in lines 24 and 25. A call to `print_function()` in line 26 lets the value be printed. Lines 28 through 30 and 32 through 34 repeat this process with other values.

The `print_function()` is the heart of this listing. Although this function is used to print the value from a `generic_tag` variable, a similar function could have been used to initialize it. `print_function()` will evaluate the `type` variable in order to print a statement with the appropriate variable type. This prevents getting erroneous data such as that in Listing 11.6.

---

## *typedef and Structures*

You can use the `typedef` keyword to create a synonym for a structure or union type. For example, the following statements define `coord` as a synonym for the indicated structure:

```
typedef struct {
    int x;
    int y;
} coord;
```

You can then declare instances of this structure using the `coord` identifier:

```
coord topleft, bottomright;
```

Note that a `typedef` is different from a structure tag, as described earlier in this chapter. If you write

```
struct coord {
    int x;
    int y;
```

```
};
```

the identifier `coord` is a tag for the structure. You can use the tag to declare instances of the structure, but unlike with a typedef, you must include the `struct` keyword:

```
struct coord topleft, bottomright;
```

Whether you use typedef or a structure tag to declare structures makes little difference. Using typedef results in slightly more concise code, because the `struct` keyword doesn't need to be used. On the other hand, using a tag and having the `struct` keyword explicit

## Some more on Strings, and Arrays of Strings

Well, let's go back to strings for a bit. In the following all assignments are to be understood as being global, i.e. made outside of any function, including `main()`.

We pointed out in an earlier chapter that we could write:

```
char my_string[40] = "Ted";
```

Which would allocate space for a 40 byte array and put the string in the first 4 bytes (three for the characters in the quotes and a 4th to handle the terminating `'\\0'`).

Actually, if all we wanted to do was store the name "Ted" we could write:

```
char my_name[] = "Ted";
```

And the compiler would count the characters, leave room for the nul character and store the total of the four characters in memory the location of which would be returned by the array name, in this case **my\_name**.

In some code, instead of the above, you might see:

```
char *my_name = "Ted";
```

Which is an alternate approach. Is there a difference between these? The answer is.. yes. Using the array notation 4 bytes of storage in the static memory block are taken up, one for each character and one for the terminating nul character. But, in the pointer notation the same 4 bytes required, **plus** N bytes to store the pointer variable **my\_name** (where N depends on the system but is usually a minimum of 2 bytes and can be 4 or more).

In the array notation, **my\_name** is short for **&myname[0]** which is the address of the first element of the array. Since the location of the array is fixed during run time, this is a constant (not a variable). In the pointer notation **my\_name** is a variable. As to which is the **better** method, that depends on what you are going to do within the rest of the program.

Let's now go one step further and consider what happens if each of these declarations are done within a function as opposed to globally outside the bounds of any function.

```
void my_function_A(char *ptr)
```

```
{
    char a[] = "ABCDE"
```

```
    .
    .
```

```
}
```

```
void my_function_B(char *ptr)
```

```
{
    char *cp = "FGHIJ"
```

```
    .
    .
```

```
}
```

In the case of **my\_function\_A**, the content, or value(s), of the array **a[]** is considered to be the data. The array is said to be initialized to the values ABCDE. In the case of **my\_function\_B**, the value of the pointer **cp** is considered to be the data. The pointer has been initialized to point to the string **FGHIJ**. In both **my\_function\_A** and **my\_function\_B** the definitions are local variables and thus the string **ABCDE** is stored on the stack, as is the value of the pointer **cp**. The string **FGHIJ** can be stored anywhere. On my system it gets stored in the data segment.

By the way, array initialization of automatic variables as is done in **my\_function\_A** was illegal in the older K&R C and only "came of age" in the newer ANSI C. A fact that may be important when one is considering portability and backwards compatibility.

As long as we are discussing the relationship/differences between pointers and arrays, let's move on to multi-dimensional arrays. Consider, for example the array:

```
char multi[5][10];
```

Just what does this mean? Well, let's consider it in the following light.

```
char multi[5][10];
```

Let's take the underlined part to be the "name" of an array. Then prepending the **char** and appending the **[10]** we have an array of 10 characters. But, the name **multi[5]** is itself an array indicating that there are 5 elements each being an array of 10 characters. Hence we have an array of 5 arrays of 10 characters each..

Assume we have filled this two dimensional array with data of some kind. In memory, it might look as if it had been formed by initializing 5 separate arrays using something like:

```
multi[0] = {'0','1','2','3','4','5','6','7','8','9'}
multi[1] = {'a','b','c','d','e','f','g','h','i','j'}
multi[2] = {'A','B','C','D','E','F','G','H','I','J'}
multi[3] = {'9','8','7','6','5','4','3','2','1','0'}
multi[4] = {'J','I','H','G','F','E','D','C','B','A'}
```

At the same time, individual elements might be addressable using syntax such as:

```
multi[0][3] = '3'
multi[1][7] = 'h'
multi[4][0] = 'J'
```

Since arrays are contiguous in memory, our actual memory block for the above should look like:

```
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJ9876543210JIHGFEDCBA
```

^

|\_\_\_\_\_ starting at the address &multi[0][0]

Note that we did **not** write **multi[0] = "0123456789"**. Had we done so a terminating **'\0'** would have been implied since whenever double quotes are used a **'\0'** character is appended to the characters contained within those quotes. Had that been the case we would have had to set aside room for 11 characters per row instead of 10.

Our goal in the above is to illustrate how memory is laid out for 2 dimensional arrays. That is, this is a 2 dimensional array of characters, NOT an array of "strings".

Now, the compiler knows how many columns are present in the array so it can interpret **multi + 1** as the address of the 'a' in the 2nd row above. That is, it adds 10, the number of columns, to get this location. If we were dealing with integers and an array with the same dimension the compiler would add **10\*sizeof(int)** which, on my machine, would be 20. Thus, the address of the 9 in the 4th row above would be **&multi[3][0]** or **\*(multi + 3)** in pointer notation. To get to the content of the 2nd element in the 4th row we add 1 to this address and dereference the result as in

```
*(*(multi + 3) + 1)
```

With a little thought we can see that:

```
*(*(multi + row) + col) and
multi[row][col] yield the same results.
```

The following program illustrates this using integer arrays instead of character arrays.

----- program 6.1 -----

```
#include <stdio.h>
#define ROWS 5
#define COLS 10
```

```
int multi[ROWS][COLS];
```

```
int main(void)
{
    int row, col;
    for (row = 0; row < ROWS; row++)
    {
        for (col = 0; col < COLS; col++)
        {
            multi[row][col] = row*col;
        }
    }

    for (row = 0; row < ROWS; row++)
    {
        for (col = 0; col < COLS; col++)
        {
            printf("\n%d ",multi[row][col]);
            printf("%d ",*(multi + row) + col));
        }
    }

    return 0;
}
```

----- end of program 6.1 -----

Because of the double de-referencing required in the pointer version, the name of a 2 dimensional array is often said to be equivalent to a pointer to a pointer. With a three dimensional array we would be dealing with an array of arrays of arrays and some might say its name would be equivalent to a pointer to a pointer to a pointer. However, here we have initially set aside the block of memory for the array by defining it using array notation. Hence, we are dealing with a constant, not a variable. That is we are talking about a fixed address not a variable pointer. The dereferencing function used above permits us to access any element in the array of arrays without the need of changing the value of that address (the address of **multi[0][0]** as given by the symbol **multi**).

## More on Multi-Dimensional Arrays

In the previous chapter we noted that given

```
#define ROWS 5  
#define COLS 10
```

```
int multi[ROWS][COLS];
```

we can access individual elements of the array **multi** using either:

```
multi[row][col]
```

or

```
*(*(multi + row) + col)
```

To understand more fully what is going on, let us replace

```
*(multi + row)
```

with **X** as in:

```
*(X + col)
```

Now, from this we see that **X** is like a pointer since the expression is de-referenced and we know that **col** is an integer. Here the arithmetic being used is of a special kind called "pointer arithmetic" is being used. That means that, since we are talking about an integer array, the address pointed to by (i.e. value of) **X + col + 1** must be greater than the address **X + col** by and amount equal to **sizeof(int)**.

Since we know the memory layout for 2 dimensional arrays, we can determine that in the expression **multi + row** as used above, **multi + row + 1** must increase by value an amount equal to that needed to "point to" the next row, which in this case would be an amount equal to **COLS \* sizeof(int)**.

That says that if the expression **\*(\*(multi + row) + col)** is to be evaluated correctly at run time, the compiler must generate code which takes into consideration the value of **COLS**, i.e. the 2nd dimension. Because of the equivalence of the two forms of expression, this is true whether we are using the pointer expression as here or the array expression **multi[row][col]**. Thus, to evaluate either expression, a total of 5 values must be known:

The address of the first element of the array, which is returned by the expression **multi**, i.e., the name of the array.

The size of the type of the elements of the array, in this case **sizeof(int)**.

The 2nd dimension of the array

The specific index value for the first dimension, **row** in this case.

The specific index value for the second dimension, **col** in this case.

Given all of that, consider the problem of designing a function to manipulate the element values of a previously declared array. For example, one which would set all the elements of the array **multi** to the value 1.

```
void set_value(int m_array[][COLS])  
{  
    int row, col;  
    for (row = 0; row < ROWS; row++)  
    {  
        for (col = 0; col < COLS; col++)  
        {  
            m_array[row][col] = 1;  
        }  
    }  
}
```

And to call this function we would then use:

```
set_value(multi);
```

Now, within the function we have used the values #defined by **ROWS** and **COLS** that set the limits on the for loops. But, these #defines are just constants as far as the compiler is concerned, i.e. there is nothing to connect them to the array size within the function. **row** and

**col** are local variables, of course. The formal parameter definition permits the compiler to determine the characteristics associated with the pointer value that will be passed at run time. We really don't need the first dimension and, as will be seen later, there are occasions where we would prefer not to define it within the parameter definition, out of habit or consistency, we have not used it here. But, the second dimension must be used as has been shown in the expression for the parameter. The reason is that we need this in the evaluation of **m\_array[row][col]** as has been described. While the parameter defines the data type (**int** in this case) and the automatic variables for row and column are defined in the for loops, only one value can be passed using a single parameter. In this case, that is the value of **multi** as noted in the call statement, i.e. the address of the first element, often referred to as a pointer to the array. Thus, the only way we have of informing the compiler of the 2nd dimension is by explicitly including it in the parameter definition.

In fact, in general all dimensions of higher order than one are needed when dealing with multi-dimensional arrays. That is if we are talking about 3 dimensional arrays, the 2nd **and** 3rd dimension must be specified in the parameter definition.



## 1.29 Pointers to Arrays

Pointers, of course, can be "pointed at" any type of data object, including arrays. To review, we stated that given an array of integers we could point an integer pointer at that array using:

```
int *ptr;  
ptr = &my_array[0];    /* point our pointer at the first  
                        integer in our array */
```

As we stated there, the type of the pointer variable must match the type of the first element of the array.

In addition, we can use a pointer as a formal parameter of a function which is designed to manipulate an array. e.g.

Given:

```
int array[3] = {'1', '5', '7'};  
void a_func(int *p);
```

Some programmers might prefer to write the function prototype as:

```
void a_func(int p[]);
```

Which would tend to inform others who might use this function that the function is designed to manipulate the elements of an array. Of course, in either case, what actually gets passed is the value of a pointer to the first element of the array, independent of which notation is used in the function prototype or definition. Note that if the array notation is used, there is no need to pass the actual dimension of the array since we are not passing the whole array, only the address to the first element.

We now turn to the problem of the 2 dimensional array. As stated in the last chapter, C interprets a 2 dimensional array as an array of one dimensional arrays. That being the case, the first element of a 2 dimensional array of integers is a one dimensional array of integers. And a pointer to a two dimensional array of integers must be a pointer to that data type. One way of accomplishing this is through the use of the keyword "typedef". typedef assigns a new name to a specified data type. For example:

```
typedef unsigned char byte;
```

causes the name **byte** to mean type **unsigned char**. Hence

```
byte b[10];    would be an array of unsigned characters.
```

Note that in the typedef declaration, the word **byte** has replaced that which would normally be the name of our **unsigned char**. That is, the rule for using **typedef** is that the new name for the data type is the name used in the definition of the data type. Thus in:

```
typedef int Array[10];
```

Array becomes a data type for an array of 10 integers. i.e. **Array my\_arr**; declares **my\_arr** as an array of 10 integers and **Array arr2d[5]**; makes **arr2d** an array of 5 arrays of 10 integers each.

Also note that **Array \*p1d**; makes **p1d** a pointer to an array of 10 integers. Because **\*p1d** points to the same type as **arr2d**, assigning the address of the two dimensional array **arr2d** to **p1d**, the pointer to a one dimensional array of 10 integers is acceptable. i.e. **p1d = &arr2d[0]**; or **p1d = arr2d**; are both correct.

Since the data type we use for our pointer is an array of 10 integers we would expect that incrementing **p1d** by 1 would change its value by **10\*sizeof(int)**, which it does. That is, **sizeof(\*p1d)** is 20. You can prove this to yourself by writing and running a simple short program.

Now, while using typedef makes things clearer for the reader and easier on the programmer, it is not really necessary. What we need is a way of declaring a pointer like **p1d** without the need of the **typedef** keyword. It turns out that this can be done and that

```
int (*p1d)[10];
```



is the proper declaration, i.e. **p1d** here is a pointer to an array of 10 integers just as it was under the declaration using the Array type. Note that this is different from  
`int *p1d[10];`

Which would make **p1d** the name of an array of 10 pointers to type **int**.

## *Pointers and Dynamic Allocation of Memory*

There are times when it is convenient to allocate memory at run time using **malloc()**, **calloc()**, or other allocation functions. Using this approach permits postponing the decision on the size of the memory block need to store an array, for example, until run time. Or it permits using a section of memory for the storage of an array of integers at one point in time, and then when that memory is no longer needed it can be freed up for other uses, such as the storage of an array of structures.

When memory is allocated, the allocating function (such as **malloc()**, **calloc()**, etc.) returns a pointer. The type of this pointer depends on whether you are using an older K&R compiler or the newer ANSI type compiler. With the older compiler the type of the returned pointer is **char**, with the ANSI compiler it is **void**.

If you are using an older compiler, and you want to allocate memory for an array of integers you will have to cast the char pointer returned to an integer pointer. For example, to allocate space for 10 integers we might write:

```
int *iptr;  
iptr = (int *)malloc(10 * sizeof(int));  
if (iptr == NULL)
```

```
{ .. ERROR ROUTINE GOES HERE .. }
```

If you are using an ANSI compliant compiler, **malloc()** returns a **void** pointer and since a void pointer can be assigned to a pointer variable of any object type, the **(int \*)** cast shown above is not needed. The array dimension can be determined at run time and is not needed at compile time. That is, the **10** above could be a variable read in from a data file or keyboard, or calculated based on some need, at run time.

Because of the equivalence between array and pointer notation, once **iptr** has been assigned as above, one can use the array notation. For example, one could write:

```
int k;  
for (k = 0; k < 10; k++)  
    iptr[k] = 2;
```

to set the values of all elements to 2.

Even with a reasonably good understanding of pointers and arrays, one place the newcomer to C is likely to stumble at first is in the dynamic allocation of multi-dimensional arrays. In general, we would like to be able to access elements of such arrays using array notation, not pointer notation, wherever possible. Depending on the application we may or may not know both dimensions at compile time. This leads to a variety of ways to go about our task.

As we have seen, when dynamically allocating a one dimensional array its dimension can be determined at run time. Now, when using dynamic allocation of higher order arrays, we never need to know the first dimension at compile time. Whether we need to know the higher dimensions depends on how we go about writing the code. Here we will discuss various methods of dynamically allocating room for 2 dimensional arrays of integers.

First we will consider cases where the 2nd dimension is known at compile time.

METHOD 1:

One way of dealing with the problem is through the use of the **typedef** keyword. To allocate a 2 dimensional array of integers recall that the following two notations result in the same object code being generated:

```
multi[row][col] = 1;    (*(multi + row) + col) = 1;
```

It is also true that the following two notations generate the same code:

```
multi[row]              *(multi + row)
```

Since the one on the right must evaluate to a pointer, the array notation on the left must also evaluate to a pointer. In fact **multi[0]** will return a pointer to the first integer in the first row, **multi[1]** a pointer to the first integer of the second row, etc. Actually, **multi[n]** evaluates to a pointer to that array of integers that make up the n-th row of our 2 dimensional array. That is, **multi** can be thought of as an array of arrays and **multi[n]** as a pointer to the n-th array of this array of arrays. Here the word **pointer** is being used to represent an address value. While such usage is common in the literature, when reading such statements one must be careful to distinguish between the constant address of an array and a variable pointer which is a data object in itself.

Consider now:

----- Program 9.1 -----

```
#include <stdio.h>
#include <stdlib.h>

#define COLS 5

typedef int RowArray[COLS];
RowArray *rptr;

int main(void)
{
    int nrows = 10;
    int row, col;
    rptr = malloc(nrows * COLS * sizeof(int));
    for (row = 0; row < nrows; row++)
    {
        for (col = 0; col < COLS; col++)
        {
            rptr[row][col] = 17;
        }
    }

    return 0;
}
```

Here it is assumed an ANSI compiler so a cast on the void pointer returned by **malloc()** is not required. If you are using an older K&R compiler you will have to cast using:

```
rptr = (RowArray *)malloc(... etc.
```

Using this approach, **rptr** has all the characteristics of an array name name, (except that **rptr** is modifiable), and array notation may be used throughout the rest of the program. That also means that if you intend to write a function to modify the array contents, you must use **COLS** as a part of the formal parameter in that function, just as we did when discussing the passing of two dimensional arrays to a function.

#### METHOD 2:

In the METHOD 1 above, **rptr** turned out to be a pointer to type "one dimensional array of **COLS** integers". It turns out that there is syntax which can be used for this type without the need of **typedef**. If we write:

```
int (*xptr)[COLS];
```

The variable **xptr** will have all the same characteristics as the variable **rptr** in METHOD 1 above, and we need not use the **typedef** keyword. Here **xptr** is a pointer to an array of integers and the size of that array is given by the **#defined COLS**. The parenthesis placement makes the pointer notation predominate, even though the array notation has higher precedence. i.e. had we written

```
int *xptr[COLS];
```

We would have defined **xptr** as an array of pointers holding the number of pointers equal to that #defined by COLS. That is not the same thing at all. However, arrays of pointers have their use in the dynamic allocation of two dimensional arrays, as will be seen in the next 2 methods.

#### METHOD 3:

Consider the case where we do not know the number of elements in each row at compile time, i.e. both the number of rows and number of columns must be determined at run time. One way of doing this would be to create an array of pointers to type **int** and then allocate space for each row and point these pointers at each row. Consider:

----- Program 9.2 -----

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int nrows = 5; /* Both nrows and ncols could be evaluated */
    int ncols = 10; /* or read in at run time */
    int row;
    int **rowptr;
    rowptr = malloc(nrows * sizeof(int *));
    if (rowptr == NULL)
    {
        puts("\nFailure to allocate room for row pointers.\n");
        exit(0);
    }

    printf("\n\n\nIndex  Pointer(hex)  Pointer(dec)  Diff.(dec)");

    for (row = 0; row < nrows; row++)
    {
        rowptr[row] = malloc(ncols * sizeof(int));
        if (rowptr[row] == NULL)
        {
            printf("\nFailure to allocate for row[%d]\n", row);
            exit(0);
        }
        printf("\n%d      %p      %d", row, rowptr[row], rowptr[row]);
        if (row > 0)
            printf("      %d", (int)(rowptr[row] - rowptr[row-1]));
    }

    return 0;
}
```

----- End 9.2 -----

In the above code **rowptr** is a pointer to pointer to type **int**. In this case it points to the first element of an array of pointers to type **int**. Consider the number of calls to **malloc()**:

|                              |   |       |
|------------------------------|---|-------|
| To get the array of pointers | 1 | call  |
| To get space for the rows    | 5 | calls |
| -----                        |   |       |
| Total                        | 6 | calls |

If you choose to use this approach note that while you can use the array notation to access individual elements of the array, e.g. **rowptr[row][col] = 17;**, it does not mean that the data in the "two dimensional array" is contiguous in memory.

You can, however, use the array notation just as if it were a continuous block of memory. For example, you can write:

```
rowptr[row][col] = 176;
```

just as if `rowptr` were the name of a two dimensional array created at compile time. Of course **row** and **col** must be within the bounds of the array you have created, just as with an array created at compile time.

If you want to have a contiguous block of memory dedicated to the storage of the elements in the array you can do it as follows:

#### METHOD 4:

In this method we allocate a block of memory to hold the whole array first. We then create an array of pointers to point to each row. Thus even though the array of pointers is being used, the actual array in memory is contiguous. The code looks like this:

----- Program 9.3 -----

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int **rptr;
    int *aptr;
    int *testptr;
    int k;
    int nrows = 5; /* Both nrows and ncols could be evaluated */
    int ncols = 8; /* or read in at run time */
    int row, col;

    /* we now allocate the memory for the array */

    aptr = malloc(nrows * ncols * sizeof(int));
    if (aptr == NULL)
    {
        puts("\nFailure to allocate room for the array");
        exit(0);
    }

    /* next we allocate room for the pointers to the rows */

    rptr = malloc(nrows * sizeof(int *));
    if (rptr == NULL)
    {
        puts("\nFailure to allocate room for pointers");
        exit(0);
    }

    /* and now we 'point' the pointers */

    for (k = 0; k < nrows; k++)
    {
        rptr[k] = aptr + (k * ncols);
    }

    /* Now we illustrate how the row pointers are incremented */
    printf("\n\nIllustrating how row pointers are incremented");
    printf("\n\nIndex   Pointer(hex) Diff.(dec)");

    for (row = 0; row < nrows; row++)
    {
        printf("\n%d      %p", row, rptr[row]);
        if (row > 0)
            printf("\n");
    }
}
```

```
        printf("        %d", (rptr[row] - rptr[row-1]));
    }
    printf("\n\nAnd now we print out the array\n");
    for (row = 0; row < nrows; row++)
    {
        for (col = 0; col < ncols; col++)
        {
            rptr[row][col] = row + col;
            printf("%d ", rptr[row][col]);
        }
        putchar('\n');
    }

    puts("\n");

    /* and here we illustrate that we are, in fact, dealing with
       a 2 dimensional array in a contiguous block of memory. */
    printf("And now we demonstrate that they are contiguous in memory\n");

    testptr = aptr;
    for (row = 0; row < nrows; row++)
    {
        for (col = 0; col < ncols; col++)
        {
            printf("%d ", *(testptr++));
        }
        putchar('\n');
    }

    return 0;
}
```

----- End Program 9.3 -----

Consider again, the number of calls to malloc()

|                                   |   |       |
|-----------------------------------|---|-------|
| To get room for the array itself  | 1 | call  |
| To get room for the array of ptrs | 1 | call  |
| ----                              |   |       |
| Total                             | 2 | calls |

Now, each call to **malloc()** creates additional space overhead since **malloc()** is generally implemented by the operating system forming a linked list which contains data concerning the size of the block. But, more importantly, with large arrays (several hundred rows) keeping track of what needs to be freed when the time comes can be more cumbersome. This, combined with the contiguousness of the data block that permits initialization to all zeroes using **memset()** would seem to make the second alternative the preferred one.

As a final example on multidimensional arrays we will illustrate the dynamic allocation of a three dimensional array. This example will illustrate one more thing to watch when doing this kind of allocation. For reasons cited above we will use the approach outlined in alternative two. Consider the following code:

----- Program 9.4 -----

```
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>
```

```
int X_DIM=16;
int Y_DIM=5;
int Z_DIM=3;

int main(void)
{
    char *space;
    char ***Arr3D;
    int y, z;
    ptrdiff_t diff;

    /* first we set aside space for the array itself */

    space = malloc(X_DIM * Y_DIM * Z_DIM * sizeof(char));

    /* next we allocate space of an array of pointers, each
       to eventually point to the first element of a
       2 dimensional array of pointers to pointers */

    Arr3D = malloc(Z_DIM * sizeof(char **));

    /* and for each of these we assign a pointer to a newly
       allocated array of pointers to a row */

    for (z = 0; z < Z_DIM; z++)
    {
        Arr3D[z] = malloc(Y_DIM * sizeof(char *));

        /* and for each space in this array we put a pointer to
           the first element of each row in the array space
           originally allocated */

        for (y = 0; y < Y_DIM; y++)
        {
            Arr3D[z][y] = space + (z*(X_DIM * Y_DIM) + y*X_DIM);
        }
    }

    /* And, now we check each address in our 3D array to see if
       the indexing of the Arr3d pointer leads through in a
       continuous manner */

    for (z = 0; z < Z_DIM; z++)
    {
        printf("Location of array %d is %p\n", z, *Arr3D[z]);
        for (y = 0; y < Y_DIM; y++)
        {
            printf(" Array %d and Row %d starts at %p", z, y, Arr3D[z][y]);
            diff = Arr3D[z][y] - space;
            printf(" diff = %d ", diff);
            printf(" z = %d y = %d\n", z, y);
        }
    }
    return 0;
}
```

----- End of Prog. 9.4 -----

If you have followed this tutorial up to this point you should have no problem deciphering the above on the basis of the comments alone. There are a couple of points that should be made however. Let's start with the line which reads:

```
Arr3D[z][y] = space + (z*(X_DIM * Y_DIM) + y*X_DIM);
```

Note that here **space** is a character pointer, which is the same type as **Arr3D[z][y]**. It is important that when adding an integer, such as that obtained by evaluation of the expression **(z\*(X\_DIM \* Y\_DIM) + y\*X\_DIM)**, to a pointer, the result is a new pointer value. And when assigning pointer values to pointer variables the data types of the value and variable must match.

## 1.30 Pointers to Functions

Up to this point we have been discussing pointers to data objects. C also permits the declaration of pointers to functions. Pointers to functions have a variety of uses and some of them will be discussed here.

Consider the following real problem. You want to write a function that is capable of sorting virtually any collection of data that can be stored in an array. This might be an array of strings, or integers, or floats, or even structures. The sorting algorithm can be the same for all. For example, it could be a simple bubble sort algorithm, or the more complex shell or quick sort algorithm. We'll use a simple bubble sort for demonstration purposes.

Sedgewick [1] has described the bubble sort using C code by setting up a function which when passed a pointer to the array would sort it. If we call that function **bubble()**, a sort program is described by bubble\_1.c, which follows:

```
/*----- bubble_1.c -----*/
```

```
#include <stdio.h>
```

```
int arr[10] = { 3,6,1,2,3,8,4,1,7,2};
```

```
void bubble(int a[], int N);
```

```
int main(void)
{
    int i;
    putchar('\n');
    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    bubble(arr, 10);
    putchar('\n');

    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

```
void bubble(int a[], int N)
```



```
{
    int i, j, t;
    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            if (a[j-1] > a[j])
            {
                t = a[j-1];
                a[j-1] = a[j];
                a[j] = t;
            }
        }
    }
}
```

```
/*----- end bubble_1.c -----*/
```

The bubble sort is one of the simpler sorts. The algorithm scans the array from the second to the last element comparing each element with the one which precedes it. If the one that precedes it is larger than the current element, the two are swapped so the larger one is closer to the end of the array. On the first pass, this results in the largest element ending up at the end of the array. The array is now limited to all elements except the last and the process repeated. This puts the next largest element at a point preceding the largest element. The process is repeated for a number of times equal to the number of elements minus 1. The end result is a sorted array.

Here our function is designed to sort an array of integers. Thus in line 1 we are comparing integers and in lines 2 through 4 we are using temporary integer storage to store integers.

What we want to do now is see if we can convert this code so we can use any data type, i.e. not be restricted to integers.

At the same time we don't want to have to analyze our algorithm and the code associated with it each time we use it. We start by removing the comparison from within the function **bubble()** so as to make it relatively easy to modify the comparison function without having to re-write portions related to the actual algorithm. This results in bubble\_2.c:

```
/*----- bubble_2.c -----*/
```

```
/* Separating the comparison function */
```

```
#include <stdio.h>
```

```
int arr[10] = { 3,6,1,2,3,8,4,1,7,2};
```

```
void bubble(int a[], int N);
int compare(int m, int n);
```

```
int main(void)
{
    int i;
    putchar('\n');
    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    bubble(arr,10);
    putchar('\n');
```

```
    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}

void bubble(int a[], int N)
{
    int i, j, t;
    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            if (compare(a[j-1], a[j]))
            {
                t = a[j-1];
                a[j-1] = a[j];
                a[j] = t;
            }
        }
    }
}

int compare(int m, int n)
{
    return (m > n);
}

/*----- end of bubble_2.c -----*/
```

If our goal is to make our sort routine data type independent, one way of doing this is to use pointers to type void to point to the data instead of using the integer data type. As a start in that direction let's modify a few things in the above so that pointers can be used. To begin with, we'll stick with pointers to type integer.

```
/*----- bubble_3.c -----*/
```

```
#include <stdio.h>

int arr[10] = { 3,6,1,2,3,8,4,1,7,2};

void bubble(int *p, int N);
int compare(int *m, int *n);

int main(void)
{
    int i;
    putchar('\n');

    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    bubble(arr, 10);
    putchar('\n');
```

```
    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}

void bubble(int *p, int N)
{
    int i, j, t;
    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            if (compare(&p[j-1], &p[j]))
            {
                t = p[j-1];
                p[j-1] = p[j];
                p[j] = t;
            }
        }
    }
}

int compare(int *m, int *n)
{
    return (*m > *n);
}

/*----- end of bubble3.c -----*/
```

Note the changes. We are now passing a pointer to an integer (or array of integers) to **bubble()**. And from within bubble we are passing pointers to the elements of the array that we want to compare to our comparison function. And, of course we are dereferencing these pointer in our **compare()** function in order to make the actual comparison. Our next step will be to convert the pointers in **bubble()** to pointers to type void so that that function will become more type insensitive. This is shown in bubble\_4.

```
/*----- bubble_4.c -----*/
```

```
#include <stdio.h>

int arr[10] = { 3,6,1,2,3,8,4,1,7,2};

void bubble(int *p, int N);
int compare(void *m, void *n);

int main(void)
{
    int i;
    putchar('\n');

    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    bubble(arr, 10);
    putchar('\n');

    for (i = 0; i < 10; i++)
```

```
{
    printf("%d ", arr[i]);
}
return 0;
}

void bubble(int *p, int N)
{
    int i, j, t;
    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            if (compare((void *)&p[j-1], (void *)&p[j]))
            {
                t = p[j-1];
                p[j-1] = p[j];
                p[j] = t;
            }
        }
    }
}

int compare(void *m, void *n)
{
    int *m1, *n1;
    m1 = (int *)m;
    n1 = (int *)n;
    return (*m1 > *n1);
}

/*----- end of bubble_4.c -----*/
```

Note that, in doing this, in **compare()** we had to introduce the casting of the void pointer types passed to the actual type being sorted. But, as we'll see later that's okay. And since what is being passed to **bubble()** is still a pointer to an array of integers, we had to cast these pointers to void pointers when we passed them as parameters in our call to **compare()**.

We now address the problem of what we pass to **bubble()**. We want to make the first parameter of that function a void pointer also. But, that means that within **bubble()** we need to do something about the variable **t**, which is currently an integer. Also, where we use **t = p[j-1];** the type of **p[j-1]** needs to be known in order to know how many bytes to copy to the variable **t** (or whatever we replace **t** with).

Currently, in bubble\_4.c, knowledge within **bubble()** as to the type of the data being sorted (and hence the size of each individual element) is obtained from the fact that the first parameter is a pointer to type integer. If we are going to be able to use **bubble()** to sort any type of data, we need to make that pointer a pointer to type **void**. But, in doing so we are going to lose information concerning the size of individual elements within the array. So, in bubble\_5.c we will add a separate parameter to handle this size information.

These changes, from bubble4.c to bubble5.c are, perhaps, a bit more extensive than those we have made in the past. So, compare the two modules carefully for differences.

```
/*----- bubble5.c -----*/
```

```
#include <stdio.h>
#include <string.h>
```

```
long arr[10] = { 3,6,1,2,3,8,4,1,7,2};
```

```
void bubble(void *p, size_t width, int N);
```

```
int compare(void *m, void *n);

int main(void)
{
    int i;
    putchar('\n');

    for (i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    bubble(arr, sizeof(long), 10);
    putchar('\n');

    for (i = 0; i < 10; i++)
    {
        printf("%ld ", arr[i]);
    }

    return 0;
}

void bubble(void *p, size_t width, int N)
{
    int i, j;
    unsigned char buf[4];
    unsigned char *bp = p;

    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            if (compare((void *) (bp + width*(j-1)),
                        (void *) (bp + j*width))) /* 1 */
            {
                /* t = p[j-1]; */
                memcpy(buf, bp + width*(j-1), width);
                /* p[j-1] = p[j]; */
                memcpy(bp + width*(j-1), bp + j*width, width);
                /* p[j] = t; */
                memcpy(bp + j*width, buf, width);
            }
        }
    }
}

int compare(void *m, void *n)
{
    long *m1, *n1;
    m1 = (long *)m;
    n1 = (long *)n;
    return (*m1 > *n1);
}

/*----- end of bubble5.c -----*/
```

Note that data type of the array from **int** to **long** to illustrate the changes needed in the **compare()** function. Within **bubble()** variable **t** (which we would have had to change from type **int** to type **long**). Buffer of size 4 unsigned characters is added, which is the size needed to hold a long (this will change again in future modifications to this code). The unsigned

character pointer **\*bp** is used to point to the base of the array to be sorted, i.e. to the first element of that array.

We also had to modify what we passed to **compare()**, and how we do the swapping of elements that the comparison indicates need swapping. Use of **memcpy()** and pointer notation instead of array notation work towards this reduction in type sensitivity.

Again, making a careful comparison of bubble5.c with bubble4.c can result in improved understanding of what is happening and why.

We move now to bubble6.c where we use the same function bubble() that we used in bubble5.c to sort strings instead of long integers. Of course we have to change the comparison function since the means by which strings are compared is different from that by which long integers are compared. And, in bubble6.c we have deleted the lines within **bubble()** that were commented out in bubble5.c.

```
/*----- bubble6.c -----*/
```

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX_BUF 256
```

```
char arr2[5][20] = { "Mickey Mouse",
                    "Donald Duck",
                    "Minnie Mouse",
                    "Goofy",
                    "Ted Jensen" };
```

```
void bubble(void *p, int width, int N);
int compare(void *m, void *n);
```

```
int main(void)
{
    int i;
    putchar('\n');

    for (i = 0; i < 5; i++)
    {
        printf("%s\n", arr2[i]);
    }
    bubble(arr2, 20, 5);
    putchar("\n\n");

    for (i = 0; i < 5; i++)
    {
        printf("%s\n", arr2[i]);
    }
    return 0;
}
```

```
void bubble(void *p, int width, int N)
{
    int i, j, k;
    unsigned char buf[MAX_BUF];
```

```
    unsigned char *bp = p;

    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            k = compare((void *)(bp + width*(j-1)), (void *)(bp + j*width));
            if (k > 0)
            {
                memcpy(buf, bp + width*(j-1), width);
                memcpy(bp + width*(j-1), bp + j*width, width);
                memcpy(bp + j*width, buf, width);
            }
        }
    }
}

int compare(void *m, void *n)
{
    char *m1 = m;
    char *n1 = n;
    return (strcmp(m1,n1));
}

/*----- end of bubble6.c -----*/
```

But, the fact that **bubble()** was unchanged from that used in bubble5.c indicates that that function is capable of sorting a wide variety of data types. What is left to do is to pass to **bubble()** the name of the comparison function we want to use so that it can be truly universal. Just as the name of an array is the address of the first element of the array in the data segment, the name of a function decays into the address of that function in the code segment. Thus we need to use a pointer to a function. In this case the comparison function.

Pointers to functions must match the functions pointed to in the number and types of the parameters and the type of the return value. In our case, we declare our function pointer as:

```
int (*fptr)(const void *p1, const void *p2);
```

Note that were we to write:

```
int *fptr(const void *p1, const void *p2);
```

we would have a function prototype for a function which returned a pointer to type **int**. That is because in C the parenthesis () operator have a higher precedence than the pointer \* operator. By putting the parenthesis around the string (\*fptr) we indicate that we are declaring a function pointer.

We now modify our declaration of **bubble()** by adding, as its 4th parameter, a function pointer of the proper type. It's function prototype becomes:

```
void bubble(void *p, int width, int N,
            int(*fptr)(const void *, const void *));
```

When we call the **bubble()**, we insert the name of the comparison function that we want to use. bubble7.c illustrate how this approach permits the use of the same **bubble()** function for sorting different types of data.

```
/*----- bubble7.c -----*/
```

```
#include <stdio.h>
#include <string.h>
```

```
#define MAX_BUF 256
```

```
long arr[10] = { 3,6,1,2,3,8,4,1,7,2};
char arr2[5][20] = { "Mickey Mouse",
                    "Donald Duck",
```

```
        "Minnie Mouse",
        "Goofy",
        "Ted Jensen" };

void bubble(void *p, int width, int N,
            int(*fptr)(const void *, const void *));
int compare_string(const void *m, const void *n);
int compare_long(const void *m, const void *n);

int main(void)
{
    int i;
    puts("\nBefore Sorting:\n");

    for (i = 0; i < 10; i++)          /* show the long ints */
    {
        printf("%ld ",arr[i]);
    }
    puts("\n");

    for (i = 0; i < 5; i++)           /* show the strings */
    {
        printf("%s\n", arr2[i]);
    }
    bubble(arr, 4, 10, compare_long); /* sort the longs */
    bubble(arr2, 20, 5, compare_string); /* sort the strings */
    puts("\n\nAfter Sorting:\n");

    for (i = 0; i < 10; i++)          /* show the sorted longs */
    {
        printf("%d ",arr[i]);
    }
    puts("\n");

    for (i = 0; i < 5; i++)           /* show the sorted strings */
    {
        printf("%s\n", arr2[i]);
    }
    return 0;
}

void bubble(void *p, int width, int N,
            int(*fptr)(const void *, const void *))
{
    int i, j, k;
    unsigned char buf[MAX_BUF];
    unsigned char *bp = p;

    for (i = N-1; i >= 0; i--)
    {
        for (j = 1; j <= i; j++)
        {
            k = fptr((void *)(bp + width*(j-1)), (void *)(bp + j*width));
            if (k > 0)
            {
                memcpy(buf, bp + width*(j-1), width);
                memcpy(bp + width*(j-1), bp + j*width, width);
                memcpy(bp + j*width, buf, width);
            }
        }
    }
}
```



```
    }  
}  
  
int compare_string(const void *m, const void *n)  
{  
    char *m1 = (char *)m;  
    char *n1 = (char *)n;  
    return (strcmp(m1,n1));  
}  
  
int compare_long(const void *m, const void *n)  
{  
    long *m1, *n1;  
    m1 = (long *)m;  
    n1 = (long *)n;  
    return (*m1 > *n1);  
}  
  
/*----- end of bubble7.c -----*/
```

## 9. Streams and Disk Files

Most of the programs you write will use disk files for one purpose or another: data storage, configuration information, and so on. Today you will learn about

- Relating streams to disk files
- C's two disk file types
- Opening a file
- Writing data to a file
- Reading data from a file
- Closing a file
- Disk file management

C performs all input and output, including disk files, by means of streams. This is one of the advantages of stream input/output--techniques for using one stream can be used with little or no change for other streams. The major difference with disk file streams is that your program must explicitly create a stream associated with a specific disk file.

### 1.31 Types of Disk Files

C streams come in two flavors: text and binary. You can associate either type of stream with a file, and it's important that you understand the distinction in order to use the proper mode for your files.

Text streams are associated with text-mode files. Text-mode files consist of a sequence of lines. Each line contains zero or more characters and ends with one or more characters that signal end-of-line. The maximum line length is 255 characters. It's important to remember that a "line" isn't a C string; there is no terminating NULL character (\0). When you use a text-mode stream, translation occurs between C's newline character (\n) and whatever character(s) the operating system uses to mark end-of-line on disk files. On DOS systems, it's a carriage-return linefeed (CR-LF) combination. When data is written to a text-mode file, each \n is translated to a CR-LF; when data is read from a disk file, each CR-LF is translated to a \n. On UNIX systems, no translation is done--newline characters remain unchanged.

Binary streams are associated with binary-mode files. Any and all data is written and read unchanged, with no separation into lines and no use of end-of-line characters. The NULL and end-of-line characters have no special significance and are treated like any other byte of data. Some file input/output functions are restricted to one file mode, whereas other functions can use either mode. This chapter teaches you which mode to use with which functions.

### 1.32 Filenames

Every disk file has a name, and you must use filenames when dealing with disk files. Filenames are stored as strings, just like other text data. The rules as to what is acceptable for filenames and what is not differ from one operating system to another. In DOS and Windows 3.x, a complete filename consists of a name that has from one to eight characters, optionally followed by a period and an extension that has from one to three characters. In contrast, the Windows 95 and Windows NT operating systems, as well as most UNIX systems, permit filenames up to 256 characters long.

Operating systems also differ in the characters that are permitted in filenames. In Windows 95, for example, the following characters are not permitted:

/ \ : \* ? " < > |

You must be aware of the filename rules of whichever operating system you're writing for.

A filename in a C program also can contain path information. The *path* specifies the drive and/or directory (or folder) where the file is located. If you specify a filename without a path, it will be assumed that the file is located at whatever location the operating system currently

designates as the default. It's good programming practice to always specify path information as part of your filenames.

On PCs, the backslash character is used to separate directory names in a path. For example, to DOS and Windows, the name

c:\data\list.txt

refers to a file named LIST.TXT in the directory \DATA on drive C. Remember that the backslash character has a special meaning to C when it's in a string. To represent the backslash character itself, you must precede it with another backslash. Thus, in a C program, you would represent the filename as follows:

```
char *filename = "c:\\data\\list.txt";
```

If you're entering a filename using the keyboard, however, enter only a single backslash. Not all systems use the backslash as the directory separator. For example, UNIX uses the forward slash (/).

## 1.33 Opening a File

The process of creating a stream linked to a disk file is called *opening* the file. When you open a file, it becomes available for reading (meaning that data is input from the file to the program), writing (meaning that data from the program is saved in the file), or both. When you're done using the file, you must close it. Closing a file is covered later in this chapter.

To open a file, you use the `fopen()` library function. The prototype of `fopen()` is located in `STDIO.H` and reads as follows:

```
FILE *fopen(const char *filename, const char *mode);
```

This prototype tells you that `fopen()` returns a pointer to type `FILE`, which is a structure declared in `STDIO.H`. The members of the `FILE` structure are used by the program in the various file access operations, but you don't need to be concerned about them. However, for each file that you want to open, you must declare a pointer to type `FILE`. When you call `fopen()`, that function creates an instance of the `FILE` structure and returns a pointer to that structure. You use this pointer in all subsequent operations on the file. If `fopen()` fails, it returns `NULL`. Such a failure could be caused, for example, by a hardware error or by trying to open a file on a diskette that hasn't been formatted.

The argument *filename* is the name of the file to be opened. As noted earlier, *filename* can--and should--contain a path specification. The *filename* argument can be a literal string enclosed in double quotation marks or a pointer to a string variable.

The argument *mode* specifies the mode in which to open the file. In this context, *mode* controls whether the file is binary or text and whether it is for reading, writing, or both. The permitted values for *mode* are listed in Table 16.1.

**Table 16.1. Values of mode for the fopen() function.**

| <i>m<br/>od<br/>e</i> | Meaning                                                                                                                                                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| r                     | Opens the file for reading. If the file doesn't exist, <code>fopen()</code> returns <code>NULL</code> .                                                                                                  |
| w                     | Opens the file for writing. If a file of the specified name doesn't exist, it is created. If a file of the specified name does exist, it is deleted without warning, and a new, empty file is created.   |
| a                     | Opens the file for appending. If a file of the specified name doesn't exist, it is created. If the file does exist, new data is appended to the end of the file.                                         |
| r+                    | Opens the file for reading and writing. If a file of the specified name doesn't exist, it is created. If the file does exist, new data is added to the beginning of the file, overwriting existing data. |
| w+                    | Opens the file for reading and writing. If a file of the specified name doesn't exist, it is created. If the file does exist, it is overwritten.                                                         |
| a+                    | Opens a file for reading and appending. If a file of the specified name doesn't exist, it is created. If the                                                                                             |

|                                                               |
|---------------------------------------------------------------|
| file does exist, new data is appended to the end of the file. |
|---------------------------------------------------------------|

The default file mode is text. To open a file in binary mode, you append a *b* to the *mode* argument. Thus, a *mode* argument of *a* would open a text-mode file for appending, whereas *ab* would open a binary-mode file for appending.

Remember that `fopen()` returns `NULL` if an error occurs. Error conditions that can cause a return value of `NULL` include the following:

- Using an invalid filename.
- Trying to open a file on a disk that isn't ready (the drive door isn't closed or the disk isn't formatted, for example).
- Trying to open a file in a nonexistent directory or on a nonexistent disk drive.
- Trying to open a nonexistent file in mode *r*.

Whenever you use `fopen()`, you need to test for the occurrence of an error. There's no way to tell exactly which error occurred, but you can display a message to the user and try to open the file again, or you can end the program. Most C compilers include non-ANSI extensions that let you obtain information about the nature of the error; refer to your compiler documentation for information.

Listing 16.1 demonstrates `fopen()`.

**Listing 16.1. Using `fopen()` to open disk files in various modes.**

```
1: /* Demonstrates the fopen() function. */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: main()
6: {
7:     FILE *fp;
8:     char filename[40], mode[4];
9:
10:    while (1)
11:    {
12:
13:        /* Input filename and mode. */
14:
15:        printf("\nEnter a filename: ");
16:        gets(filename);
17:        printf("\nEnter a mode (max 3 characters): ");
18:        gets(mode);
19:
20:        /* Try to open the file. */
21:
22:        if ( (fp = fopen( filename, mode )) != NULL )
23:        {
24:            printf("\nSuccessful opening %s in mode %s.\n",
25:                filename, mode);
26:            fclose(fp);
27:            puts("Enter x to exit, any other to continue.");
28:            if ( (getc(stdin)) == 'x' )
29:                break;
30:            else
31:                continue;
32:        }
33:        else
34:        {
35:            fprintf(stderr, "\nError opening file %s in mode %s.\n",
36:                filename, mode);
37:            puts("Enter x to exit, any other to try again.");
38:            if ( (getc(stdin)) == 'x' )
39:                break;
```

```
40:         else
41:             continue;
42:     }
43: }
44: }
Enter a filename: junk.txt
Enter a mode (max 3 characters): w
Successful opening junk.txt in mode w.
Enter x to exit, any other to continue.
```

```
j
Enter a filename: morejunk.txt
Enter a mode (max 3 characters): r
Error opening morejunk.txt in mode r.
Enter x to exit, any other to try again.
```

**x**

**]**

**ANALYSIS:** This program prompts you for both the filename and the mode specifier on lines 15 through 18. After getting the names, line 22 attempts to open the file and assign its file pointer to fp. As an example of good programming practice, the if statement on line 22 checks to see that the opened file's pointer isn't equal to NULL. If fp isn't equal to NULL, a message stating that the open was successful and that the user can continue is printed. If the file pointer is NULL, the else condition of the if loop executes. The else condition on lines 33 through 42 prints a message stating that there was a problem. It then prompts the user to determine whether the program should continue.

You can experiment with different names and modes to see which ones give you an error. In the output just shown, you can see that trying to open MOREJUNK.TXT in mode r resulted in an error because the file didn't exist on the disk. If an error occurs, you're given the choice of entering the information again or quitting the program. To force an error, you could enter an invalid filename such as [].

## 1.34 Writing and Reading File Data

A program that uses a disk file can write data to a file, read data from a file, or a combination of the two. You can write data to a disk file in three ways:

- You can use formatted output to save formatted data to a file. You should use formatted output only with text-mode files. The primary use of formatted output is to create files containing text and numeric data to be read by other programs such as spreadsheets or databases. You rarely, if ever, use formatted output to create a file to be read again by a C program.
- You can use character output to save single characters or lines of characters to a file. Although technically it's possible to use character output with binary-mode files, it can be tricky. You should restrict character-mode output to text files. The main use of character output is to save text (but not numeric) data in a form that can be read by C, as well as other programs such as word processors.
- You can use direct output to save the contents of a section of memory directly to a disk file. This method is for binary files only. Direct output is the best way to save data for later use by a C program.

When you want to read data from a file, you have the same three options: formatted input, character input, or direct input. The type of input you use in a particular case depends almost entirely on the nature of the file being read. Generally, you will read data in the same mode that it was saved in, but this is not a requirement. However, reading a file in a mode different from the one it was written in requires a thorough knowledge of C and file formats.

The previous descriptions of the three types of file input and output suggest tasks best suited for each type of output. This is by no means a set of strict rules. The C language is very flexible (this is one of its advantages!), so a clever programmer can make any type of file

output suit almost any need. As a beginning programmer, it might make things easier if you follow these guidelines, at least initially.

## *Formatted File Input and Output*

Formatted file input/output deals with text and numeric data that is formatted in a specific way. It is directly analogous to formatted keyboard input and screen output done with the `printf()` and `scanf()` functions

## *Formatted File Output*

Formatted file output is done with the library function `fprintf()`. The prototype of `fprintf()` is in the header file `STDIO.H`, and it reads as follows:

```
int fprintf(FILE *fp, char *fmt, ...);
```

The first argument is a pointer to type `FILE`. To write data to a particular disk file, you pass the pointer that was returned when you opened the file with `fopen()`.

The second argument is the format string.. The format string used by `fprintf()` follows exactly the same rules as `printf()`.

In a function prototype, ellipses represent a variable number of arguments. In other words, in addition to the file pointer and the format string arguments, `fprintf()` takes zero, one, or more additional arguments. This is just like `printf()`. These arguments are the names of the variables to be output to the specified stream.

Remember, `fprintf()` works just like `printf()`, except that it sends its output to the stream specified in the argument list. In fact, if you specify a stream argument of `stdout`, `fprintf()` is identical to `printf()`.

Listing 16.2 demonstrates the use of `fprintf()`.

### **Listing 16.2. The equivalence of `fprintf()` formatted output to both a file and to `stdout`.**

```
1: /* Demonstrates the fprintf() function. */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: void clear_kb(void);
6:
7: main()
8: {
9:     FILE *fp;
10:    float data[5];
11:    int count;
12:    char filename[20];
13:
14:    puts("Enter 5 floating-point numerical values.");
15:
16:    for (count = 0; count < 5; count++)
17:        scanf("%f", &data[count]);
18:
19:    /* Get the filename and open the file. First clear stdin */
20:    /* of any extra characters. */
21:
22:    clear_kb();
23:
24:    puts("Enter a name for the file.");
25:    gets(filename);
26:
27:    if ( (fp = fopen(filename, "w")) == NULL)
28:    {
29:        fprintf(stderr, "Error opening file %s.", filename);
30:        exit(1);
31:    }
```

```
32:
33:  /* Write the numerical data to the file and to stdout. */
34:
35:  for (count = 0; count < 5; count++)
36:  {
37:      fprintf(fp, "\ndata[%d] = %f", count, data[count]);
38:      fprintf(stdout, "\ndata[%d] = %f", count, data[count]);
39:  }
40:  fclose(fp);
41:  printf("\n");
42:  return(0);
43: }
44:
45: void clear_kb(void)
46: /* Clears stdin of any waiting characters. */
47: {
48:     char junk[80];
49:     gets(junk);
50: }
Enter 5 floating-point numerical values.
3.14159
9.99
1.50
3.
1000.0001
```

Enter a name for the file.

**numbers.txt**

```
data[0] = 3.141590
data[1] = 9.990000
data[2] = 1.500000
data[3] = 3.000000
data[4] = 1000.000122
```

**ANALYSIS:** You might wonder why the program displays 1000.000122 when the value you entered was 1000.0001. This isn't an error in the program. It's a normal consequence of the way C stores numbers internally. Some floating-point values can't be stored exactly, so minor inaccuracies such as this one sometimes result.

This program uses `fprintf()` on lines 37 and 38 to send some formatted text and numeric data to `stdout` and to the disk file whose name you specified. The only difference between the two lines is the first argument--that is, the stream to which the data is sent. After running the program, use your editor to look at the contents of the file `NUMBERS.TXT` (or whatever name you assigned to it), which will be in the same directory as the program files. You'll see that the text in the file is an exact copy of the text that was displayed on-screen.

Note that Listing 16.2 uses the `clear_kb()` function. This is necessary to remove from `stdin` any extra characters that might be left over from the call to `scanf()`. If you don't clear `stdin`, these extra characters (specifically, the newline) are read by the `gets()` that inputs the filename, and the result is a file creation error.

## Formatted File Input

For formatted file input, use the `fscanf()` library function, which is used like `scanf()`, except that input comes from a specified stream instead of from `stdin`. The prototype for `fscanf()` is `int fscanf(FILE *fp, const char *fmt, ...);`

The argument `fp` is the pointer to type `FILE` returned by `fopen()`, and `fmt` is a pointer to the format string that specifies how `fscanf()` is to read the input. The components of the format string are the same as for `scanf()`. Finally, the ellipses (...) indicate one or more additional arguments, the addresses of the variables where `fscanf()` is to assign the input.

Before getting started with `fscanf()`, you might want to review the section on `scanf()`. The function `fscanf()` works exactly the same as `scanf()`, except that characters are taken from the specified stream rather than from `stdin`.

To demonstrate `fscanf()`, you need a text file containing some numbers or strings in a format that can be read by the function. Use your editor to create a file named `INPUT.TXT`, and enter five floating-point numbers with some space between them (spaces or newlines). For example, your file might look like this:

```
123.45  87.001
100.02
0.00456  1.0005
```

Now, compile and run Listing 16.3.

**Listing 16.3. Using `fscanf()` to read formatted data from a disk file.**

```
1: /* Reading formatted file data with fscanf(). */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: main()
6: {
7:     float f1, f2, f3, f4, f5;
8:     FILE *fp;
9:
10:    if ( (fp = fopen("INPUT.TXT", "r")) == NULL)
11:    {
12:        fprintf(stderr, "Error opening file.\n");
13:        exit(1);
14:    }
15:
16:    fscanf(fp, "%f %f %f %f %f", &f1, &f2, &f3, &f4, &f5);
17:    printf("The values are %f, %f, %f, %f, and %f\n.",
18:        f1, f2, f3, f4, f5);
19:
20:    fclose(fp);
21:    return(0);
22: }
```

The values are 123.45, 87.0001, 100.02, 0.00456, and 1.0005.

---

**NOTE:** The precision of the values might cause some numbers to not display as the exact values you entered. For example, 100.02 might appear as 100.01999.

---

**ANALYSIS:** This program reads the five values from the file you created and then displays them on-screen. The `fopen()` call on line 10 opens the file for read mode. It also checks to see that the file opened correctly. If the file wasn't opened, an error message is displayed on line 12, and the program exits (line 13). Line 16 demonstrates the use of the `fscanf()` function. With the exception of the first parameter, `fscanf()` is identical to `scanf()`, which you have been using throughout this book. The first parameter points to the file that you want the program to read. You can do further experiments with `fscanf()`, creating input files with your programming editor and seeing how `fscanf()` reads the data.

## *Character Input and Output*

When used with disk files, the term *character I/O* refers to single characters as well as lines of characters. Remember, a line is a sequence of zero or more characters terminated by the newline character. Use character I/O with text-mode files. The following sections describe character input/output functions, and then you'll see a demonstration program.



## Character Input

There are three character input functions: `getc()` and `fgetc()` for single characters, and `fgets()` for lines.

### The `getc()` and `fgetc()` Functions

The functions `getc()` and `fgetc()` are identical and can be used interchangeably. They input a single character from the specified stream. Here is the prototype of `getc()`, which is in `STDIO.H`:

```
int getc(FILE *fp);
```

The argument `fp` is the pointer returned by `fopen()` when the file is opened. The function returns the character that was input or EOF on error.

You've seen `getc()` used in earlier programs to input a character from the keyboard. This is another example of the flexibility of C's streams--the same function can be used for keyboard or file input.

If `getc()` and `fgetc()` return a single character, why are they prototyped to return a type `int`? The reason is that, when reading files, you need to be able to read in the end-of-file marker, which on some systems isn't a type `char` but a type `int`. You'll see `getc()` in action later, in Listing 16.10.

### The `fgets()` Function

To read a line of characters from a file, use the `fgets()` library function. The prototype is

```
char *fgets(char *str, int n, FILE *fp);
```

The argument `str` is a pointer to a buffer in which the input is to be stored, `n` is the maximum number of characters to be input, and `fp` is the pointer to type `FILE` that was returned by `fopen()` when the file was opened.

When called, `fgets()` reads characters from `fp` into memory, starting at the location pointed to by `str`. Characters are read until a newline is encountered or until `n-1` characters have been read, whichever occurs first. By setting `n` equal to the number of bytes allocated for the buffer `str`, you prevent input from overwriting memory beyond allocated space. (The `n-1` is to allow space for the terminating `\0` that `fgets()` adds to the end of the string.) If successful, `fgets()` returns `str`. Two types of errors can occur, as indicated by the return value of `NULL`:

- If a read error or EOF is encountered before any characters have been assigned to `str`, `NULL` is returned, and the memory pointed to by `str` is unchanged.
- If a read error or EOF is encountered after one or more characters have been assigned to `str`, `NULL` is returned, and the memory pointed to by `str` contains garbage.

You can see that `fgets()` doesn't necessarily input an entire line (that is, everything up to the next newline character). If `n-1` characters are read before a newline is encountered, `fgets()` stops. The next read operation from the file starts where the last one leaves off. To be sure that `fgets()` reads in entire strings, stopping only at newlines, be sure that the size of your input buffer and the corresponding value of `n` passed to `fgets()` are large enough.

## Character Output

You need to know about two character output functions: `putc()` and `fputs()`.

### The `putc()` Function

The library function `putc()` writes a single character to a specified stream. Its prototype in `STDIO.H` reads

```
int putc(int ch, FILE *fp);
```

The argument `ch` is the character to output. As with other character functions, it is formally called a type `int`, but only the lower-order byte is used. The argument `fp` is the pointer associated with the file (the pointer returned by `fopen()` when the file was opened). The function `putc()` returns the character just written if successful or EOF if an error occurs. The symbolic constant `EOF` is defined in `STDIO.H`, and it has the value `-1`. Because no "real" character has that numeric value, `EOF` can be used as an error indicator (with text-mode files only).

## The fputs() Function

To write a line of characters to a stream, use the library function `fputs()`. This function works just like `puts()`. The only difference is that with `fputs()` you can specify the output stream. Also, `fputs()` doesn't add a newline to the end of the string; if you want it, you must explicitly include it. Its prototype in `STDIO.H` is

```
char fputs(char *str, FILE *fp);
```

The argument `str` is a pointer to the null-terminated string to be written, and `fp` is the pointer to type `FILE` returned by `fopen()` when the file was opened. The string pointed to by `str` is written to the file, minus its terminating `\0`. The function `fputs()` returns a nonnegative value if successful or EOF on error.

## Direct File Input and Output

You use direct file I/O most often when you save data to be read later by the same or a different C program. Direct I/O is used only with binary-mode files. With direct output, blocks of data are written from memory to disk. Direct input reverses the process: A block of data is read from a disk file into memory. For example, a single direct-output function call can write an entire array of type `double` to disk, and a single direct-input function call can read the entire array from disk back into memory. The direct I/O functions are `fread()` and `fwrite()`.

### The fwrite() Function

The `fwrite()` library function writes a block of data from memory to a binary-mode file. Its prototype in `STDIO.H` is

```
int fwrite(void *buf, int size, int count, FILE *fp);
```

The argument `buf` is a pointer to the region of memory holding the data to be written to the file. The pointer type is `void`; it can be a pointer to anything.

The argument `size` specifies the size, in bytes, of the individual data items, and `count` specifies the number of items to be written. For example, if you wanted to save a 100-element integer array, `size` would be 2 (because each `int` occupies 2 bytes) and `count` would be 100 (because the array contains 100 elements). To obtain the `size` argument, you can use the `sizeof()` operator.

The argument `fp` is, of course, the pointer to type `FILE`, returned by `fopen()` when the file was opened. The `fwrite()` function returns the number of items written on success; if the value returned is less than `count`, it means that an error has occurred. To check for errors, you usually program `fwrite()` as follows:

```
if( (fwrite(buf, size, count, fp)) != count)
    fprintf(stderr, "Error writing to file.");
```

Here are some examples of using `fwrite()`. To write a single type `double` variable `x` to a file, use the following:

```
fwrite(&x, sizeof(double), 1, fp);
```

To write an array `data[]` of 50 structures of type `address` to a file, you have two choices:

```
fwrite(data, sizeof(address), 50, fp);
```

```
fwrite(data, sizeof(data), 1, fp);
```

The first method writes the array as 50 elements, with each element having the size of a single type `address` structure. The second method treats the array as a single element. The two methods accomplish exactly the same thing.

The following section explains `fread()` and then presents a program demonstrating `fread()` and `fwrite()`.

### The fread() Function

The `fread()` library function reads a block of data from a binary-mode file into memory. Its prototype in `STDIO.H` is

```
int fread(void *buf, int size, int count, FILE *fp);
```

The argument `buf` is a pointer to the region of memory that receives the data read from the file. As with `fwrite()`, the pointer type is `void`.

The argument `size` specifies the size, in bytes, of the individual data items being read, and `count` specifies the number of items to read. Note how these arguments parallel the arguments used by `fwrite()`. Again, the `sizeof()` operator is typically used to provide the `size`

argu-ment. The argument *fp* is (as always) the pointer to type `FILE` that was returned by `fopen()` when the file was opened. The `fread()` function returns the number of items read; this can be less than *count* if end-of-file was reached or an error occurred.

Listing 16.4 demonstrates the use of `fwrite()` and `fread()`.

**Listing 16.4. Using `fwrite()` and `fread()` for direct file access.**

```
1: /* Direct file I/O with fwrite() and fread(). */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: #define SIZE 20
6:
7: main()
8: {
9:     int count, array1[SIZE], array2[SIZE];
10:    FILE *fp;
11:
12:    /* Initialize array1[]. */
13:
14:    for (count = 0; count < SIZE; count++)
15:        array1[count] = 2 * count;
16:
17:    /* Open a binary mode file. */
18:
19:    if ( (fp = fopen("direct.txt", "wb")) == NULL)
20:    {
21:        fprintf(stderr, "Error opening file.");
22:        exit(1);
23:    }
24:    /* Save array1[] to the file. */
25:
26:    if (fwrite(array1, sizeof(int), SIZE, fp) != SIZE)
27:    {
28:        fprintf(stderr, "Error writing to file.");
29:        exit(1);
30:    }
31:
32:    fclose(fp);
33:
34:    /* Now open the same file for reading in binary mode. */
35:
36:    if ( (fp = fopen("direct.txt", "rb")) == NULL)
37:    {
38:        fprintf(stderr, "Error opening file.");
39:        exit(1);
40:    }
41:
42:    /* Read the data into array2[]. */
43:
44:    if (fread(array2, sizeof(int), SIZE, fp) != SIZE)
45:    {
46:        fprintf(stderr, "Error reading file.");
47:        exit(1);
48:    }
49:
50:    fclose(fp);
51:
52:    /* Now display both arrays to show they're the same. */
53:
54:    for (count = 0; count < SIZE; count++)
```

```
55:    printf("%d\t%d\n", array1[count], array2[count]);
56:    return(0);
57: }
0    0
2    2
4    4
6    6
8    8
10   10
12   12
14   14
16   16
18   18
20   20
22   22
24   24
26   26
28   28
30   30
32   32
34   34
36   36
38   38
```

**ANALYSIS:** Listing 16.4 demonstrates the use of the `fwrite()` and `fread()` functions. This program initializes an array on lines 14 and 15. It then uses `fwrite()` on line 26 to save the array to disk. The program uses `fread()` on line 44 to read the data into a different array. Finally, it displays both arrays on-screen to show that they now hold the same data (lines 54 and 55).

When you save data with `fwrite()`, not much can go wrong besides some type of disk error. With `fread()`, you need to be careful, however. As far as `fread()` is concerned, the data on the disk is just a sequence of bytes. The function has no way of knowing what it represents. For example, on a 16-bit system, a block of 100 bytes could be 100 char variables, 50 int variables, 25 long variables, or 25 float variables. If you ask `fread()` to read that block into memory, it obediently does so. However, if the block was saved from an array of type int and you retrieve it into an array of type float, no error occurs, but you get strange results. When writing programs, you must be sure that `fread()` is used properly, reading data into the appropriate types of variables and arrays. Notice that in Listing 16.4, all calls to `fopen()`, `fwrite()`, and `fread()` are checked to ensure that they worked correctly.

## 1.35 File Buffering: Closing and Flushing Files

When you're done using a file, you should close it using the `fclose()` function. You saw `fclose()` used in programs presented earlier in this chapter. Its prototype is

```
int fclose(FILE *fp);
```

The argument `fp` is the FILE pointer associated with the stream; `fclose()` returns 0 on success or -1 on error. When you close a file, the file's buffer is flushed (written to the file). You can also close all open streams except the standard ones (`stdin`, `stdout`, `stderr`, and `stdaux`) by using the `fcloseall()` function. Its prototype is

```
int fcloseall(void);
```

This function also flushes any stream buffers and returns the number of streams closed. When a program terminates (either by reaching the end of `main()` or by executing the `exit()` function), all streams are automatically flushed and closed. However, it's a good idea to close streams explicitly--particularly those linked to disk files--as soon as you're finished with them.

The reason has to do with stream buffers.

When you create a stream linked to a disk file, a buffer is automatically created and associated with the stream. A buffer is a block of memory used for temporary storage of data

being written to and read from the file. Buffers are needed because disk drives are block-oriented devices, which means that they operate most efficiently when data is read and written in blocks of a certain size. The size of the ideal block differs, depending on the specific hardware in use. It's typically on the order of a few hundred to a thousand bytes. You don't need to be concerned about the exact block size, however.

The buffer associated with a file stream serves as an interface between the stream (which is character-oriented) and the disk hardware (which is block-oriented). As your program writes data to the stream, the data is saved in the buffer until the buffer is full, and then the entire contents of the buffer are written, as a block, to the disk. An analogous process occurs when reading data from a disk file. The creation and operation of the buffer are handled by the operating system and are entirely automatic; you don't have to be concerned with them. (C does offer some functions for buffer manipulation, but they are beyond the scope of this book.)

In practical terms, this buffer operation means that, during program execution, data that your program wrote to the disk might still be in the buffer, not on the disk. If your program hangs up, if there's a power failure, or if some other problem occurs, the data that's still in the buffer might be lost, and you won't know what's contained in the disk file.

You can flush a stream's buffers without closing it by using the `fflush()` or `flushall()` library functions. Use `fflush()` when you want a file's buffer to be written to disk while still using the file. Use `flushall()` to flush the buffers of all open streams. The prototypes of these two functions are as follows:

```
int fflush(FILE *fp);  
int flushall(void);
```

The argument `fp` is the `FILE` pointer returned by `fopen()` when the file was opened. If a file was opened for writing, `fflush()` writes its buffer to disk. If the file was opened for reading, the buffer is cleared. The function `fflush()` returns 0 on success or EOF if an error occurred. The function `flushall()` returns the number of open streams.

---

---

## 1.36 Sequential Versus Random File Access

Every open file has a file position indicator associated with it. The position indicator specifies where read and write operations take place in the file. The position is always given in terms of bytes from the beginning of the file. When a new file is opened, the position indicator is always at the beginning of the file, position 0. (Because the file is new and has a length of 0, there's no other location to indicate.) When an existing file is opened, the position indicator is at the end of the file if the file was opened in append mode, or at the beginning of the file if the file was opened in any other mode.

The file input/output functions covered earlier in this chapter make use of the position indicator, although the manipulations go on behind the scenes. Writing and reading operations occur at the location of the position indicator and update the position indicator as well. For example, if you open a file for reading, and 10 bytes are read, you input the first 10 bytes in the file (the bytes at positions 0 through 9). After the read operation, the position indicator is at position 10, and the next read operation begins there. Thus, if you want to read all the data in a file sequentially or write data to a file sequentially, you don't need to be concerned about the position indicator, because the stream I/O functions take care of it automatically.

When you need more control, use the C library functions that let you determine and change the value of the file position indicator. By controlling the position indicator, you can perform random file access. Here, *random* means that you can read data from, or write data to, any position in a file without reading or writing all the preceding data.

## *The ftell() and rewind() Functions*

To set the position indicator to the beginning of the file, use the library function `rewind()`. Its prototype, in `STDIO.H`, is  
`void rewind(FILE *fp);`

The argument `fp` is the `FILE` pointer associated with the stream. After `rewind()` is called, the file's position indicator is set to the beginning of the file (byte 0). Use `rewind()` if you've read some data from a file and you want to start reading from the beginning of the file again without closing and reopening the file.

To determine the value of a file's position indicator, use `ftell()`. This function's prototype, located in `STDIO.H`, reads  
`long ftell(FILE *fp);`

The argument `fp` is the `FILE` pointer returned by `fopen()` when the file was opened. The function `ftell()` returns a type `long` that gives the current file position in bytes from the start of the file (the first byte is at position 0). If an error occurs, `ftell()` returns `-1L` (a type `long -1`). To get a feel for the operation of `rewind()` and `ftell()`, look at Listing 16.5.

### **Listing 16.5. Using ftell() and rewind().**

```
1: /* Demonstrates ftell() and rewind(). */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: #define BUFLen 6
6:
7: char msg[] = "abcdefghijklmnopqrstuvwxyz";
8:
9: main()
10: {
11:     FILE *fp;
12:     char buf[BUFLen];
13:
14:     if ( (fp = fopen("TEXT.TXT", "w")) == NULL)
15:     {
16:         fprintf(stderr, "Error opening file.");
17:         exit(1);
18:     }
19:
20:     if (fputs(msg, fp) == EOF)
21:     {
22:         fprintf(stderr, "Error writing to file.");
23:         exit(1);
24:     }
25:
26:     fclose(fp);
27:
28:     /* Now open the file for reading. */
29:
30:     if ( (fp = fopen("TEXT.TXT", "r")) == NULL)
31:     {
32:         fprintf(stderr, "Error opening file.");
33:         exit(1);
34:     }
35:     printf("\nImmediately after opening, position = %ld", ftell(fp));
36:
37:     /* Read in 5 characters. */
```

```
38:
39:  fgets(buf, BUFLen, fp);
40:  printf("\nAfter reading in %s, position = %ld", buf, ftell(fp));
41:
42:  /* Read in the next 5 characters. */
43:
44:  fgets(buf, BUFLen, fp);
45:  printf("\n\nThe next 5 characters are %s, and position now = %ld",
46:        buf, ftell(fp));
47:
48:  /* Rewind the stream. */
49:
50:  rewind(fp);
51:
52:  printf("\n\nAfter rewinding, the position is back at %ld",
53:        ftell(fp));
54:
55:  /* Read in 5 characters. */
56:
57:  fgets(buf, BUFLen, fp);
58:  printf("\n\nand reading starts at the beginning again: %s\n", buf);
59:  fclose(fp);
60:  return(0);
61: }
```

Immediately after opening, position = 0

After reading in abcde, position = 5

The next 5 characters are fghij, and position now = 10

After rewinding, the position is back at 0

and reading starts at the beginning again: abcde

**ANALYSIS:** This program writes a string, msg, to a file called TEXT.TXT. The message consists of the 26 letters of the alphabet, in order. Lines 14 through 18 open TEXT.TXT for writing and test to ensure that the file was opened successfully. Lines 20 through 24 write msg to the file using fputs() and check to ensure that the write was successful. Line 26 closes the file with fclose(), completing the process of creating a file for the rest of the program to use.

Lines 30 through 34 open the file again, only this time for reading. Line 35 prints the return value of ftell(). Notice that this position is at the beginning of the file. Line 39 performs a gets() to read five characters. The five characters and the new file position are printed on line 40. Notice that ftell() returns the correct offset. Line 50 calls rewind() to put the pointer back at the beginning of the file, before line 52 prints the file position again. This should confirm for you that rewind() resets the position. An additional read on line 57 further confirms that the program is indeed back at the beginning of the file. Line 59 closes the file before ending the program.

## The fseek() Function

More precise control over a stream's position indicator is possible with the fseek() library function. By using fseek(), you can set the position indicator anywhere in the file. The function prototype, in STDIO.H, is

```
int fseek(FILE *fp, long offset, int origin);
```

The argument fp is the FILE pointer associated with the file. The distance that the position indicator is to be moved is given by offset in bytes. The argument origin specifies the move's relative starting point. There can be three values for origin, with symbolic constants defined in IO.H, as shown in Table 16.2.

**Table 16.2. Possible origin values for fseek().**

| Constant | Value | Description |
|----------|-------|-------------|
|----------|-------|-------------|

|          |   |                                                                         |
|----------|---|-------------------------------------------------------------------------|
| SEEK_SET | 0 | Moves the indicator <i>offset</i> bytes from the beginning of the file. |
| SEEK_CUR | 1 | Moves the indicator <i>offset</i> bytes from its current position.      |
| SEEK_END | 2 | Moves the indicator <i>offset</i> bytes from the end of the file.       |

The function `fseek()` returns 0 if the indicator was successfully moved or nonzero if an error occurred. Listing 16.6 uses `fseek()` for random file access.

**Listing 16.6. Random file access with `fseek()`.**

```
1: /* Random access with fseek(). */
2:
3: #include <stdlib.h>
4: #include <stdio.h>
5:
6: #define MAX 50
7:
8: main()
9: {
10:     FILE *fp;
11:     int data, count, array[MAX];
12:     long offset;
13:
14:     /* Initialize the array. */
15:
16:     for (count = 0; count < MAX; count++)
17:         array[count] = count * 10;
18:
19:     /* Open a binary file for writing. */
20:
21:     if ( (fp = fopen("RANDOM.DAT", "wb")) == NULL)
22:     {
23:         fprintf(stderr, "\nError opening file.");
24:         exit(1);
25:     }
26:
27:     /* Write the array to the file, then close it. */
28:
29:     if ( (fwrite(array, sizeof(int), MAX, fp)) != MAX)
30:     {
31:         fprintf(stderr, "\nError writing data to file.");
32:         exit(1);
33:     }
34:
35:     fclose(fp);
36:
37:     /* Open the file for reading. */
38:
39:     if ( (fp = fopen("RANDOM.DAT", "rb")) == NULL)
40:     {
41:         fprintf(stderr, "\nError opening file.");
42:         exit(1);
43:     }
44:
45:     /* Ask user which element to read. Input the element */
46:     /* and display it, quitting when -1 is entered. */
47:
48:     while (1)
49:     {
```



```
50:    printf("\nEnter element to read, 0-%d, -1 to quit: ",MAX-1);
51:    scanf("%ld", &offset);
52:
53:    if (offset < 0)
54:        break;
55:    else if (offset > MAX-1)
56:        continue;
57:
58:    /* Move the position indicator to the specified element. */
59:
60:    if ( (fseek(fp, (offset*sizeof(int)), SEEK_SET)) != 0)
61:    {
62:        fprintf(stderr, "\nError using fseek().");
63:        exit(1);
64:    }
65:
66:    /* Read in a single integer. */
67:
68:    fread(&data, sizeof(int), 1, fp);
69:
70:    printf("\nElement %ld has value %d.", offset, data);
71: }
72:
73: fclose(fp);
74: return(0);
75: }
```

Enter element to read, 0-49, -1 to quit: **5**

Element 5 has value 50.

Enter element to read, 0-49, -1 to quit: **6**

Element 6 has value 60.

Enter element to read, 0-49, -1 to quit: **49**

Element 49 has value 490.

Enter element to read, 0-49, -1 to quit: **1**

Element 1 has value 10.

Enter element to read, 0-49, -1 to quit: **0**

Element 0 has value 0.

Enter element to read, 0-49, -1 to quit: **-1**

**ANALYSIS:** Lines 14 through 35 are similar to Listing 16.5. Lines 16 and 17 initialize an array called `data` with 50 type `int` values. The value stored in each array element is equal to 10 times the index. Then the array is written to a binary file called `RANDOM.DAT`. You know it is binary because the file was opened with mode `"wb"` on line 21.

Line 39 reopens the file in binary read mode before going into an infinite while loop. The while loop prompts users to enter the number of the array element that they want to read. Notice that lines 53 through 56 check to see that the entered element is within the range of the file. Does C let you read an element that is beyond the end of the file? Yes. Like going beyond the end of an array with values, C also lets you read beyond the end of a file. If you do read beyond the end (or before the beginning), your results are unpredictable. It's always best to check what you're doing (as lines 53 through 56 do in this listing).

After you have input the element to find, line 60 jumps to the appropriate offset with a call to `fseek()`. Because `SEEK_SET` is being used, the seek is done from the beginning of the file. Notice that the distance into the file is not just *offset*, but *offset* multiplied by the size of the elements being read. Line 68 then reads the value, and line 70 prints it.

## 1.37 Detecting the End of a File

Sometimes you know exactly how long a file is, so there's no need to be able to detect the file's end. For example, if you used `fwrite()` to save a 100-element integer array, you know the file is 200 bytes long (assuming 2-byte integers). At other times, however, you don't know how

long the file is, but you still want to read data from the file, starting at the beginning and proceeding to the end. There are two ways to detect end-of-file.

When reading from a text-mode file character-by-character, you can look for the end-of-file character. The symbolic constant EOF is defined in STDIO.H as -1, a value never used by a "real" character. When a character input function reads EOF from a text-mode stream, you can be sure that you've reached the end of the file. For example, you could write the following:

```
while ( (c = fgetc( fp )) != EOF )
```

With a binary-mode stream, you can't detect the end-of-file by looking for -1, because a byte of data from a binary stream could have that value, which would result in premature end of input. Instead, you can use the library function feof(), which can be used for both binary- and text-mode files:

```
int feof(FILE *fp);
```

The argument fp is the FILE pointer returned by fopen() when the file was opened. The function feof() returns 0 if the end of file fp hasn't been reached, or a nonzero value if end-of-file has been reached. If a call to feof() detects end-of-file, no further read operations are permitted until a rewind() has been done, fseek() is called, or the file is closed and reopened. Listing 16.7 demonstrates the use of feof(). When you're prompted for a filename, enter the name of any text file--one of your C source files, for example, or a header file such as STDIO.H. Just be sure that the file is in the current directory, or else enter a path as part of the filename. The program reads the file one line at a time, displaying each line on stdout, until feof() detects end-of-file.

#### **Listing 16.7. Using feof() to detect the end of a file.**

```
1: /* Detecting end-of-file. */
2: #include <stdlib.h>
3: #include <stdio.h>
4:
5: #define BUFSIZE 100
6:
7: main()
8: {
9:     char buf[BUFSIZE];
10:    char filename[60];
11:    FILE *fp;
12:
13:    puts("Enter name of text file to display: ");
14:    gets(filename);
15:
16:    /* Open the file for reading. */
17:    if ( (fp = fopen(filename, "r")) == NULL )
18:    {
19:        fprintf(stderr, "Error opening file.");
20:        exit(1);
21:    }
22:
23:    /* If end of file not reached, read a line and display it. */
24:
25:    while ( !feof(fp) )
26:    {
27:        fgets(buf, BUFSIZE, fp);
28:        printf("%s", buf);
29:    }
30:
31:    fclose(fp);
32:    return(0);
33: }
```

Enter name of text file to display:

**hello.c**

```
#include <stdio.h>
```

```
main()
{
    printf("Hello, world.");
    return(0);
}
```

**ANALYSIS:** The while loop in this program (lines 25 through 29) is typical of a while used in more complex programs that do sequential processing. As long as the end of the file hasn't been reached, the code within the while statement (lines 27 and 28) continues to execute repeatedly. When the call to `feof()` returns a nonzero value, the loop ends, the file is closed, and the program ends.

---

## 1.38 File Management Functions

The term *file management* refers to dealing with existing files--not reading from or writing to them, but deleting, renaming, and copying them. The C standard library contains functions for deleting and renaming files, and you can also write your own file-copying program.

### *Deleting a File*

To delete a file, you use the library function `remove()`. Its prototype is in `STDIO.H`, as follows:  
`int remove( const char *filename );`

The variable *\*filename* is a pointer to the name of the file to be deleted. (See the section on filenames earlier in this chapter.) The specified file must not be open. If the file exists, it is deleted (just as if you used the DEL command from the DOS prompt or the rm command in UNIX), and `remove()` returns 0. If the file doesn't exist, if it's read-only, if you don't have sufficient access rights, or if some other error occurs, `remove()` returns -1.

Listing 16.8 demonstrates the use of `remove()`. Be careful: If you `remove()` a file, it's gone forever.

#### **Listing 16.8. Using the `remove()` function to delete a disk file.**

```
1: /* Demonstrates the remove() function. */
2:
3: #include <stdio.h>
4:
5: main()
6: {
7:     char filename[80];
8:
9:     printf("Enter the filename to delete: ");
10:    gets(filename);
11:
12:    if ( remove(filename) == 0)
13:        printf("The file %s has been deleted.\n", filename);
14:    else
15:        fprintf(stderr, "Error deleting the file %s.\n", filename);
16:    return(0);
17: }
```

Enter the filename to delete: \*.bak

Error deleting the file \*.bak.

Enter the filename to delete: list1414.bak

The file list1414.bak has been deleted.

**ANALYSIS:** This program prompts the user on line 9 for the name of the file to be deleted. Line 12 then calls `remove()` to delete the entered file. If the return value is 0, the file was removed, and a message is displayed stating this fact. If the return value is not zero, an error occurred, and the file was not removed.

## Renaming a File

The `rename()` function changes the name of an existing disk file. The function prototype, in `STDIO.H`, is as follows:

```
int rename( const char *oldname, const char *newname );
```

The filenames pointed to by `oldname` and `newname` follow the rules given earlier in this chapter. The only restriction is that both names must refer to the same disk drive; you can't rename a file to a different disk drive. The function `rename()` returns 0 on success, or -1 if an error occurs. Errors can be caused by the following conditions (among others):

- The file `oldname` does not exist.
- A file with the name `newname` already exists.
- You try to rename to another disk.

Listing 16.9 demonstrates the use of `rename()`.

### Listing 16.9. Using `rename()` to change the name of a disk file.

```
1: /* Using rename() to change a filename. */
2:
3: #include <stdio.h>
4:
5: main()
6: {
7:     char oldname[80], newname[80];
8:
9:     printf("Enter current filename: ");
10:    gets(oldname);
11:    printf("Enter new name for file: ");
12:    gets(newname);
13:
14:    if ( rename( oldname, newname ) == 0 )
15:        printf("%s has been renamed %s.\n", oldname, newname);
16:    else
17:        fprintf(stderr, "An error has occurred renaming %s.\n", oldname);
18:    return(0);
19: }
```

Enter current filename: **list1609.c**

Enter new name for file: **rename.c**

list1609.c has been renamed rename.c.

**ANALYSIS:** Listing 16.9 shows how powerful C can be. With only 18 lines of code, this program replaces an operating system command, and it's a much friendlier function. Line 9 prompts for the name of the file to be renamed. Line 11 prompts for the new filename. The call to the `rename()` function is wrapped in an if statement on line 14. The if statement checks to ensure that the renaming of the file was carried out correctly. If so, line 15 prints an affirmative message; otherwise, line 17 prints a message stating that there was an error.

## Copying a File

It's frequently necessary to make a copy of a file--an exact duplicate with a different name (or with the same name but in a different drive or directory). In DOS, you do this with the `COPY` command, and other operating systems have equivalents. How do you copy a file in C? There's no library function, so you need to write your own.

This might sound a bit complicated, but it's really quite simple thanks to C's use of streams for input and output. Here are the steps you follow:

1. Open the source file for reading in binary mode. (Using binary mode ensures that the function can copy all sorts of files, not just text files.)

2. Open the destination file for writing in binary mode.
3. Read a character from the source file. Remember, when a file is first opened, the pointer is at the start of the file, so there's no need to position the file pointer explicitly.
4. If the function feof() indicates that you've reached the end of the source file, you're done and can close both files and return to the calling program.
5. If you haven't reached end-of-file, write the character to the destination file, and then loop back to step 3.

Listing 16.10 contains a function, copy\_file(), that is passed the names of the source and destination files and then performs the copy operation just as the preceding steps outlined. If there's an error opening either file, the function doesn't attempt the copy operation and returns -1 to the calling program. When the copy operation is complete, the program closes both files and returns 0.

**Listing 16.10. A function that copies a file.**

```
1: /* Copying a file. */
2:
3: #include <stdio.h>
4:
5: int file_copy( char *oldname, char *newname );
6:
7: main()
8: {
9:     char source[80], destination[80];
10:
11:     /* Get the source and destination names. */
12:
13:     printf("\nEnter source file: ");
14:     gets(source);
15:     printf("\nEnter destination file: ");
16:     gets(destination);
17:
18:     if ( file_copy( source, destination ) == 0 )
19:         puts("Copy operation successful");
20:     else
21:         fprintf(stderr, "Error during copy operation");
22:     return(0);
23: }
24: int file_copy( char *oldname, char *newname )
25: {
26:     FILE *fold, *fnew;
27:     int c;
28:
29:     /* Open the source file for reading in binary mode. */
30:
31:     if ( ( fold = fopen( oldname, "rb" ) ) == NULL )
32:         return -1;
33:
34:     /* Open the destination file for writing in binary mode. */
35:
36:     if ( ( fnew = fopen( newname, "wb" ) ) == NULL )
37:     {
38:         fclose ( fold );
39:         return -1;
40:     }
41:
42:     /* Read one byte at a time from the source; if end of file */
43:     /* has not been reached, write the byte to the */
```

```
44:  /* destination. */
45:
46:  while (1)
47:  {
48:      c = fgetc( fold );
49:
50:      if ( !feof( fold ) )
51:          fputc( c, fnew );
52:      else
53:          break;
54:  }
55:
56:  fclose ( fnew );
57:  fclose ( fold );
58:
59:  return 0;
60: }
```

Enter source file: **list1610.c**

Enter destination file: **tmpfile.c**

Copy operation successful

**ANALYSIS:** The function `copy_file()` works perfectly well, letting you copy anything from a small text file to a huge program file. It does have limitations, however. If the destination file already exists, the function deletes it without asking. A good programming exercise for you would be to modify `copy_file()` to check whether the destination file already exists, and then query the user as to whether the old file should be overwritten.

`main()` in Listing 16.10 should look very familiar. It's nearly identical to the `main()` in Listing 16.9, with the exception of line 14. Instead of `rename()`, this function uses `copy()`. Because C doesn't have a copy function, lines 24 through 60 create a copy function. Lines 31 and 32 open the source file, `fold`, in binary read mode. Lines 36 through 40 open the destination file, `fnew`, in binary write mode. Notice that line 38 closes the source file if there is an error opening the destination file. The while loop in lines 46 through 54 does the actual copying of the file. Line 48 gets a character from the source file, `fold`. Line 50 checks to see whether the end-of-file marker was read. If the end of the file has been reached, a break statement is executed in order to get out of the while loop. If the end of the file has not been reached, the character is written to the destination file, `fnew`. Lines 56 and 57 close the two files before returning to `main()`.

## 1.39 Using Temporary Files

Some programs make use of one or more temporary files during execution. A temporary file is a file that is created by the program, used for some purpose during program execution, and then deleted before the program terminates. When you create a temporary file, you don't really care what its name is, because it gets deleted. All that is necessary is that you use a name that isn't already in use for another file. The C standard library includes a function `tmpnam()` that creates a valid filename that doesn't conflict with any existing file. Its prototype in `STDIO.H` is as follows:

```
char *tmpnam(char *s);
```

The argument `s` must be a pointer to a buffer large enough to hold the filename. You can also pass a null pointer (`NULL`), in which case the temporary name is stored in a buffer internal to `tmpnam()`, and the function returns a pointer to that buffer. Listing 16.11 demonstrates both methods of using `tmpnam()` to create temporary filenames.

### Listing 16.11. Using `tmpnam()` to create temporary filenames.

```
1:  /* Demonstration of temporary filenames. */
2:
3:  #include <stdio.h>
4:
5:  main()
```

```
6: {  
7:   char buffer[10], *c;  
8:  
9:   /* Get a temporary name in the defined buffer. */  
10:  
11:   tmpnam(buffer);  
12:  
13:   /* Get another name, this time in the function's */  
14:   /* internal buffer. */  
15:  
16:   c = tmpnam(NULL);  
17:  
18:   /* Display the names. */  
19:  
20:   printf("Temporary name 1: %s", buffer);  
21:   printf("\nTemporary name 2: %s\n", c);  
22: }
```

Temporary name 1: TMP1.\$\$\$

Temporary name 2: TMP2.\$\$\$

**ANALYSIS:** The temporary names generated on your system will probably be different from these. This program only generates and prints the temporary names; it doesn't actually create any files. Line 11 stores a temporary name in the character array, buffer. Line 16 assigns the character pointer to the name returned by tmpnam() to c. Your program would have to use the generated name to open the temporary file and then delete the file before program execution terminates. The following code fragment illustrates:

```
char tempname[80];  
FILE *tmpfile;  
tmpnam(tempname);  
tmpfile = fopen(tempname, "w"); /* Use appropriate mode */  
fclose(tmpfile);  
remove(tempname);
```

---